

Bachelor Thesis

**Deep Graph-Based Anomaly Detection
on Image-Derived Representations**

Dehn Xu

May 19, 2025

Supervisors:

Prof. Dr. Emmanuel Müller

M. Sc. Tim Katzke

Technical University of Dortmund
Department of Computer Science
Chair of Data Science and Data Engineering
<https://ls9-www.cs.tu-dortmund.de/>

Contents

Mathematical Notations	vi
1 Introduction	1
1.1 Research Question and Contributions	2
1.2 Structure of this thesis	3
2 Theoretical Foundation and Background	4
2.1 Graph Theory	4
2.2 Artificial Neural Networks	6
2.3 Graph Representation Learning	12
2.4 Graph Anomaly Detection	17
3 Methodology	26
3.1 Image-to-Graph Transformation	26
3.2 Graph Anomaly Detection Framework	39
4 Evaluation	45
4.1 Experimental Setup	45
4.2 Results	55
4.3 Runtime Analysis	69
5 Conclusion and Outlook	72
A PR-ROC Values on Torchvision Datasets	76
B Results With More Labeled Anomalies	78
C Unsupervised Results Without Edge Loss	82
D Additional Figures	83
Bibliography	86

List of Figures

2.1	Graph	4
2.2	Comparison between biological neuron and perceptron	6
2.3	Comparison of Sigmoid and ReLU Activation Functions	7
2.4	Example of a Feedforward Neural Network	8
2.5	Training of a neural network	10
2.6	Example of two isomorphic graphs	13
2.7	Message passing in GNNs illustrated for one central node v	13
2.8	Non-isomorphic graphs for which 1-WL fails [21]	16
2.9	Different variants of anomalies encountered in GAD	18
2.10	A typical Autoencoder consisting of an Encoder and Decoder.	19
2.11	Example of deep graph-level anomaly detection	21
2.12	Confusion Matrix for Binary Classification	22
2.13	Comparison of ROC and PR curves for different classifier performances.	24
3.1	Methodology of preprocessing and GAD	26
3.2	Grid representation	27
3.3	Supapixel segmentation	28
3.4	Patch decomposition	30
3.5	Region adjacency graphs for SLIC and Patches	32
3.6	Visualization of GLCM	34
3.7	Architecture of OCGTL	40
3.8	Architecture of the Graph Autoencoder with Edge Features	42
4.1	Visualization of MNIST [59]	46
4.2	Visualization of Fashion-MNIST [61]	48
4.3	Visualization of CIFAR-10 [62]	50
4.4	Visualization of HAM10000 [63]	51
4.5	Visualization of virtual nodes used on HAM10000 [63]	52
4.6	Comparison of configurations trained on MNIST	58
4.7	Comparison of configurations trained on Fashion-MNIST	61
4.8	Comparison of configurations trained on CIFAR-10	64
4.9	Comparison of configurations trained on HAM10000	67
5.1	Anomaly localization on the MVTec dataset [67] with FCDD [68]	73

List of Figures

5.2	Example of an adversarial attack	74
-----	--	----

List of Tables

4.1	MNIST class distribution [59]	46
4.2	Graph metrics for the MNIST dataset	47
4.3	Fashion-MNIST class distribution [61]	48
4.4	Graph Metrics for the Fashion-MNIST dataset	49
4.5	CIFAR-10 class distribution [62]	49
4.6	Graph Metrics for the transformed CIFAR-10 dataset	50
4.7	HAM10000 class distribution [63]	51
4.8	Graph Metrics for the transformed HAM10000 dataset	52
4.9	HAM10000 features	53
4.10	Unsupervised MNIST ROC-AUC results	56
4.11	Semi-supervised MNIST ROC-AUC results	57
4.12	Unsupervised Fashion-MNIST ROC-AUC results	59
4.13	Semi-supervised Fashion-MNIST ROC-AUC results	60
4.14	Unsupervised CIFAR-10 ROC-AUC results	62
4.15	Semi-supervised CIFAR-10 ROC-AUC results	63
4.16	Unsupervised HAM10000 ROC-AUC results	65
4.17	Unsupervised HAM10000 PR-AUC results	65
4.18	Semi-supervised HAM10000 ROC-AUC results	68
4.19	Semi-supervised HAM10000 PR-AUC results	68
4.20	Runtime of preprocessing, training, and final evaluation for all configurations	69
4.21	Processing times of an image for each dataset	70

Mathematical Notations

Notation	Explanation
$\mathbb{N}$	Set of the natural numbers $1, 2, 3, \dots$
$\mathbb{R}$	Set of the real numbers
$\mathbb{R}^d$	Set of all d -dimensional vectors whose entries are real numbers
$\mathbb{R}^{m \times n}$	Set of all $m \times n$ matrices whose entries are real numbers
$\mathbb{R}^{l \times m \times n}$	Set of all $l \times m \times n$ tensors whose entries are real numbers
$\mathcal{M} = \{m_1, \dots, m_n\}$	Unordered set $\mathcal{M}$ of n elements m_i
$\mathcal{M} = \{\{m_1, \dots, m_n\}\}$	Unordered multiset $\mathcal{M}$ with potentially multiple occurrences of elements
$\mathcal{M} = (m_1, \dots, m_n)$	Ordered n-tuple $\mathcal{M}$ of n elements m_i
$ \mathcal{M} $	Cardinality or size of $\mathcal{M}$: $ \mathcal{M} = \{m_1, \dots, m_n\} = n$
$f: \mathcal{M} \rightarrow \mathcal{N}$	Function f that maps inputs of set $\mathcal{M}$ to outputs of set $\mathcal{N}$
$\mathcal{M} \times \mathcal{N}$	Cartesian product between $\mathcal{M}$ and $\mathcal{N}$: $\{(m_1, n_1), \dots, (m_1, n_{ \mathcal{N} }), \dots, (m_{ \mathcal{M} }, n_1), \dots, (m_{ \mathcal{M} }, n_{ \mathcal{N} })\}$
$\bigcup_{i=1}^n \mathcal{X}_i$	Union of all sets $\mathcal{X}_i$
$\bigcap_{i=1}^n \mathcal{X}_i$	Intersection of all sets $\mathcal{X}_i$
$\sum_{i=1}^n x_i$	Sum of x_i : $x_1 + x_2 + \dots + x_n$
$ x $	Absolute value of scalar x , e. g., $ -2.5 = 2.5 = 2.5$
$\lfloor x \rfloor$	Floor of x , e. g., $\lfloor 3.8 \rfloor = 3$
$\lceil x \rceil$	Ceiling of x , e. g., $\lceil 3.8 \rceil = 4$
$f'(x)$	The derivative of function $f: \mathbb{R} \rightarrow \mathbb{R}$ with respect to x
$\mathbf{v}$	Vector $\mathbf{v} = [v_1, \dots, v_n]^\top$ with n elements v_i
$\mathbf{u} \cdot \mathbf{v}$	Dot product between $\mathbf{u} \cdot \mathbf{v} = u_1 \cdot v_1 + u_2 \cdot v_2 + \dots + u_n \cdot v_n$
$\ \mathbf{v}\ $	L2-norm of $\mathbf{v}$: $\ \mathbf{v}\ = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$
$\mathbf{A}$	Matrix $\mathbf{A}$ with vectors $\mathbf{a}_i^\top$ and scalars $a_{i,j}$
$\mathbf{A}\mathbf{x}$	Matrix multiplication between matrix $\mathbf{A}$ and vector $\mathbf{x}$
$\ \mathbf{A}\ _F$	Frobenius norm of $\mathbf{A}$: $\ \mathbf{A}\ _F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{i,j} ^2}$ with $\mathbf{A} \in \mathbb{R}^{m \times n}$
$\frac{\partial f}{\partial x}$	The partial derivative of function $f: \mathbb{R}^d \rightarrow \mathbb{R}$ with respect to x
$\nabla_{\mathbf{x}} f$	The gradient vector of f with respect to $\mathbf{x}$: $\left[\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n}\right]^\top$

1 Introduction

In today’s digital age, automated data collection tools store tremendous amounts of data, such as system activity logs or observations of entities [1]. Within this vast amount of data, the detection of *anomalies* is a crucial task, often revealing useful information about abnormal system behavior. An anomaly, or outlier, originally defined as “an observation that differs so much from other observations as to arouse suspicion that a different mechanism generated it” [2], can be found across various domains, from defective equipment or malignant tumors to spam emails, cybersecurity intrusions, fraud, and more. This need for identifying anomalies has led to the development of many classic, shallow anomaly detection methods like k -nearest neighbor (k -NN) [3] and Local Outlier Factor (LOF) [4].

However, with the great advances in deep learning for many machine learning tasks in recent years, these methods began to reveal their limitations when applied to high-dimensional data, such as images. To perform effectively on high-dimensional data, they require extensive feature engineering or the adaptation of ensembles within feature subspaces. Consequently, researchers began to explore ways to learn relevant features automatically for anomaly detection through deep learning [5], [6]. Although these deep-learning approaches show promise, achieving the best results within a reasonable timeframe requires millions of trainable parameters and substantial computational resources. Moreover, these methods are primarily designed for structured Euclidean data, making them less suitable for handling unstructured data representations.

Unstructured data is commonly represented as graphs across many fields, where nodes represent entities and edges model their interactions. This representation is highly flexible, with several fundamental data structures being considered as instances of graphs. In certain cases, even data that does not naturally exhibit a graph-like structure, such as text, is transformed into graph representations [7]. Therefore, with the emergence of graph neural networks (GNN) [8], [9], researchers made major efforts to advance graph anomaly detection (GAD) methods. So far, according to the most recent independent surveys [10], [11], many unsupervised deep learning methods have been proposed—mostly at a node level. In contrast, graph-level anomaly detection, especially semi-supervised methods, as defined in [12], remains mostly under-explored.

Similar to text data, images can also be transformed into graph representations despite not possessing an inherent graph-like structure. One approach is the use of superpixels, which are relevant in computer vision for depth estimation, segmentation, and object localization [13]. Superpixels store information in sets of similar pixels instead of the fixed grid structure of images and can significantly accelerate subsequent processing, as the number of superpixels in an image typically ranges from 25 to 2500, compared to the hundreds of thousands of individual pixels in a standard image [14]. Therefore, using superpixel-based graphs to represent images reduces the input size, decreasing the computational cost of graph-based deep learning methods. Various approaches have been proposed to combine superpixel graph representations and graph classification [15]–[18], but their potential for anomaly detection, which typically requires few or no labels, has yet to be investigated.

In the real world, where the volume of data increases daily, procuring labeled data is not consistently possible or time-consuming and costly. Labeling may, for example, require the expertise of domain specialists, such as medical professionals or technical experts. As a result, several studies have repurposed datasets, originally for classification tasks, to be used for anomaly detection in an unsupervised manner. Typically, they used one class as a representative of normal data, with other classes as anomalies [1], [5], [6], [12], [19]. Validating the previously mentioned studies on graph classification of superpixel graphs with unsupervised and semi-supervised GAD approaches is therefore of significant interest. To the best of our knowledge, this work represents the first effort to evaluate the effectiveness of deep graph-level anomaly detection methods for identifying anomalous graphs within image-derived representations.

1.1 Research Question and Contributions

This thesis aims to address the challenge of detecting anomalous images by combining the data reduction benefits of graph representations with the powerful capabilities of deep graph-level outlier detection methods. The central research question guiding this work is:

Can graph-based representations of images, enhanced by deep graph anomaly detection techniques, improve the performance of anomaly detection compared to classical image-based methods?

To answer this question, a robust preprocessing pipeline is developed that transforms images into graph representations. An image-derived graph representation is a transformation of the input image into a set of nodes and edges that accurately represents the image’s structure and characteristics. Nodes in this graph can not only represent

individual pixels but also whole areas of the image. The characteristics of each node are encoded as features that aggregate the aspects of its region. Through the representation of images into graphs with aggregated features, the data’s dimensionality gets effectively reduced while preserving structural information inherent in the image. Subsequently, various graph-level anomaly detection methods, both unsupervised and semi-supervised, are evaluated on the obtained graph representations. These results are then compared to those obtained from classic deep anomaly detection techniques that are applied directly to the raw images to determine the effectiveness of graph-based methods in identifying anomalous images.

The thesis seeks to contribute new insights into the use of graph representations for anomaly detection on images by demonstrating that deep graph-level methods, when applied to intelligently constructed graph representations of images, can effectively detect anomalies and offer a viable alternative to conventional image-based techniques.

1.2 Structure of this thesis

The remainder of this thesis is outlined as follows. Chapter 2 explores the theoretical foundations of graphs as well as artificial neural networks and brings a comprehensive review of related work, including background on anomaly detection and graph neural networks. Based on this foundation, Chapter 3 describes the methodology behind our approach, focusing on the transformation of images into graphs and the framework for the detection of anomalous graphs. Subsequently, we present the applied benchmark datasets, the experimental setup, and the results in Chapter 4. This chapter compares different graph representations and their performance on graph-level anomaly detection methods against classic anomaly detection applied to raw images. Additionally, it provides an analysis of the runtime associated with image-to-graph preprocessing and model execution. Finally, Chapter 5 concludes the thesis by summarizing our findings and proposing directions for future research.

2 Theoretical Foundation and Background

In this chapter, we set up the foundations and background for graph anomaly detection. First, we establish the theoretical foundation of graph theory and neural networks. Second, we introduce the background of graph neural networks under the concept of graph representation learning. Finally, we delve into relevant works of graph anomaly detection, starting with the types of graph anomalies and a brief overview of deep learning approaches for non-graph data. We focus on recent developments in deep graph-level anomaly detection and discuss evaluation metrics specific to this domain.

2.1 Graph Theory

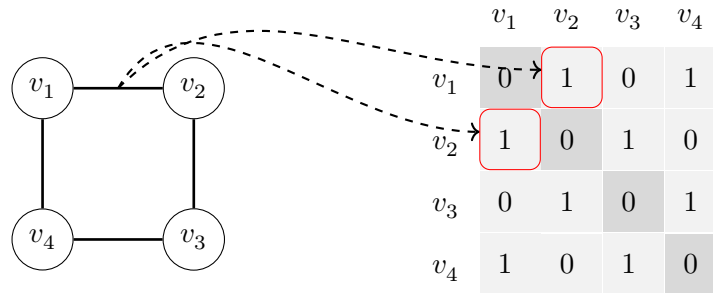


Figure 2.1: An undirected, unweighted graph and its corresponding (symmetric) adjacency matrix.

A graph $\mathcal{G}$ is an ordered pair $(\mathcal{V}, \mathcal{E})$ of $n = |\mathcal{V}|$ nodes (or vertices) forming the *node set* $\mathcal{V} = \{v_i\}_{i=1}^n = \{v_1, \dots, v_n\}$ that can be considered as abstract representations of entities together with the *edge set* $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ that show the relationships between aforementioned entities.

Two nodes $u, v \in \mathcal{V}$ are *adjacent* if $(u, v) \in \mathcal{E}$ or $(v, u) \in \mathcal{E}$ [20, p. 12]. The set of all adjacent nodes of u , also called the *neighborhood* of u is expressed as $\mathcal{N}(u) = \{v \in \mathcal{V} \mid (u, v) \in \mathcal{E}\}$. The size of a node's neighborhood, termed as the *degree* of a node v , denoted as $d_v = |\mathcal{N}(v)|$, represents the number of edges connected

to v . If a node v has $d_v = 0$, we call it an *isolated node*, and if $d_v = 1$ an *end-node* [20, p. 12]. A special case occurs when a node connects to itself through a *loop*, with $(v, v) \in \mathcal{E}$ [20, p. 3]. In this instance, the degree d_v is set to 2 to account for the handshaking lemma $\sum_{v \in \mathcal{V}} d_v = 2|\mathcal{E}|$ [20, p. 12].

To represent graphs on a computer, we often use an *adjacency matrix*, denoted as $\mathbf{A} \in \mathbb{R}_{\geq 0}^{|\mathcal{V}| \times |\mathcal{V}|}$ [21, p. 2]. Within this matrix, each row represents source nodes, while each column represents target nodes. Entries are expressed in binary form to indicate whether an edge is present or not:

$$\mathbf{A}_{i,j} = \begin{cases} 1, & \text{if } (v_i, v_j) \in \mathcal{E} \\ 0, & \text{otherwise.} \end{cases}$$

In an *undirected graph*, edges are inherently bidirectional, meaning that for vertices $v_i, v_j \in \mathcal{V} : \mathbf{A}_{i,j} = \mathbf{A}_{j,i}$. This bidirectional nature results in a symmetric adjacency matrix, where entries above and below the main diagonal mirror each other, as depicted in Figure 2.1. To avoid redundancy in sparse (zero-prevalent) matrices, edge lists, often in the Coordinate List (COO) format¹, offer an alternative by only storing existing edges. The COO format transforms a given $\mathbf{A} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ matrix into a $\mathbf{B} \in \mathbb{R}^{2 \times |\mathcal{E}|}$ matrix, where the first row corresponds to the source node indices, and the second row to the target node indices (both zero-indexed). For the adjacency matrix in Figure 2.1, the corresponding edge list (with redundant data values) in COO format is:

row _{idx}	0	0	1	1	2	2	3	3
col _{idx}	1	3	0	2	1	3	0	2

Often, the nodes within a graph carry additional attributes (or features) in addition to their connectivity. These attributes may be numerical, categorical, or even more complex data structures. An *attributed graph*, represented as $\mathcal{G} = (\mathcal{V}, \mathcal{E}, X)$, includes an attribute set $X = \{\mathbf{x}_v \in \mathbb{R}^d \mid v \in \mathcal{V}\}$, where d is the number of node features [21, p. 3]. In practise, this set is implemented as $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times d}$. Instances of node features can, for example, encompass age, gender, and location in a social network. Optionally, attributed graphs can have additional features associated with edges called *edge attributes* $E = \{\mathbf{e}_{uv} \in \mathbb{R}^k \mid (u, v) \in \mathcal{E}\}$. When $k = 1$, edge attributes are also termed *weights*. For weighted graphs, each entry $\mathbf{A}_{i,j}$ corresponds to the edge weight between nodes v_i and v_j . The weights can also be incorporated into the COO format by extending $\mathbf{B}$ to $\mathbb{R}^{3 \times |\mathcal{E}|}$ with the last row serving for the weights.

¹Implementation details at:

<https://docs.pytorch.org/docs/stable/sparse.html#sparse-coo-docs>

Edge attributes can be used to model costs, distances, the edge type, direction, or the relative coordinates from the source to the target node.

Throughout the remainder of this thesis, unless stated otherwise, graphs are assumed to be attributed, undirected, loop-free, and free of duplicate edges between any pair of vertices.

2.2 Artificial Neural Networks

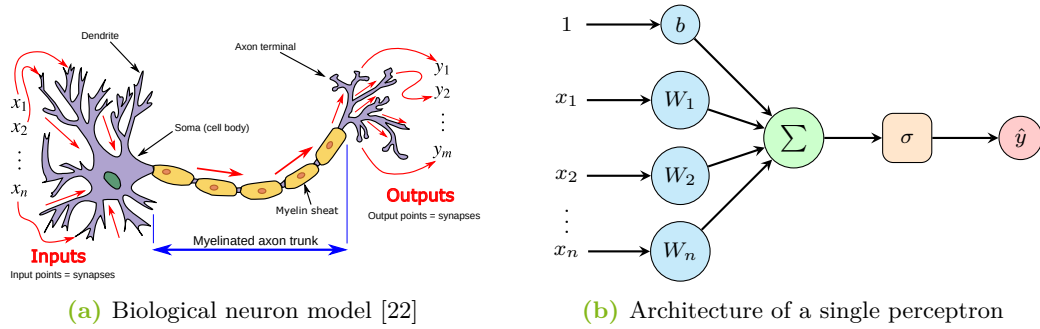


Figure 2.2: Comparison of a biological neuron (a) and single perceptron (b). The perceptron’s structure resembles that of a biological neuron.

Artificial Neural Networks (ANNs) are biologically inspired computational models composed of interconnected processing components, also called neurons [23, p. 1]. ANNs greatly abstract the biological brain architecture. However, they retain fundamental concepts of parallel processing and collective knowledge representation to solve challenging real-world issues [24].

To understand the biological inspiration of ANNs, consider how a neuron in the brain functions as stated in [25]: A biological neuron (see Figure 2.2a) “fires” or activates through an electrochemical process. Inputs, which are typically chemical neurotransmitters from other nerve cells, are received at the neuron’s dendrites and cell body. This causes small changes in the electrical potential of the neuron’s membrane. The potentials are summed and if and only if the resulting voltage surpasses a certain threshold, an “all-or-none” electrical signal known as an action potential is triggered. The action potential travels along the axon (myelin, when present, accelerates this journey) as an electrical signal. When the electrical impulse reaches the axon terminal, it triggers the release of chemical messengers called neurotransmitters into the synapse as outputs.

2.2.1 Perceptron

The fundamental building block of a neural network is the *perceptron*, introduced by Rosenblatt in 1958 [26]. Similarly to how biological neurons transmit and process information in an all-or-none principle i.e., by either staying inactive or activating completely depending on its inputs, Figure 2.2b illustrates how a perceptron emulates this behavior by computing a weighted sum of its inputs and then applying an activation function σ to produce an output. Mathematically, the output (or activation) $\hat{y}$ of a perceptron is expressed as:

$$\hat{y} = \sigma(\mathbf{W}^\top \mathbf{x} + b) = \sigma \left(\sum_{i=1}^n W_i x_i + b \right), \quad (2.1)$$

where:

- $\mathbf{x} = [x_1, x_2, \dots, x_n]^\top$ is the input vector,
- $\mathbf{W} = [W_1, W_2, \dots, W_n]^\top$ is the weight vector,
- and b is the bias term.

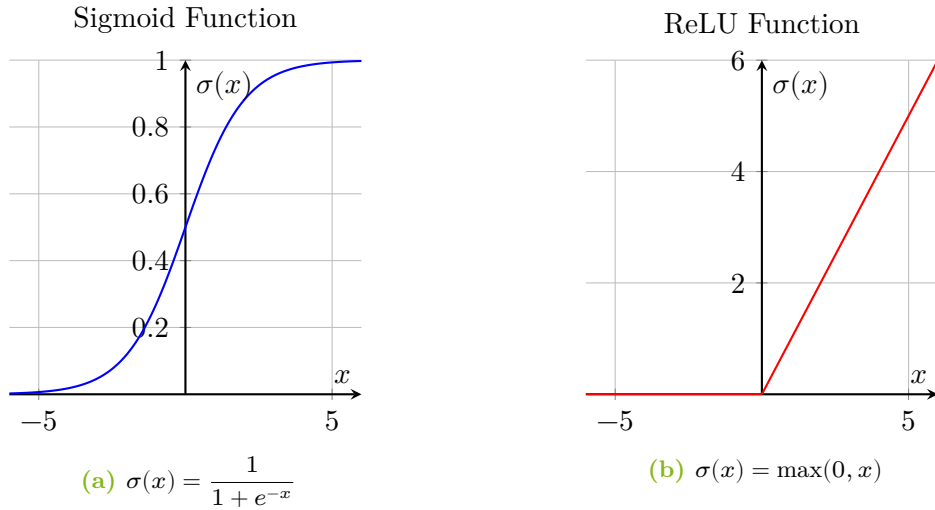


Figure 2.3: Comparison of Sigmoid and ReLU Activation Functions

In the historical context of single-layered perceptrons, the activation function is the *Heaviside step function*:

$$\sigma(x) = \begin{cases} 1, & x > 0 \\ 0, & \text{otherwise} \end{cases},$$

mirroring the all-or-none functionality of biological neurons. However, for neural networks discussed hereafter, learning relies on gradient-based backpropagation. Since the derivative of the Heaviside step function is zero (where defined), multiplying by this zero during the chain rule step of backpropagation prevents error signals from propagating to previous layers. Thus, prevalent activation functions are the *Sigmoid* and *ReLU* [27] functions, both depicted in Figure 2.3.

The Sigmoid function $\sigma(x) = 1/(1 + e^{-x})$ acts as a smooth approximation of the Heaviside step function that maps any real-valued number to $(0, 1)$, while the ReLU function $\sigma(x) = \max(0, x)$ (an indefinite integral of the Heaviside step function) is often favored for its efficient computational efficiency. Note that these functions are non-linear, whereas linear functions can be expressed with $\sigma(x) = mx + b$.

This distinction is meaningful when we try to model more complex data with multiple layers of perceptrons. As proven in [23, pp. 30–31], without non-linear transformations, any multilayer neural network would be equivalent to a linear model, regardless of its depth. Consequently, a neural network using only linear activation functions is incapable of learning unknown functions that produce linearly inseparable data.

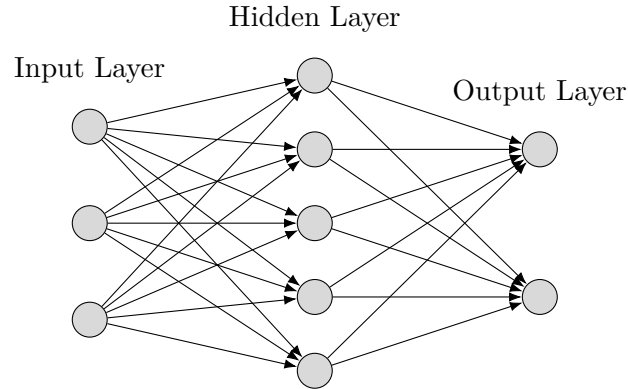


Figure 2.4: An example of a Feedforward Neural Network with three fully-connected layers

2.2.2 Multilayer perceptron

Stacking multiple perceptrons in layers forms a *multilayer perceptron (MLP)* (or feedforward neural network). The structure of an MLP (see Figure 2.4) consists of an input layer, one or more hidden layers, and an output layer. Drawing parallels with biological neural networks, where specific neurons respond to particular learned

patterns, MLPs develop internal representations of the data through their layers of artificial neurons.

By learning from data samples, MLPs approximate an unknown function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ that transforms input features into desired outputs. Notably, a relatively simple feedforward neural network can have powerful approximation capabilities. More precisely, Hornik et al. [28], [29], showed that a feedforward network, even with only a single hidden layer and a finite number of neurons, can approximate any real-valued function, subject to the condition that the activation function is non-constant, bounded, and continuous. This property is also known as the *Universal Approximation Theorem*.

2.2.3 Network Training

To fully utilize this approximation capability, there has to be an automated way to effectively set weights and biases of ANNs through training for any unknown function with only data samples. Given a set of N input-output pairs $S = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$, with $\mathbf{x}_i \in \mathbb{R}^n$ and $\mathbf{y}_i \in \mathbb{R}$, the *training objective* of a neural network is to find the optimal set of weights and biases, also called *trainable parameters* that produce outputs $\hat{\mathbf{y}}_i$ as close as possible to the target outputs $\mathbf{y}_i$. The difference between outputs is captured by a loss function $\mathcal{L}$, which the network seeks to *minimize* by adjusting its trainable parameters. A classic example of a loss function is the mean squared error (MSE):

$$\mathcal{L}_{\text{MSE}}(S) = \frac{1}{|S|} \sum_{(\mathbf{x}, \mathbf{y}) \in S} (\hat{\mathbf{y}}(\mathbf{x}) - \mathbf{y})^2. \quad (2.2)$$

Optimization

The optimization of the trainable parameters is generally performed using gradient-based methods, with the *backpropagation* algorithm [30] providing an efficient means of computing the necessary gradients. Backpropagation makes use of the chain rule to compute the gradient (or partial derivative) of the loss function with respect to each weight and bias in the network.

For training, the dataset S is split into $S_{\text{train}} \subset S$, $S_{\text{test}} = S \setminus S_{\text{train}}$ and converted into $k = \lceil |S_{\text{train}}|/m \rceil$ mini-batches:

$$S_{\text{train}} = \bigcup_{j=1}^k \mathcal{X}_j, \quad \text{where} \quad \mathcal{X}_j = \left\{ (\mathbf{x}_i, \mathbf{y}_i) \mid i = (j-1)m + 1, \dots, \min(jm, |S_{\text{train}}|) \right\}$$

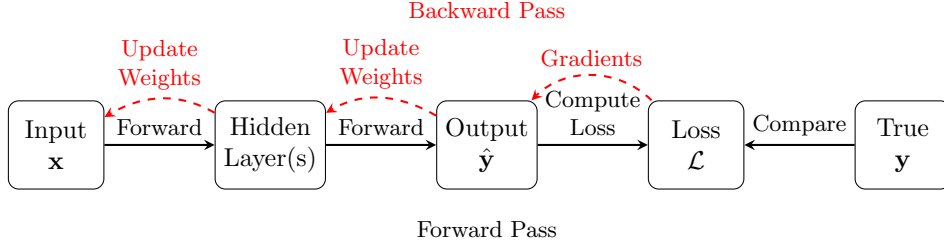


Figure 2.5: Training a neural network with a forward (black arrows) and backward pass (red dashed arrows)

and m is the *batch size*. Processing data in mini-batches not only reduces memory consumption but also introduces stochastic noise into the gradient updates, helping the optimization process overcome local minima and saddle points [31]. With the dataset divided into mini-batches, the training of neural networks progresses iteratively, as depicted in Figure 2.5. This iterative process consists of two main phases: the forward pass and the backward pass.

First, the inputs of the j -th batch, $\mathbf{x} \in \mathcal{X}_j$, are propagated through the network during the *forward pass* to calculate the output $\hat{\mathbf{y}}$. The activations of each neuron are stored to be used for backpropagation later.

In the *backward pass*, for reasons of computational efficiency, the *error* $\delta^{(l)}$, an auxiliary vector with the length of the number of neurons in layer l , is computed before the gradient is calculated. This error is then propagated backward through the network, layer by layer. In the output layer, the error for the i -th sample in the mini-batch is given by [32]:

$$\delta_i^{(L)} = \nabla_{\hat{\mathbf{y}}_i} \mathcal{L} \odot \sigma'(\mathbf{z}_i^{(L)}), \quad (2.3)$$

and in the other layers by:

$$\delta_i^{(l)} = (\mathbf{W}^{(l+1)})^\top \delta_i^{(l+1)} \odot \sigma'(\mathbf{z}_i^{(l)}), \quad (2.4)$$

where:

- $\nabla_{\hat{\mathbf{y}}_i} \mathcal{L}$ is the vector whose components are the partial derivatives of $\mathcal{L}$ with respect to the output activations $\hat{\mathbf{y}}_i$ of the i -th sample
- $\mathbf{W}^{(l+1)}$ is the weight matrix of the next layer (usually initialized randomly)
- $\odot$ symbolizes the element-wise multiplication

- $\sigma'(\mathbf{z}_i^{(l)})$ is the derivative of the activation function σ evaluated on the pre-activation inputs $\mathbf{z}_i^{(l)}$ of layer l

The gradients with respect to the weights and biases are then calculated as the sum over the mini-batch with batch size m :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \sum_{i=1}^m \delta_i^{(l)} \left(\sigma \left(\mathbf{z}_i^{(l-1)} \right) \right)^\top, \quad (2.5)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \sum_{i=1}^m \delta_i^{(l)}. \quad (2.6)$$

Only after calculating the average gradients across all samples in the mini-batch are the weights and biases of the network updated through an optimizer such as *Stochastic Gradient Descent (SGD)*:

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}, \quad (2.7)$$

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}}, \quad (2.8)$$

where η is a user-adjustable parameter (or hyperparameter) called the learning rate.

Completing one forward and backward pass for all samples in the training dataset is called an *epoch*. Since the dataset is divided into k mini-batches, one epoch executes k iterations of gradient descent. Typically, training of neural networks iterates for multiple epochs to allow the model to find more underlying patterns in the data.

Regularization

However, training a neural network for too many epochs without validation against unseen data can cause a phenomenon known as *overfitting*. Overfitting arises when the model learns not only the patterns of the training data but also the present noise and outliers. Consequently, an overfitted model performs well on the training data but generalizes poorly to new data in S_{test} .

To combat this, many machine learning approaches employ regularization techniques. The most intuitive one is called *early stopping*. In early stopping, we monitor the minimal validation loss throughout training using an additional validation dataset of unseen data. The model is only permitted to make predictions on this data without gradient computation to maintain the data's unseen status. If the validation loss doesn't improve after a set number of epochs, the model halts training early before completing all scheduled epochs. Another technique is called *L2 regularization*, also

widely known as weight decay in machine learning. In simple terms, to prevent the model from becoming overconfident in patterns that may only be present in the training dataset, it adjusts the loss function to penalize the weights such that they become less complex. Let $\mathcal{L}_\theta$ be the loss function with trainable weights $\mathbf{W}$ and biases $\mathbf{b}$ of a neural network with L layers, the adjusted loss is defined as:

$$\tilde{\mathcal{L}}_\theta = \mathcal{L}_\theta + \frac{\lambda}{2} \sum_{l=1}^L \|\mathbf{W}^{(l)}\|_F^2, \quad (2.9)$$

and the weights' update then:

$$\begin{aligned} \mathbf{W}^{(l)} &\leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial \tilde{\mathcal{L}}}{\partial \mathbf{W}^{(l)}} \\ \Leftrightarrow \mathbf{W}^{(l)} &\leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} - \eta \lambda \|\mathbf{W}^{(l)}\|_F, \end{aligned} \quad (2.10)$$

where λ is a hyperparameter to adjust the level of regularization. For brevity, all deep learning models discussed hereafter include the additional weight decay term in their loss functions.

2.3 Graph Representation Learning

Graphs are data structures that model relationships and interactions between entities over a wide range of domains, e.g., social networks, biological systems, and transportation networks, abstractly and uniformly. Consequently, developing methods to analyze and learn from graph data has become a key area within machine learning. However, traditional machine learning algorithms are limited by the fact that they require carefully handcrafted features, e.g., node degrees, clustering coefficients, or spectral properties, to incorporate the non-Euclidean graph structure and node attributes. To address this, *graph representation learning* (GRL) [21] has emerged as a powerful approach. GRL methods automatically transform node features into a low-dimensional vector space, known as *feature space*, often referred to as embedding space or latent space, while preserving their inherent structural and attribute information.

Consider an attributed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, X)$ with node features $X = \{\mathbf{x}_v \in \mathbb{R}^d \mid v \in \mathcal{V}\}$. The goal of GRL is to learn a mapping function $f : \mathcal{G} \rightarrow \mathbb{R}^{|\mathcal{V}| \times m}$ that embeds each node v into a m -dimensional vector $\mathbf{h}_v$. The learned embeddings should capture patterns from both the graph structure and the attribute information of the nodes, assisting the progress of downstream tasks such as classification or anomaly detection of nodes, edges, and graphs.

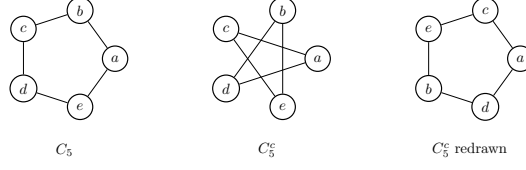


Figure 2.6: Example: Visualization of two isomorphic graphs C_5 and its complement C_5^c . The redrawn C_5^c exhibits the same structure as C_5 while preserving the adjacency of C_5^c [33].

An important consideration in GRL is the notion of *graph isomorphism*. Two graphs $\mathcal{G}$ and $\mathcal{G}'$ are said to be isomorphic if there exists a bijection between their node sets that preserves adjacency relationships [34]. This property is visually exemplified in Figure 2.6. In the context of representation learning, an ideal learned graph representation mapping f would satisfy the condition $f(\mathcal{G}_1) = f(\mathcal{G}_2)$ if and only if $\mathcal{G}_1$ and $\mathcal{G}_2$ are isomorphic [21, p. 95]. However, while graph isomorphism has not been proven to be NP-complete, there is also no currently known general polynomial-time algorithm.

2.3.1 Graph Neural Networks and the Message Passing Framework

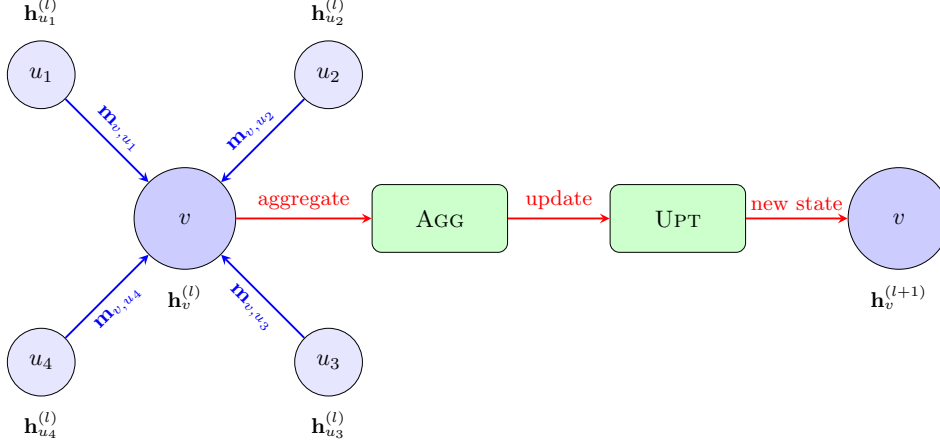


Figure 2.7: Message passing in GNNs illustrated for one central node v .

Neural networks whose input are graphs are called *Graph Neural Networks (GNNs)*. A key element in GNNs is how node representations are iteratively updated via the exchange of information between neighbors. Gilmer et al. [35] were the first to formalize the *message passing* framework illustrated in Figure 2.7. This framework

2 Theoretical Foundation and Background

provides a general template for designing core components of GNN architectures known as *graph convolutions*.

Consider $\mathbf{h}_u^{(l)}$ to be the representation of node u at the l -th layer of the GNN. According to the formulation presented in [36, pp. 71–72], the message passing framework follows these subsequent steps:

At layer $l = 0$, the node representations are initialized using the node features:

$$\mathbf{h}_v^{(0)} = \mathbf{x}_v, \quad \forall v \in \mathcal{V}. \quad (2.11)$$

For each layer $l = 1, 2, \dots, L$, the node representations are updated by aggregating messages from their neighbors. This process involves three functions: message computation (MSG), aggregation (AGG), and update (UPT).

Messages are computed for each edge $(v, u) \in \mathcal{E}$ using the representations from the previous layer:

$$\mathbf{m}_{v,u}^{(l)} = \text{MSG} \left(\mathbf{h}_v^{(l-1)}, \mathbf{h}_u^{(l-1)} \right). \quad (2.12)$$

The message function MSG defines how information is exchanged between neighboring nodes e.g., through a neural network.

Each node v aggregates the messages from its neighbors using an aggregation function AGG:

$$\mathbf{a}_v^{(l)} = \text{AGG} \left(\left\{ \left\{ \mathbf{m}_{v,u}^{(l)} \mid u \in \mathcal{N}(v) \right\} \right\} \right), \quad (2.13)$$

where $\mathcal{N}(v)$ is the set of neighbors of node v and $\{\{\cdot\}\}$ denotes a *multiset*, in which duplicate elements may also appear. The aggregation function must be permutation-invariant to ensure that the node representation is independent of the ordering of its neighbors [37].

The node representations are then updated using the aggregated messages:

$$\mathbf{h}_v^{(l)} = \text{UPT} \left(\mathbf{h}_v^{(l-1)}, \mathbf{a}_v^{(l)} \right). \quad (2.14)$$

The update function UPT combines the previous node representation with the aggregated information to produce the updated representation.

Different graph convolutions implement the MSG, AGG, and UPT functions in various ways, leading to models with distinct properties and capabilities. Notable examples include *Graph Convolutional Networks (GCNs)*, *Graph Attention Networks (GATs)*, and *Graph Isomorphism Networks (GINs)*.

GCNs, introduced by Kipf and Welling [38], generalize the convolution operation to graph-structured data. The update rule for the node representations in GCNs is determined by:

$$\mathbf{h}_v^{(l)} = \sigma \left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{d_v d_u}} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)} \right), \quad (2.15)$$

where σ is an activation function (e.g., ReLU), d_v is the degree of node v , and $\mathbf{W}^{(l)}$ is the learnable weight matrix for layer l . Including the self-loop (v in the summation) allows nodes to retain their features during the update. The normalization term $1/\sqrt{d_v d_u}$ weighs the influence of neighboring nodes based on their connectivity. Intuitively, a neighboring node $u \in \mathcal{N}(v)$ with abundant edges, e.g., a celebrity in a social network, passes its message to its many neighbors in $\mathcal{N}(u)$, making the information less specific for node v .

GATs [39] introduce an attention mechanism to dynamically learn the relative importance of neighboring nodes. Contrary to GCN's static normalization based on node degrees, GAT computes attention coefficients α_{vu} that determine how much node v should attend to node u 's features:

$$\mathbf{h}_v^{(l)} = \sigma \left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \alpha_{vu}^{(l)} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)} \right), \quad (2.16)$$

where the attention coefficients α_{vu} are computed using a learnable attention mechanism:

$$\alpha_{vu}^{(l)} = \frac{\exp \left(\text{LeakyReLU} \left(\mathbf{a}^{(l)\top} [\mathbf{W}^{(l)} \mathbf{h}_v \| \mathbf{W}^{(l)} \mathbf{h}_u] \right) \right)}{\sum_{k \in \mathcal{N}(v) \cup \{v\}} \exp \left(\text{LeakyReLU} \left(\mathbf{a}^{(l)\top} [\mathbf{W}^{(l)} \mathbf{h}_v \| \mathbf{W}^{(l)} \mathbf{h}_k] \right) \right)}. \quad (2.17)$$

Here, $\|$ denotes vector concatenation, $\mathbf{a}$ is the weight vector learned through a single-layered neural network, and the softmax $\sigma(x)_i = e^{x_i} / \sum_{j=1}^K e^{x_j}$ ensures that attention coefficients sum to 1 across all neighbors. The key intuition behind GATs is to enable each node to focus on the most relevant parts of its neighborhood by assigning different importance to different neighbors. In GCNs, the influence of each neighbor is statically normalized based on node degrees, assuming uniform importance among neighbors after adjusting for connectivity. However, in many real-world graphs, not all neighbors are equally important to a node's representation.

GINs [37] intent to achieve maximum expressive power (more in Chapter 2.3.2) among GNNs by closely approximating the *Weisfeiler-Lehman (WL) graph isomorphism test* [40]. The update rule in GINs is:

$$\mathbf{h}_v^{(l)} = \text{MLP}^{(l)} \left((1 + \epsilon^{(l)}) \mathbf{h}_v^{(l-1)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(l-1)} \right), \quad (2.18)$$

where $\text{MLP}^{(l)}$ is a multilayer perceptron, and $\epsilon^{(l)}$ is a learnable parameter or a fixed scalar.

2.3.2 Expressive Power and Graph Isomorphism

The effectiveness of GNNs in various applications is closely tied to their ability to capture and distinguish complex graph structures. This capability, often referred to as the *expressive power* of a GNN, determines how well the network can represent different graphs and their substructures through learned embeddings. A study by Xu et al. [37] analyzes this representational capacity of GNNs. They established that the design choices of the *aggregation* and *update* functions within the message passing framework significantly affect a GNN’s ability to capture graph structures. Specifically, they showed that GNNs employing *injective* (one-to-one) aggregation functions over multisets, such as the sum aggregator function (i. e., adding up all received messages), combined with injective update functions, can achieve maximum discriminative power among message passing neural networks. Under these conditions, GNNs become as expressive as the WL graph isomorphism test, a well-known algorithm for determining whether two graphs are isomorphic.

The 1-dimensional Weisfeiler-Lehman (1-WL) test, as described in [37], iteratively refines node labels through two steps: First, each node’s neighborhood labels, usually initialized with node degrees, are collected in a multiset and aggregated to the node’s label. Second, a hashing operation assigns unique new labels to these aggregations. During any iteration, if the multisets of node labels differ between two graphs, they are deemed non-isomorphic. The process terminates when the labels stabilize or after a fixed number of iterations. The parallel between the WL test and GNNs arises from the message-passing mechanism in GNNs, which mirrors the iterative neighborhood aggregation in the WL test.



Figure 2.8: An example of two non-isomorphic graphs for which the 1-WL test fails [21, p. 97].

However, the 1-WL test does not distinguish all non-isomorphic graphs; it fails for specific graph structures. Consider the two graphs illustrated in Figure 2.8: a cycle with six nodes ($\mathcal{G}_1 = C_6$) and two disjoint cycles with three nodes each ($\mathcal{G}_2 = 2C_3$). Although these graphs are structurally different, the 1-WL test cannot distinguish them. This inability is because all nodes in both graphs have degree 2,

and this pattern remains unchanged after applying the algorithm (assuming the hash function generates the same hash value for the same inputs).

2.3.3 Node-Level and Graph-Level Representations

For *node-level tasks*, such as node classification or anomaly detection, the final node representations $\mathbf{h}_v^{(L)}$ are used directly, as they capture the structural properties and attribute information relevant to each node.

Whereas *graph-level tasks* require a single representation for the entire graph, achieved through a *readout* function that aggregates the node representations:

$$\mathbf{h}_{\mathcal{G}} = \text{READOUT} \left(\left\{ \mathbf{h}_v^{(L)} \mid v \in \mathcal{V} \right\} \right). \quad (2.19)$$

Commonly, readout functions combine the use of an MLP with one of the following pooling functions:

$$\text{Sum Pooling: } \mathbf{h}_{\mathcal{G}} = \sum_{v \in \mathcal{V}} \mathbf{h}_v^{(L)}, \quad (2.20)$$

$$\text{Mean Pooling: } \mathbf{h}_{\mathcal{G}} = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \mathbf{h}_v^{(L)}, \quad (2.21)$$

$$\text{Max Pooling: } \mathbf{h}_{\mathcal{G}} = \max_{v \in \mathcal{V}} \left(\mathbf{h}_v^{(L)} \right). \quad (2.22)$$

These pooling functions generate permutation-invariant graph representations that preserve the essential structural and feature-based characteristics of the graph.

2.4 Graph Anomaly Detection

Graph anomaly detection (GAD) aims to identify anomalous nodes or edges within a single graph or anomalous graphs among a collection of graphs [10]. This task is closely related to classification, except that one class (the anomalies) is significantly underrepresented compared to the normal class.

One of the primary challenges in anomaly detection compared to classification, especially with graph data, is the scarcity of labeled anomalies. Labels are typically challenging to obtain because anomalies are rare and diverse, and labeling them requires significant cost, effort, and domain expertise (e.g., medicinal images, defective equipment) [12]. Therefore, contrary to supervised classification, anomaly detection is developed with unsupervised and semi-supervised methods that do not heavily rely on labeled data.

2.4.1 Graph Anomalies

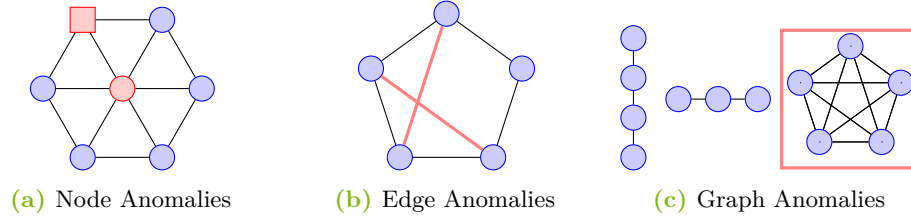


Figure 2.9: Different variants of anomalies encountered in GAD. The square node in (a) exhibits abnormal attributes, while the central node displays structural anomalies.

As presented in Figure 2.9, graph anomalies can manifest at node-, edge-, and graph-level.

An anomalous node [10] exhibits abnormal attributes or structural properties compared to the rest of the nodes in the graph. As an instance of abnormal attributes, a node of a transaction network might show transactional volumes several orders of magnitude larger than the network’s average. While structurally, a node might have an unexpected number of connections, e.g., a hub node in a network where most nodes have few connections, or an isolated node in a highly connected network, indicating a potential anomaly.

Anomalous edges (or links) [10] show irregularities in the connections between two otherwise unrelated nodes. Those unusual relationships can manifest as a connection between two nodes in a transaction network, where one is located overseas, which might indicate suspicious activity between a benign user and a fraudster. Alternatively, the absence of an expected connection can also be anomalous, such as missing links in a transportation network indicating potential disruptions.

Graph anomalies [10] deviate from the normal patterns observed within a dataset. As shown in Figure 2.9c, in a scenario where normal graphs may be sparse with nodes having few connections, an anomalous graph might be fully connected, displaying an unusual level of density. For instance, in a dataset of molecular compounds, anomalies could represent molecules that show activity against HIV.

2.4.2 Deep Anomaly Detection

The purpose of the Autoencoder or Autoassociative Network introduced in 1987 by [41], [42] as shown in Figure 2.10 was not primarily for anomaly detection after its discovery, but rather for image compression [43], dimensionality reduction and

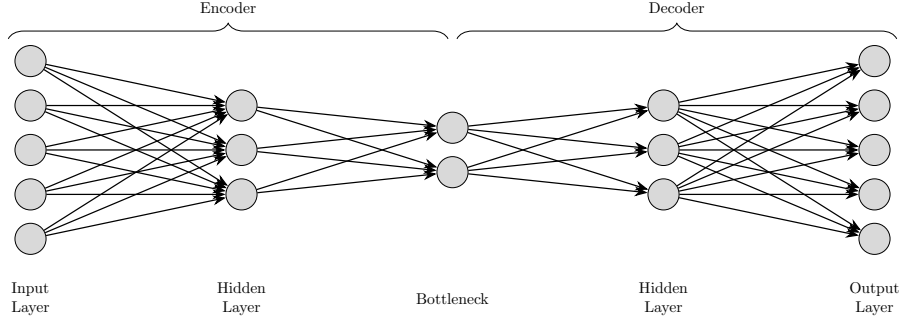


Figure 2.10: A typical Autoencoder consisting of an Encoder and Decoder.

feature extraction [44]. Only in 1995, eight years after its release, was its potential for novelty detection applied in [45]. An Autoencoder $\phi(\cdot; \theta) = \phi_D(\phi_E(\cdot; \theta_E); \theta_D)$ with parameters θ consists of an Encoder $\phi_E(\cdot; \theta_E)$, that transforms the original data into a compressed, low-dimensional representation and a Decoder $\phi_D(\cdot; \theta_D)$, that reconstructs the data based on the representation. To optimize these components i. e., transforming data to optimal encodings, the reconstruction loss between the original data $\mathbf{x} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ and the reconstructed output $\hat{\mathbf{x}} = \phi(\mathbf{x}; \theta)$ is minimized during training e. g., through the mean squared error (MSE):

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \|\phi(\mathbf{x}_i; \theta) - \mathbf{x}_i\|^2. \quad (2.23)$$

The main idea of autoencoders in the context of anomaly detection is that abundant normal data and rare anomalies within the training dataset lead to a lower reconstruction error for normal data and a higher reconstruction error for anomalies.

Ruff et al. [5] proposed the first deep one-class classification-based anomaly detection method with Deep SVDD. Inspired by the Autoencoder and Support Vector Data Descriptor (SVDD) [46], their model automatically extracts features into embeddings and maps them to a hypersphere. Given a set of normal data samples $\mathbf{x} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$, the objective is to learn a neural network $\phi(\cdot; \theta)$ with parameters θ that minimizes:

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \|\phi(\mathbf{x}_i; \theta) - \mathbf{c}\|^2, \quad (2.24)$$

where $\mathbf{c}$ is the predefined center of the hypersphere in the feature space.

The objective $\min_{\theta} \|\phi(\mathbf{x}; \theta) - \mathbf{c}\|^2$ is trivially solved by mapping all inputs to a single point $\phi(\mathbf{x}; \theta) = \mathbf{c}$. To prevent this phenomenon called *hypersphere collapse*, the

authors proposed several constraints in their work [5]:

1. Fix the hypersphere center and disallow the network to train it.
2. The center must avoid the initialization $c_0 = \phi(\mathbf{x}; \mathcal{W}_0)$, with $\mathcal{W}_0$ being the set of all-zero weights.
3. Remove bias terms in the network to prevent learning constant functions.
4. Do not use bounded activation functions, as they could otherwise emulate the bias term.

Building upon Deep SVDD, Ruff et al. [6] developed Deep SAD to incorporate labeled data in a semi-supervised setting. This extension addresses scenarios with a small amount of labeled anomaly data, appearing frequently in many practical applications e.g., medical imaging. These situations allow experts to only label some cases, with exhaustive labeling being impractical. Assume we have n unlabeled $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ and m labeled observations $\{(\tilde{\mathbf{x}}_1, \tilde{y}_1), \dots, (\tilde{\mathbf{x}}_m, \tilde{y}_m)\}$, where $\tilde{y} = +1$ indicate labeled normal samples and $\tilde{y} = -1$ labeled anomalies [6]. The Deep SAD objective is as follows:

$$\min_{\theta} \quad \frac{1}{n+m} \left(\sum_{i=1}^n \|\phi(\mathbf{x}_i; \theta) - \mathbf{c}\|^2 + \eta \sum_{j=1}^m (\|\phi(\tilde{\mathbf{x}}_j; \theta) - \mathbf{c}\|^2)^{\tilde{y}_j} \right). \quad (2.25)$$

Here, $\eta > 0$ is not the learning rate, but a hyperparameter that controls the balance between the labeled and unlabeled data. The exponent $\tilde{y}_j$ takes the value $+1$ for normal samples, emphasizing minimization of the distance to the center (equivalent to unlabeled data with $\eta = 1$) and -1 for anomalies, thereby inverting small distances and encouraging mappings away from the center. This loss function allows the model to leverage labeled anomalies by pushing their embeddings away from the normal data concentration, improving detection performance. Deep SAD also mitigates the hypersphere collapse issue through the balanced contribution of labeled and unlabeled objectives.

2.4.3 Deep Graph-level Anomaly Detection

Historically, GAD has involved a two-stage method: Converting graph data into tabular form and then applying traditional anomaly detection techniques [47]. Specifically, this approach first extracts handcrafted features from the graph and represents the graph or its components as feature vectors. The vector representations

are then analyzed with techniques such as Local Outlier Factor (LOF) [4], One-Class Support Vector Machines (OC-SVM) [48], or SVDD. However, this strategy suffers from limited representation power of non-linear properties and scalability challenges when applied to large or complex graphs [10]. Furthermore, while this method facilitates node-level anomaly detection, it does not inherently support anomaly detection at the graph level due to node-level features. The success of deep learning

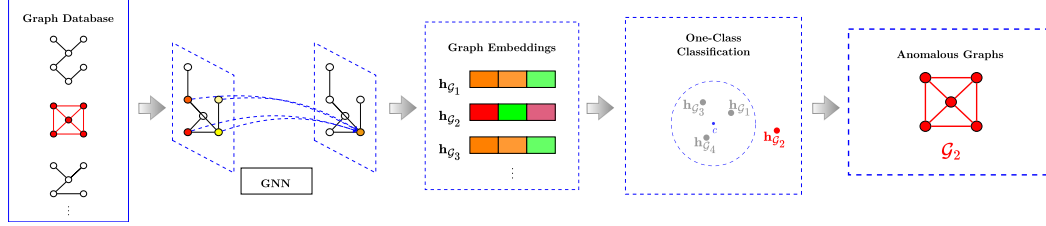


Figure 2.11: Example of deep graph-level anomaly detection. The message passing-based GNN aggregates features from neighboring nodes to create node embeddings, as shown in Equation 2.14. Graph Embeddings are generated through a readout function (see Equation 2.19) and subsequently mapped to a hypersphere. Embeddings further away from the center $\mathbf{c}$ are detected as anomalies.

and GNNs through GRL have led to the development of specialized graph-level anomaly detection methods. These methods use the message passing framework of GNNs as a basis while incorporating principles from one-class classification as illustrated in Figure 2.11.

Zhao and Akoglu [1] introduced *One-Class GIN (OCGIN)*, which combines the Graph Isomorphism Network with the Deep SVDD objective. In OCGIN, each graph $\mathcal{G}_i$ is transformed to an embedded vector representation $\mathbf{h}_{\mathcal{G}_i}$ with mean pooling as the READOUT function and $\mathbf{h}_v^{(l)}$, the node embedding of layer l , is calculated as in Equation 2.18:

$$\mathbf{h}_{\mathcal{G}_i} = \text{GIN}(\mathcal{G}_i; \theta) = \text{CONCAT} \left(\text{READOUT} \left(\left\{ \mathbf{h}_v^{(l)} \mid v \in \mathcal{V}_i \right\} \right) \mid l = 0, 1, \dots, L \right). \quad (2.26)$$

The objective function mirrors Deep SVDD of Equation 2.24:

$$\min_{\theta} \quad \frac{1}{n} \sum_{i=1}^n \|\mathbf{h}_{\mathcal{G}_i} - \mathbf{c}\|^2. \quad (2.27)$$

However, OCGIN and several other graph-level anomaly detectors using the two-stage method, were found to suffer from a “performance flip” issue [1] in classification datasets, effectively performing worse than random guessing in at least one One vs. Rest setting. Consequently, Qiu et al. [19] proposed *One-Class Graph Transformation Learning (OCGTL)*, extending OCGIN with self-supervised contrastive

learning, thereby overcoming its issue. Further details on OCGTL are provided in Chapter 3.2.

2.4.4 Evaluation Metrics for Anomaly Detection

Actual	Positive	Negative
	True Positive (TP)	False Negative (FN)
Negative	False Positive (FP)	True Negative (TN)
		Predicted
		Positive Negative

Figure 2.12: Confusion Matrix for Binary Classification

Evaluating machine learning models is one of the most important parts of determining their effectiveness, yet it is particularly challenging within the scope of anomaly detection. The inherent imbalance of these datasets, in which anomalies are scarce relative to normal instances, complicates the assessment of model performance. Since some evaluation metrics may be misleading when applied directly to imbalanced data, only a careful selection of appropriate measures is used.

At the center of model evaluation lies the *confusion matrix*, a tool that succinctly summarizes the performance of a classification model by tabulating actual versus predicted classifications (see Figure 2.12). The confusion matrix covers four fundamental components:

- **True Positives (TP):** Anomalies correctly identified by the model.
- **True Negatives (TN):** Normal instances correctly identified as such.
- **False Positives (FP):** Normal instances incorrectly flagged as anomalies (also known as Type I errors).

- **False Negatives (FN):** Anomalies that the model failed to detect (Type II errors).

A conventional approach might suggest using *accuracy* as the primary metric, which quantifies the proportion of correct classifications across all instances:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}. \quad (2.28)$$

However, with an imbalance of data, a model that predicts every instance as normal could achieve high accuracy despite failing to identify any anomalies. In light of this problem, we turn to metrics that provide a more nuanced evaluation. Both *Precision* and *Recall* are two such metrics that offer deeper insights into positive (anomalous) samples:

Precision (also known as Positive Predictive Value) answers the question: “Of all the instances predicted as anomalies, how many were actual anomalies?”

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}. \quad (2.29)$$

As precision does not account for false negatives, Recall (also known as True Positive Rate) addresses the question: “Of all the actual anomalies, how many did the model detect?”

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \quad (2.30)$$

Note that in edge cases with a zero in the denominator, such as when all samples are predicted as anomalies ($\text{TP} + \text{FP} = 0$) or when no anomalies are present in the dataset ($\text{TP} + \text{FN} = 0$), we define $\text{Precision} = 1$ or $\text{Recall} = 1$.

The combined metric of precision and recall, also known as the *F1 score*, is then defined as follows:

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (2.31)$$

A common theme in anomaly detection is using *anomaly scores* to determine whether a data sample is anomalous. We typically utilize the same or slightly modified loss function from training to obtain these scores. Since we train the model mostly on normal data, unseen normal samples tend to have a lower anomaly score than anomalous ones. The main idea is to set the threshold of anomaly scores such that the predictions are tuned per use case. For this purpose, we employ curves such as the *Receiver Operating Characteristic (ROC) curve* and the *Precision-Recall (PR) curve* illustrated in Figure 2.13 to better visualize and compare model performance across different thresholds.

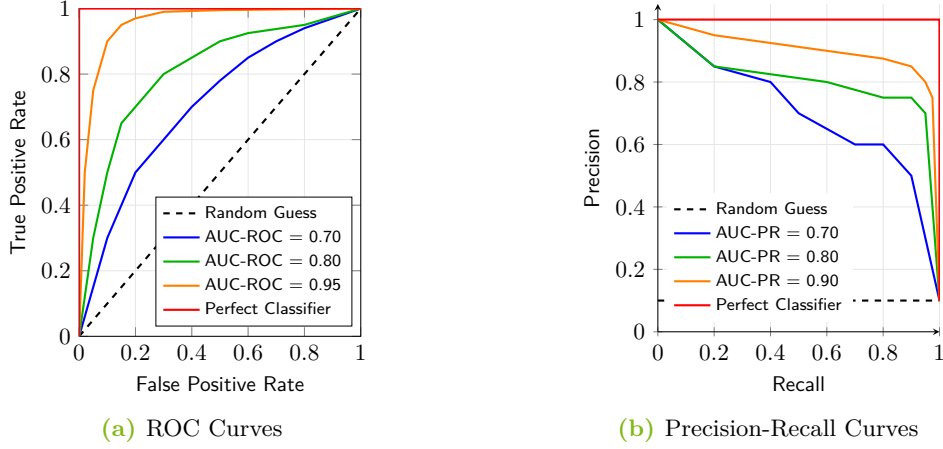


Figure 2.13: Comparison of (a) ROC and (b) PR curves for different classifier performances. The baseline in (b) is equal to the ratio of positive (anomalous) samples ($= 0.1$) in the dataset.

Let the False Positive Rate, $FPR = FP / (FP + TN)$, denote the rate of instances mistakenly detected as anomalies, the ROC curve then visualizes the trade-off between recall (TPR) and FPR. For anomaly scores ranging from $[0, 1]$, setting the detection threshold above 1 yields both TPR and FPR of 0, as the model classifies no instances as anomalies. Conversely, setting the threshold at or below 0 results in both TPR and FPR of 1, as the model classifies all instances as anomalies. The other points on the curve vary depending on the anomaly scores and their labels. We calculate the *Area Under the Curve (AUC)* to gain a summarizing scalar value from such curves. In this case, the Area under the ROC Curve (ROC-AUC) tells us the model's ability to distinguish between normal and anomalous instances.

However, in the context of highly imbalanced datasets, the values of the ROC curve can be misleading. A model that consistently assigns a score of 0 results in many true negatives, keeping the FPR low, even if it fails to detect any anomalies. In this circumstance, the PR curve becomes more informative.

Similar to the ROC curve, the PR curve plots precision against recall for varying thresholds of anomaly scores. The advantage of replacing FPR with precision is an increased sensitivity to misclassifying the minority class: anomalies. This sensitivity means that missed anomalies and false detections lead to reduced precision, making it particularly suitable for scenarios with imbalanced datasets. Likewise, the Area Under the PR Curve (PR-AUC) provides a summary metric, where a higher value indicates better performance in detecting anomalies while maintaining high precision. Notably, the PR-AUC for random classifiers is equal to the proportion of positive

samples of the dataset. Let P be the total positives and N the total negatives of the dataset. For any threshold $\tau \in [0, 1]$, a random classifier predicts $TP = \tau \cdot P$ and $FP = \tau \cdot N$ instances as positive. Calculating precision and recall for this random classifier yields

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP + FP} = \frac{\tau \cdot P}{\tau \cdot P + \tau \cdot N} = \frac{P}{P + N}, \\ \text{Recall} &= \frac{TP}{TP + FN} = \frac{\tau \cdot P}{P} = \tau, \end{aligned}$$

resulting in a horizontal line that corresponds to the positive sample ratio of the dataset over varying τ .

3 Methodology

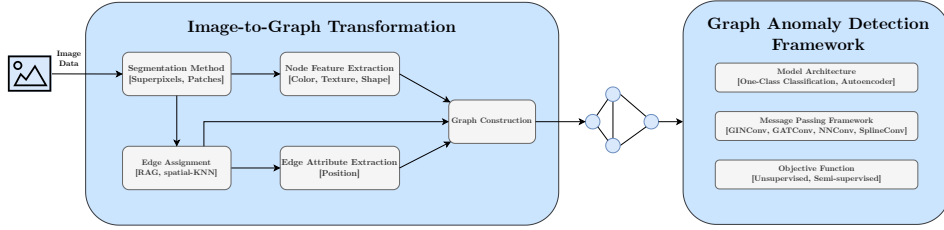


Figure 3.1: Overall process from image data to the image-to-graph transformation to graph anomaly detection

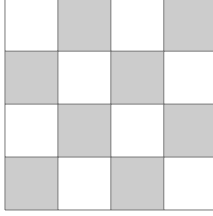
Our methodological approach is based on the realization that the graph-based anomaly detection process relies on two critical steps, which can be identified as bottlenecks as illustrated in Figure 3.1: 1) the transformation of image data into an adequate graph representation and 2) the application of advanced graph anomaly detection techniques to these representations. These steps are independent, however optimizations in the image-to-graph transformation can affect the performance of the subsequent graph-based anomaly detection and vice versa.

The following sections explain the methodological aspects in detail, starting with the image-to-graph transformation and closing with the applied graph anomaly detection frameworks.

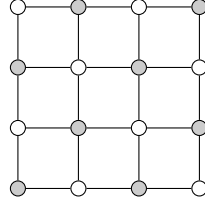
3.1 Image-to-Graph Transformation

The transformation of image data into graphs is a fundamental preprocessing step of our approach. A graph representation of an image $\mathbf{I} \in \mathbb{R}^{H \times W \times C}$ is a representation of $\mathbf{I}$ as an attributed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X}, \mathbf{E})$, whose nodes $v \in \mathcal{V}$ correspond to selected areas of $\mathbf{I}$ and the edges contain information about the local neighborhoods of each image area. The features of those areas are stored in the node feature matrix $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times d}$, while the spatial Euclidean structure is captured in edge attribute matrix $\mathbf{E} \in \mathbb{R}^{|\mathcal{E}| \times k}$.

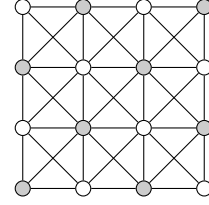
3.1.1 Grids



(a) 4x4 Pixel Grid



(b) 4-connected Pixel Adjacency Graph



(c) 8-connected Pixel Adjacency Graph

Figure 3.2: Representation of a 4x4 pixel image (a) and its 4-connected (b) and 8-connected adjacency graphs (c).

A natural way to think of images as graphs is to consider the grid structure of pixels, where each pixel in $\mathbf{I}$ can be represented as a node in the graph. As shown in Figure 3.2, such a graph connects its edges with a pixel's immediate neighbor: up to four in the case of 4-connected neighborhoods or eight in 8-connected neighborhoods. This kind of graph structure preserves the spatial adjacency of pixels, enabling operations that consider local interactions.

3.1.2 Image Segmentation

The aforementioned strategy is generally impractical due to the sheer amount of pixels in high-resolution images. One solution is to partition the image into segments of pixels through segmentation algorithms. These algorithms can range from superpixel algorithms that group pixels into semantically coherent, albeit often irregularly shaped regions, to patch-wise decomposition that produces uniformly shaped segments, preserving the grid-like structure of the original image.

Superpixels

There are various methods to generate superpixels efficiently. A prevalent algorithm, the *Simple Linear Iterative Clustering (SLIC)* as demonstrated in Figure 3.3b presented by Achanta et al. [13] (see Algorithm 1), clusters pixels based on both their color and spatial properties.

These pixels function in a five-dimensional vector space, composed of their color components (l, a, b) in the CIELAB color space and their spatial coordinates (x, y) .

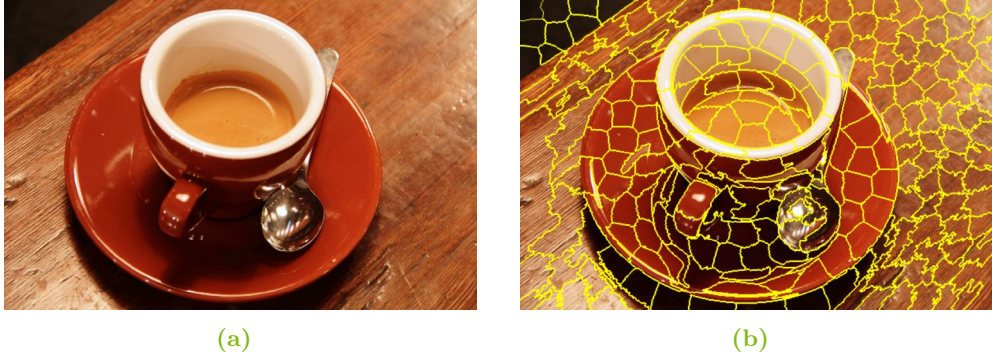


Figure 3.3: The original image (a) and the image segmented with a superpixel algorithm (b). Creative Commons Zero by the photographer (Rachel Michetti).

The choice of this color space over RGB is presumably motivated by its device independence and its closer correspondence to human color perception.

The heart of the algorithm lies in its distance measure, which cleverly balances color similarity d_c with spatial proximity d_s [13]:

$$d_c = \sqrt{(l_j - l_i)^2 + (a_j - a_i)^2 + (b_j - b_i)^2}, \quad (3.1)$$

$$d_s = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}. \quad (3.2)$$

The distance between coordinates is weighted by S and the color difference by a fixed, user-defined compactness parameter m [13]:

$$D = \sqrt{d_c^2 + \left(\frac{d_s}{S}\right) m^2}. \quad (3.3)$$

When m is large, the resulting superpixels tend to be more compact and regular in shape; when m is smaller, they more freely follow color boundaries in the image, resulting in more irregular sizes and shapes.

As an initialization step, they divide the image of N pixels into a regular grid of K cluster centers $\{C_i\}_{i=1}^K$ with $C_i = (l_i, a_i, b_i, x_i, y_i)^\top$ with spacing between these centers calculated as $S = \sqrt{N/K}$.

The clustering process then continues with three steps: First, given the previously mentioned distance measure, pixels within a $2S \times 2S$ region centered around each cluster center are assigned to their nearest cluster center. Next, new centers are

Algorithm 1 SLIC superpixel segmentation [13]

```

/* Initialization */
Initialize cluster centers  $C_k = [l_k, a_k, b_k, x_k, y_k]^\top$  by
sampling pixels at regular grid steps  $S$ .
Move cluster centers to the lowest gradient position in a
 $3 \times 3$  neighborhood.
Set label  $l(p) = -1$  for each pixel  $p$ .
Set distance  $d(p) = \infty$  for each pixel  $p$ .
repeat
  /* Assignment */
  for each cluster center  $C_k$  do
    for each pixel  $p$  in a  $2S \times 2S$  region around  $C_k$  do
      Compute the distance  $D$  between  $C_k$  and  $p$ .
      if  $D < d(p)$  then
        set  $d(p) = D$ 
        set  $l(p) = k$ 
      end if
    end for
  end for
  /* Update */
  Compute new cluster centers.
  Compute residual error  $E$ .
until  $E \leq \text{threshold}$ 

```

computed based on the average distance measured from each pixel to the current center in the cluster. These two steps repeat until the changes in cluster assignments become stable or a maximum number of iterations is reached. Finally, any small disconnected regions that form during clustering are merged with nearby larger clusters in a post-processing step, resulting in coherent superpixels that better represent the image structure.

Patches

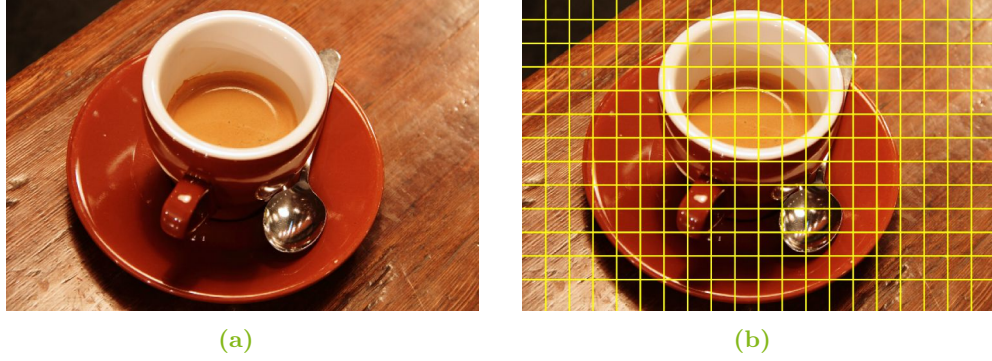


Figure 3.4: The original image (a) and the image divided into fixed-size patches (b). Creative Commons Zero by the photographer (Rachel Michetti).

While superpixels set the standard to reduce computational complexity by grouping similar pixels in regions, a comparable approach is to partition images into fixed-size *patches* as illustrated in Figure 3.4.

With this approach (see Algorithm 2), the image $\mathbf{I}$ is divided into a grid of non-overlapping patches $\mathcal{S} = \{S_i \mid i = 1, 2, \dots, n\}$, where each patch S_i is a rectangle of size $P \times P$ pixels with $H/P \cdot W/P = n$. Compared to superpixels, patches offer a more straightforward and computationally efficient representation. However, this method does not preserve semantic relationships between segments (e. g., background, object). Consequently, the model must learn these connections independently during training.

Using a uniform patch structure results in graphs with a fixed number of nodes for all graphs and invariable edge connections. This uniformity decouples the content from the structure for our anomaly detection task and allows GAD models to learn patterns based solely on the content of the patches, unlike superpixels, where the irregularly shaped regions and corresponding variable amounts of nodes and edges can affect the analysis.

Algorithm 2 Segment Image into Patches

```

Initialize patch size  $P$ 
Set  $H$  and  $W$  as image height and width
Set label  $l(p) = -1$  for each pixel  $p$ 
 $k \leftarrow 1$ 
for  $r$  in  $0, \dots, \lfloor \frac{H}{P} \rfloor - 1$  do
  for  $c$  in  $0, \dots, \lfloor \frac{W}{P} \rfloor - 1$  do
    for each pixel  $p \in \{rP, \dots, (r+1)P - 1\} \times \{cP, \dots, (c+1)P - 1\}$  do
       $l(p) \leftarrow k$ 
    end for
     $k \leftarrow k + 1$ 
  end for
end for
if  $H \bmod P \neq 0$  then
  Label remaining rows by repeating  $l(p)$  from  $p$  in row  $\lfloor \frac{H}{P} \rfloor P - 1$ 
end if
if  $W \bmod P \neq 0$  then
  Label remaining columns by repeating  $l(p)$  from  $p$  in column  $\lfloor \frac{W}{P} \rfloor P - 1$ 
end if

```

3.1.3 Graph Construction

Let $\Omega = \{0, \dots, W-1\} \times \{0, \dots, H-1\}$ be the set of all pixel coordinates. The resulting segmentation $\mathcal{S} = \{\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_n\}$ with $\mathcal{S}_i \subseteq \Omega$ from labels $l(p) \in \{1, \dots, n\}, p \in \Omega$ must satisfy the following properties:

- Each segment has to be a group of spatially adjacent pixels.
- The segmented regions must be a valid partition of Ω :

$$\begin{aligned}
& \emptyset \notin \mathcal{S} \\
& \bigcup_{i=1}^n \mathcal{S}_i = \Omega \\
& \mathcal{S}_i \cap \mathcal{S}_j = \emptyset, \quad i, j \in \{1, \dots, n\}, i \neq j.
\end{aligned}$$

Following this image segmentation, a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is constructed. This graph represents the image, where nodes in $\mathcal{V}$ correspond directly to the segments $\mathcal{S}_i$:

$$\mathcal{V} = \{v_i \mid \mathcal{S}_i \in \mathcal{S}, i = 1, 2, \dots, n\}.$$

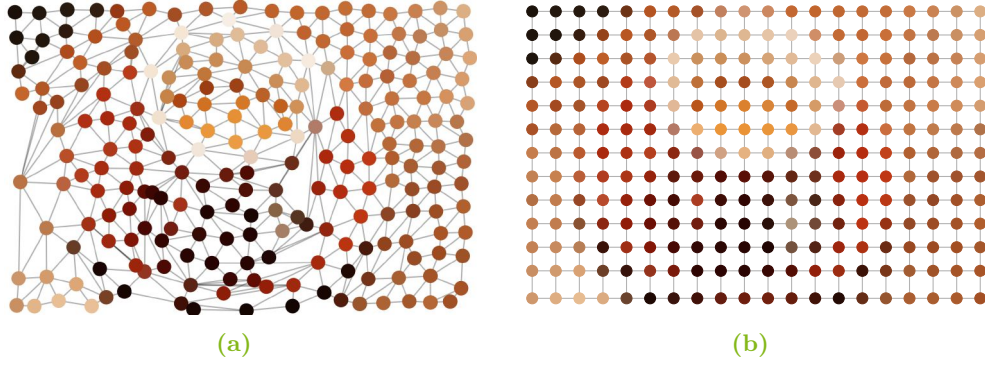


Figure 3.5: The segmented images 3.3b and 3.4b represented as region adjacency graphs for (a) superpixels (SLIC) and (b) patches. Nodes are colored with the mean RGB values across pixel intensities within a segment.

One approach of creating edges $\mathcal{E}$ is through *region adjacency graphs (RAGs)*. In RAGs, adjacent segments are connected akin to the mentioned pixel-level nodes in grids as demonstrated in Figure 3.5. Formally, let $C \in \{4, 8\}$ denote the connectivity, $\mathcal{E}_{\text{RAG}}$ is then defined as:

$$\mathcal{E}_{\text{RAG}} = \left\{ (v_i, v_j) \mid \mathcal{S}_i \neq \mathcal{S}_j \wedge \exists p \in \mathcal{S}_i, p' \in \mathcal{S}_j : \|p - p'\| \leq \sqrt{\frac{C}{4}} \right\}. \quad (3.4)$$

In this formulation, two segments $\mathcal{S}_i$ and $\mathcal{S}_j$ are considered adjacent if there exists at least one pair of pixels $p \in \mathcal{S}_i$ and $p' \in \mathcal{S}_j$ that are immediate neighbors, defined by their Euclidean distance, where the coordinates differ by one unit in the horizontal or vertical direction.

Another method forms the edge set through *k-nearest neighbors (k-NN)* with centroid coordinates. Concretely, given a function $\text{KNN}(v_i, k, \cdot)$ that computes the set of k -nearest neighbors of node v_i based on the distance between coordinates $\text{avg}(\mathcal{S}_i)$ and $\text{avg}(\mathcal{S}_j)$ with $j \in \{1, \dots, n\}, j \neq i$, the edge set $\mathcal{E}_{k\text{-NN}}$ is defined as:

$$\mathcal{E}_{k\text{-NN}} = \left\{ (v_i, v_j) \mid v_j \in \text{KNN}(v_i, k, \{\text{avg}(\mathcal{S}_1), \dots, \text{avg}(\mathcal{S}_n)\}) \right\}. \quad (3.5)$$

Unlike RAGs, which act on a pixel level, this method operates on a segment level, making it less computationally expensive.

3.1.4 Feature extraction

Descriptive features are extracted from image segments to enable comprehensive learning of visual patterns. These features transform the raw pixel values of

segmented regions into a concise set of numerical attributes. We categorize relevant visual information into three types, which are evaluated both independently and in combination: color, texture, and shape.

Color Features

The most common color space, RGB, defines color intensities as red, green, and blue within the range $[0, 255]$. Let $\mathcal{S}_i$ be the set of pixels in segment i . For each pixel $p \in \mathcal{S}_i$, the color intensity is defined as $I(p) = [I_R(p), I_G(p), I_B(p)]$, where $I_R(p), I_G(p), I_B(p)$ are the red, green, and blue intensity values, typically normalized to $[0, 1]$.

Color moments are quantitative measures that characterize the color distributions of an image. For our purposes, we applied the first, second and third-order moments:

- *First moment (Mean)*: The average RGB value of pixels within a segment is used as the default feature for the graph. For segment $\mathcal{S}_i$, the average RGB vector $\mu_i^{RGB} = [\mu_{i,R}, \mu_{i,G}, \mu_{i,B}]$ is defined as:

$$\mu_{i,c} = \frac{1}{|\mathcal{S}_i|} \sum_{p \in \mathcal{S}_i} I_c(p), \quad c \in \{R, G, B\} \quad (3.6)$$

- *Second moment (Standard Deviation)*: Measures the spread of the color distribution.

$$\sigma_{i,c} = \sqrt{\frac{1}{|\mathcal{S}_i|} \sum_{p \in \mathcal{S}_i} (I_c(p) - \mu_{i,c})^2}, \quad c \in \{R, G, B\} \quad (3.7)$$

- *Third moment (Skewness)*: Measures the asymmetry of the color distribution.

$$s_{i,c} = \sqrt[3]{\frac{1}{|\mathcal{S}_i|} \sum_{p \in \mathcal{S}_i} \left(\frac{I_c(p) - \mu_{i,c}}{\sigma_{i,c}} \right)^3}, \quad c \in \{R, G, B\}, \sigma_{i,c} > 0 \quad (3.8)$$

We included HSV and CIELAB, in addition to RGB, to account for color perception that is both more uniform and independent of imaging devices.

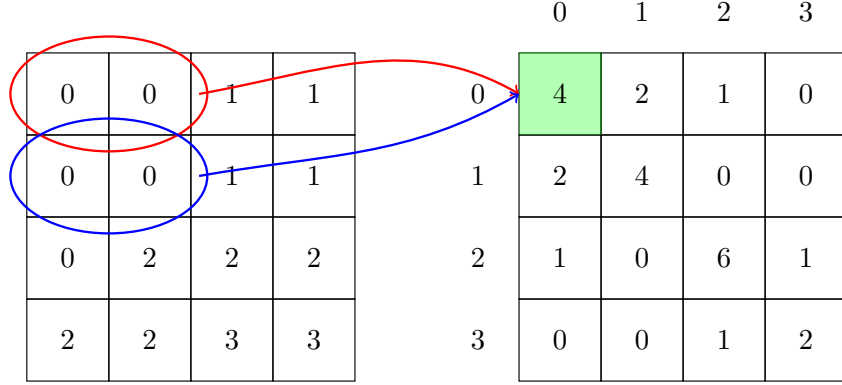


Figure 3.6: Grayscale image (left) and horizontal Gray-Level Co-occurrence Matrix (right) from [49]. Horizontal neighboring pixels are counted in both directions (left-to-right, right-to-left), exemplified by two cases of the pairs (0, 0).

Texture Features

Texture features describe the local variation of pixel intensities in a region. We used two methods to extract these features. The *Gray-Level Co-occurrence Matrix* (GLCM) [49] as shown in Figure 3.6, specifies the frequency of pairs of pixel intensities occurring at a given displacement (offset) and orientation (horizontal: 0° , vertical: 90° , diagonal: 45° , 135°). First, this technique converts the segmented image to grayscale $G_i(p)$. For a given offset $(\Delta x, \Delta y)$ and $\mathcal{S}_i$, the GLCM, $\mathbf{M}_{i,a,\Delta x,\Delta y}$, is a matrix where each entry $\mathbf{M}_{i,a,\Delta x,\Delta y}(j, k)$ is the number of times a pixel with grayscale intensity j occurs adjacent to a pixel with grayscale intensity k with the specified offset at angle $a \in \{0^\circ, 45^\circ, 90^\circ, 135^\circ\}$. After normalizing the GLCM such that $\sum_j \sum_k \mathbf{M}_{i,a,\Delta x,\Delta y} = 1$, various texture features can be derived:

- *Contrast*: Measures local intensity variation.

$$\text{Contrast} = \sum_{n=0}^{N_g-1} n^2 \left\{ \sum_{j=1}^{N_g} \sum_{k=1}^{N_g} \mathbf{M}_{i,a,\Delta x,\Delta y}(j, k) \right\}, |j - k| = n \quad (3.9)$$

where N_g is the number of gray levels.

- *Dissimilarity*: Highlights how different neighboring pixels are from each other.

$$\text{Dissimilarity} = \sum_j \sum_k \mathbf{M}_{i,a,\Delta x,\Delta y}(j, k) \cdot |j - k| \quad (3.10)$$

- *Energy (or Angular Second Moment)*: Measures textural uniformity.

$$\text{Energy} = \sum_j \sum_k \mathbf{M}_{i,a,\Delta x,\Delta y}(j, k)^2 \quad (3.11)$$

- *Correlation*: Measures the linear dependency of gray levels of neighboring pixels.

$$\text{Correlation} = \frac{\sum_j \sum_k (j - \mu_j)(k - \mu_k) \mathbf{M}_{i,a,\Delta x,\Delta y}(j, k)}{\sigma_j \sigma_k} \quad (3.12)$$

where $\mu_j, \mu_k, \sigma_j, \sigma_k$ are the means and standard deviations of the row and column sums of $\mathbf{M}_{i,a,\Delta x,\Delta y}$.

- *Homogeneity*: Measures the closeness of the distribution of elements in the GLCM to the diagonal.

$$\text{Homogeneity} = \sum_j \sum_k \frac{\mathbf{M}_{i,a,\Delta x,\Delta y}(j, k)}{1 + (j - k)^2} \quad (3.13)$$

Local Binary Patterns (LBP) [50] summarizes the local texture structure by comparing each pixel to its neighbors. For each pixel p in the grayscale segment G_i , a binary pattern code is generated by thresholding its neighbors based on the center pixel value. For a neighborhood of P pixels around p with radius R , the LBP code $LBP_{P,R}(p)$ is calculated as:

$$LBP_{P,R}(p) = \sum_{n=0}^{P-1} s(G_i(p_n) - G_i(p)) 2^n, \quad s(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases} \quad (3.14)$$

where p_n are the coordinates of the P neighbors of p .

We used all GLCM features over all angles and a histogram of LBP codes over the segment $\mathcal{S}_i$, H_i^{LBP} with $P = 8$, $R = 1$, as texture features.

Shape Features

Shape features quantify the geometric properties of a segmented region $\mathcal{S}_i$, such as its form, size, elongation, and orientation. These features can be especially relevant for anomaly detection, as anomalous images often display deformations or irregularities in the expected shape of objects or structures. Since segmented patches are uniformly shaped and, therefore, less informative, we utilized these features on superpixel segments, where shape variation is significant. Our approach utilizes a combination of region-based geometric measurements and moment-based descriptors as used in [16].

A fundamental way to describe shapes is using *image moments*. For a pixels coordinates of segment i , $\mathcal{S}_i \subseteq W \times H$, the raw moment of order $(p + q)$ is defined

as:

$$M_{pq} = \sum_{(x,y) \in \mathcal{S}_i} x^p y^q \quad (3.15)$$

From these raw moments, several basic features are derived:

- *Area*: The total number of pixels in the region, equivalent to the zeroth-order moment.

$$\text{Area} = M_{00} = |\mathcal{S}_i| \quad (3.16)$$

- *Centroid* $(\bar{x}_i, \bar{y}_i)$: The center of mass of the region, providing its geometric center.

$$\bar{x}_i = \frac{M_{10}}{M_{00}}, \quad \bar{y}_i = \frac{M_{01}}{M_{00}} \quad (3.17)$$

Further geometric descriptors are obtained from the segments's *bounding box*, defined by the minimum and maximum coordinates (x_{min}, y_{min}) and (x_{max}, y_{max}) surrounding the region $\mathcal{S}_i$:

- *Bounding Box Dimensions*:

$$\text{Width} : W_i = x_{max} - x_{min} + 1 \quad (3.18)$$

$$\text{Height} : H_i = y_{max} - y_{min} + 1 \quad (3.19)$$

- *Bounding Box Area*:

$$A_{bbox,i} = W_i \times H_i \quad (3.20)$$

- *Extent*: The ratio of the segment's area to its bounding box area indicates how much of the bounding box is occupied by the segment.

$$\text{Extent}_i = \frac{\text{Area}_i}{A_{bbox,i}} = \frac{M_{00}}{H_i W_i} \quad (3.21)$$

- *Equivalent Diameter*: The diameter of a circle with the same area as the segment.

$$D_{eq,i} = \sqrt{\frac{4 \cdot \text{Area}_i}{\pi}} \quad (3.22)$$

To accomplish invariance to translation, *central moments* μ_{pq} are calculated using coordinates relative to the centroid $(\bar{x}_i, \bar{y}_i)$:

$$\mu_{pq} = \sum_{(x,y) \in \mathcal{S}_i} (x - \bar{x}_i)^p (y - \bar{y}_i)^q \quad (3.23)$$

The second-order central moments $(\mu_{20}, \mu_{02}, \mu_{11})$ describe the distribution of pixels around the centroid and form the basis for an elliptical approximation of the shape:

- *Major and Minor Axis Lengths:* Derived from the eigenvalues of the covariance matrix constructed from normalized second-order central moments ($\mu'_{pq} = \mu_{pq}/\mu_{00}$). These lengths represent the primary dimensions of an ellipse having the same second-order moments as the segment, capturing its elongation.
- *Eccentricity:* Measures the elongation of the equivalent ellipse, calculated from its major and minor axis lengths (or equivalently, the eigenvalues $\lambda_{max}, \lambda_{min}$).

$$e_i = \sqrt{1 - \frac{\lambda_{min}}{\lambda_{max}}} \quad (3.24)$$

Eccentricity ranges from 0 (circle) to 1 (line segment).

- *Orientation:* The angle between the major axis of the equivalent ellipse and the horizontal axis, deduced from the second-order central moments. It determines the dominant direction of the shape.

$$\theta_i = \frac{1}{2} \arctan \left(\frac{2\mu'_{11}}{\mu'_{20} - \mu'_{02}} \right) \quad (3.25)$$

For resilience against changes in size and rotation, which are common variations that should not be considered anomalies, we apply *Hu Moments* [51]. These are a set of seven moments calculated based on the *normalized central moments* η_{pq} :

$$\eta_{pq} = \frac{\mu_{pq}}{\mu_{00}^\gamma}, \quad \text{where } \gamma = 1 + \frac{p+q}{2} \quad (3.26)$$

The normalization provides scale invariance, and the specific combinations forming the seven Hu moments (Hu1 to Hu7) yield rotation invariance.

Edge attributes

Images possess an inherent structure in Euclidean space, with pixels arranged in a regular grid. However, when we transform these images into graphs, we move to a representation that does not preserve this spatial structure. To retain this information, we explicitly encode relative Cartesian coordinates as edge attributes into the graph. This choice offers a key benefit over absolute node coordinates: invariance to translation. In later sections, we rigorously investigate the importance of these coordinates by comparing two separate graph anomaly detection models, only one of which utilizes edge attributes.

For a connected pair of nodes, we calculate the relative position vector e_{ij} from node i to node j in the image coordinate system as follows:

$$e_{ij} = \begin{pmatrix} x_i - x_j \\ y_i - y_j \end{pmatrix}, \quad (3.27)$$

where (x_i, y_i) and (x_j, y_j) represent the centroid coordinates of the segments i and j . These coordinates are subsequently normalized to the interval $[0, 1]$.

3.1.5 Dimensionality Reduction

When analyzing simple low-resolution image datasets, using the average RGB value may be sufficient to achieve effective anomaly detection performance. However, this might not be the case for high-resolution images, where noise or specific local anomalies can not be summarized by merely the average RGB value of a segment. Consequently, it necessitates the use of the above-mentioned additional features. Generally, using too many features while keeping the same number of samples leads to a problem called the *Curse of Dimensionality* [52]. Two factors describe this phenomenon as dimensionality increases: First, data space volume grows exponentially with each added dimension; second, the data becomes increasingly sparse, redundant, and similar. These factors cause additional computational expense and difficulty for anomaly detection algorithms to distinguish between normal and anomalous samples.

Intuitively, to sample the interval $[0, 1]$ in $\mathbb{R}$, 100 evenly spaced points with distance 0.01 is sufficient. However, for the unit square $[0, 1] \times [0, 1]$, with the same 0.01 distance, a total of 100^2 points is necessary. And in a general n -dimensional space, the number grows to 100^n points [53].

To mitigate this, we use the dimensionality reduction technique *Principal Component Analysis (PCA)* [54]. This technique projects the feature space onto a lower-dimensional subspace through a linear transformation that preserves maximum variance. Before applying PCA, it is necessary to normalize the features, typically by scaling them to zero mean and unit variance. Crucially, both the scaler and PCA must be fitted exclusively on the training data to prevent any information leakage to the testing dataset. The same fitted scaler and PCA are then applied to transform the features of both the training and testing datasets.

3.1.6 Special graphs with virtual nodes

Some datasets possess additional bitmasks that isolate the target within the image. For these datasets, we implemented unique graph structures incorporating virtual

nodes. These virtual or master nodes, as described by Gilmer et al. [35], contain features computed over selected parts of the image (e. g., the whole image, the masked part, or the remaining area) mirroring the dimensionality of those calculated for individual segments. Furthermore, they allow customization in terms of connectivity, such as undirected edges to all other nodes in the graph. This connectivity enables all nodes to access global image properties and summarize image-wide information, thereby bridging information between spatially separated or disconnected segments in the original segmentation masks.

3.2 Graph Anomaly Detection Framework

After the image-to-graph transformation, utilizing a suitable graph anomaly detection framework is the next central step. In this thesis, we examine two architectures in more detail: OCGTL (without edge attributes) and a Graph Autoencoder with edge attributes.

3.2.1 OCGTL Architecture

OCGTL, as proposed by Qiu et al. [19], consists of an ensemble of feature extractors: one reference extractor f and K additional extractors $\{f_k\}_{k=1}^K$, each implemented as a GIN (as defined in Equation 2.26). The OCGTL loss function combines two components: $\mathcal{L}_{\text{OCC}}$ and $\mathcal{L}_{\text{GTL}}$:

$$\mathcal{L}_{\text{OCGTL}} = \mathbb{E}_{\mathcal{G}} [\mathcal{L}_{\text{OCC}}(\mathcal{G}) + \mathcal{L}_{\text{GTL}}(\mathcal{G})], \quad (3.28)$$

where $\mathbb{E}_{\mathcal{G}}$ denotes the arithmetic mean over all graphs and feature extractors.

The one-class classification (OCC) term encourages embeddings to be close to a center $\mathbf{c}$:

$$\mathcal{L}_{\text{OCC}}(\mathcal{G}) = \sum_{k=1}^K \|f_k(\mathcal{G}) - \mathbf{c}\|. \quad (3.29)$$

This term, analogous to the loss in OCGIN, penalizes the distance of the graph embeddings from the center, aiming to enclose normal data within a minimal hypersphere. There is a minor difference, however. In OCGIN, like in Deep SVDD, the center had to be fixed to prevent hypersphere collapse. OCGTL provably prevents this issue by including the loss term $\mathcal{L}_{\text{GTL}}$, even with a trainable center.

The graph transformation learning (GTL) term encourages each feature extractor f_k to produce embeddings similar to the reference embedding $f(\mathcal{G})$ while being

dissimilar from embeddings produced by the other extractors f_l with $l \neq k$:

$$\mathcal{L}_{\text{GTL}}(\mathcal{G}) = - \sum_{k=1}^K \log \frac{c_k}{C_k}, \quad (3.30)$$

where:

$$c_k = \exp \left(\frac{1}{\tau} \text{sim}(f_k(\mathcal{G}), f(\mathcal{G})) \right), \quad (3.31)$$

$$C_k = c_k + \sum_{l \neq k} \exp \left(\frac{1}{\tau} \text{sim}(f_k(\mathcal{G}), f_l(\mathcal{G})) \right), \quad (3.32)$$

and τ denotes a temperature parameter, and $\text{sim}(\cdot, \cdot)$ is the cosine similarity defined as:

$$\text{sim}(\mathbf{z}, \mathbf{z}') = \frac{\mathbf{z}^\top \mathbf{z}'}{\|\mathbf{z}\| \|\mathbf{z}'\|}. \quad (3.33)$$

The OCGTL architecture is structured to support both unsupervised and, through an extension, semi-supervised training paradigms. This extension is analogous to the one that bridges Deep SVDD to Deep SAD, as elaborated in Chapter 2.4.2.

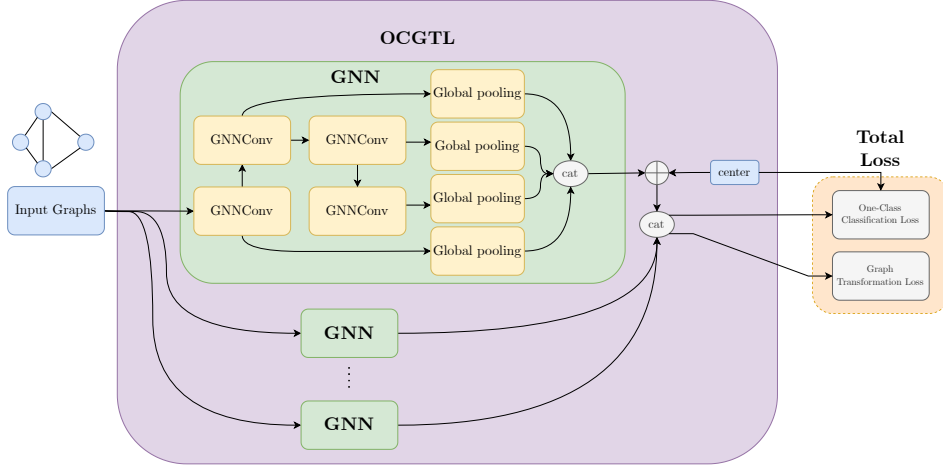


Figure 3.7: Architecture of OCGTL

Each feature extractor GNN carries a replaceable message-passing operator, originally employing a GINConv as its core module, denoted as GNNConv (see Figure 3.7). GIN was selected for its strong expressive power (see Chapter 2.3.1) in distinguishing graph structures, which we tested against GAT in an ablation study in our experiments. These GNN modules consist of multiple layers to capture

graph information at varying neighborhood depths. Within each GNN layer, node embeddings are updated via neighbor aggregation (**GNNConv**) optionally normalized with graph-specific normalization (**GraphNorm**) [55]. They are then passed through a non-linear activation (ReLU), and finally projected and aggregated into a graph-level representation for that layer using a global pooling function as described in Chapter 2.3.3. The graph-level representations from all layers within a single GNN module are concatenated, allowing the final output to incorporate features learned at different structural scales.

A key element specific to the one-class objective is the introduction of a center parameter. This parameter, initialized to match the output dimension of a single GNN module, is intended to represent the prototypical embedding of normal data. It is explicitly added ($\oplus$) to the output representation generated by the *first* GNN module.

The OCGTL model returns a list containing the final combined tensor **z_cat** and the learnable **center**. This output structure is designed to interface directly with the objective function. The architecture supports two training modes:

- **Unsupervised mode:** In the standard one-class setting, the model is trained only on normal data. The objective function (c.f. Equation 3.28) minimizes the distance of normal samples to the learned center while balancing the contrastive effect of different but relevant features.
- **Semi-Supervised mode:** Leveraging the availability of some labeled anomalies, the training can adopt a semi-supervised strategy, akin to the objective in Deep SAD from Equation 2.25. We modify the Equation 3.28 as follows:

$$\mathcal{L}_{\text{Semi-OCGTL}} = \begin{cases} \mathbb{E}_{\mathcal{G}} [\mathcal{L}_{\text{OCC}}^{-1} + \mathcal{L}_{\text{GTL}}], & y = -1 \\ \mathbb{E}_{\mathcal{G}} [\mathcal{L}_{\text{OCC}} + \mathcal{L}_{\text{GTL}}], & \text{otherwise} \end{cases} \quad (3.34)$$

Specifically, while unlabeled and normal samples ($y \in \{0, 1\}$) are encouraged to be close to the center, known anomalous samples ($y = -1$) are explicitly pushed away by modifying the distance-based term in the loss function in terms of their label.

The OCGTL objective and architecture make incorporating edge attributes challenging. Essentially, because there is no explicit edge-related loss term within the loss function, the model does not directly learn the structure of Euclidean space. Neither using absolute coordinates as node features nor utilizing a GNNConv that incorporates edge attributes, can inherently weigh the importance of the Euclidean coordinates in the resulting graph embedding or learn a representation of “normal” coordinates. As a result, the aggregated position within the graph embedding would not influence the distance between the embeddings and the center.

3.2.2 Architecture of Graph Autoencoders with Edge Features

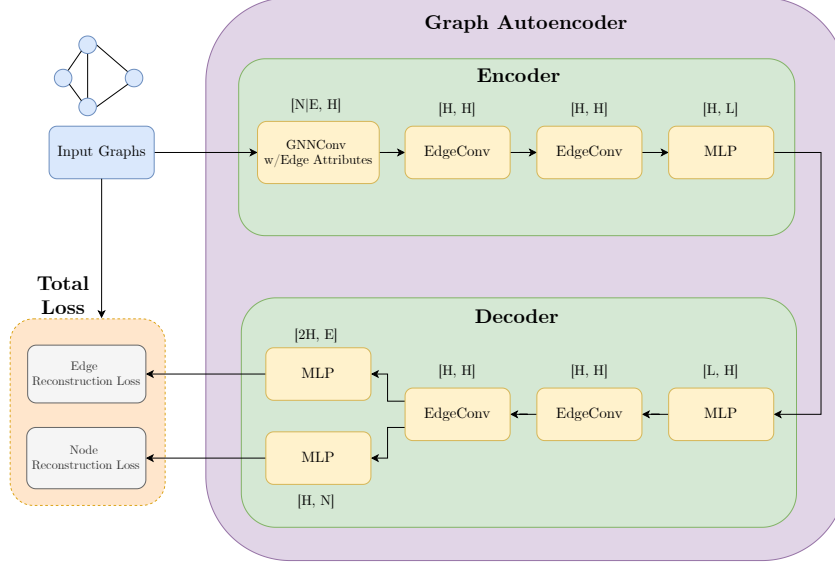


Figure 3.8: Architecture of the Graph Autoencoder with Edge Features. Node feature and edge attribute dimensions are depicted as N and E , respectively. H is the hidden dimension, and L is the latent dimension.

Inspired by [56], we implemented a Graph Autoencoder (GAE), which considers both node features and edge attributes. As introduced in Chapter 2.4.2, an autoencoder learns a compressed embedding of the input data and then reconstructs the original data from this representation. In the case of unsupervised anomaly detection, the autoencoder trains on only normal data, with the assumption that the model will learn to reconstruct normal samples with low reconstruction errors and higher errors for anomalies. In a semi-supervised setting, the autoencoder has two objectives. For normal data samples, it learns to produce accurate reconstructions. Conversely, for labeled anomalies, the autoencoder deliberately generates inferior reconstructions.

Our GAE architecture, depicted in Figure 3.8, consists of an Encoder and a Decoder module. The Encoder module utilizes graph convolutions that can handle edge attributes, enabling the model to learn representations that capture structural edge characteristics and node features. Between network layers, we used LeakyReLU as our activation function:

$$\sigma(x) = \begin{cases} \alpha x, & x < 0 \\ x, & \text{otherwise} \end{cases}$$

with $\alpha = 0.1$ as our activation function, based on prior work by [5].

The **Encoder** module maps the input graph, represented by its node features, edge topology, and edge attributes into a low-dimensional latent vector representation. The encoding process begins with a graph convolutional layer that explicitly requires edge attributes. We investigated different convolution operators for this initial step, including NNConv [35] and SplineConv [57]. A sequence of EdgeConv layers operating in the hidden dimension further processes the modified node embeddings. Finally, an MLP projects the node embeddings from the hidden dimension into the latent dimension.

The **Decoder** module reconstructs the original node features and edge features from the node embedding in the latent dimension with the original edges. The latent vector is first passed through an MLP to project from the latent space to the hidden dimension. The following EdgeConv layers, with a similar structure to the layers in the encoder, refine the resulting embedding. From the refined embeddings, two separate MLP-based reconstruction heads operate. Reconstruction of the node features is straightforward. Node embeddings are projected from the hidden dimension to the original node feature dimension. The other MLP projects adjacent node embeddings to the edge attribute dimension. These projections provide us with the reconstructed node features $\hat{\mathbf{X}} \in \mathbb{R}^{|\mathcal{V}| \times d}$ and edge features $\hat{\mathbf{E}} \in \mathbb{R}^{|\mathcal{E}| \times k}$.

Since both the node features and edge attributes are reconstructed, the loss function has to consider both aspects. Given $\mathbf{X}$ and $\hat{\mathbf{X}}$ as the original and reconstructed node features respectively, we define the node loss $\mathcal{L}_{node}$ as follows:

$$\mathcal{L}_{node} = \|\hat{\mathbf{X}} - \mathbf{X}\|_F \quad (3.35)$$

For the original and reconstructed edge attributes, $\mathbf{E}$ and $\hat{\mathbf{E}}$ the edge loss $\mathcal{L}_{edge}$ is defined as:

$$\mathcal{L}_{edge} = \|\hat{\mathbf{E}} - \mathbf{E}\|_F \quad (3.36)$$

Combining them, we obtain the following reconstruction errors depending on the two training modes:

- In **unsupervised mode**, $\mathcal{L}_{GAE}$ was simply calculated as the MSE:

$$\mathcal{L}_{GAE} = \lambda_{node} \cdot \frac{\mathcal{L}_{node}^2}{|\mathcal{V}|} + \lambda_{edge} \cdot \frac{\mathcal{L}_{edge}^2}{|\mathcal{E}|} \quad (3.37)$$

- The choice of the **semi-supervised** $\mathcal{L}_{\text{Semi-GAE}}$ was motivated by the best performing exponent-based loss function in [58]:

$$\mathcal{L}_{\text{Semi-GAE}} = \begin{cases} \left(\lambda_{\text{node}} \cdot \frac{\mathcal{L}_{\text{node}}}{|\mathcal{V}|} + \lambda_{\text{edge}} \cdot \frac{\mathcal{L}_{\text{edge}}}{|\mathcal{E}|} \right)^{-1}, & y = -1 \\ \lambda_{\text{node}} \cdot \frac{\mathcal{L}_{\text{node}}}{|\mathcal{V}|} + \lambda_{\text{edge}} \cdot \frac{\mathcal{L}_{\text{edge}}}{|\mathcal{E}|}, & \text{otherwise} \end{cases} \quad (3.38)$$

where λ_{node} and λ_{edge} are hyperparameters balancing the contribution of node and edge reconstruction errors.

During the evaluation of anomaly detection, our testing procedure supplies unseen normal and anomalous graphs to the graph autoencoder. Both unsupervised and semi-supervised approaches calculate the anomaly score for a given graph with the unsupervised weighted loss function on a per-graph basis. Graphs yielding high reconstruction errors can subsequently be flagged as potential anomalies.

4 Evaluation

In this chapter, we evaluate the graph-based anomaly detection models with the proposed image-derived graphs. We first detail our experimental setup by describing the benchmark datasets utilized in our evaluation and the specific experiments. Subsequently, we analyze our results on the effectiveness of our approach compared to image-based anomaly detectors. Finally, we perform a runtime analysis of our image-to-graph preprocessing and models.

4.1 Experimental Setup

We evaluate our approach using a dataset of labeled normal and anomalous images $\{(I_i, y_i)\}_{i=1}^N$, $I_i \in \mathbb{R}^{W \times H \times C}$, $y_i \in \{0, 1\}$ to quantify the effect of different image-to-graph transformations and graph-level anomaly detection models with performance metrics. Suitable datasets for this evaluation could range from popular benchmark classification datasets to those specialized for anomaly detection. This section outlines the experimental setup, covering the employed datasets, runtime environment, and concrete hyperparameters.

4.1.1 Datasets

We utilize four datasets for our experiments: MNIST, FashionMNIST, CIFAR-10, and HAM10000. Each dataset has its unique characteristics in the context of anomaly detection. We evaluated each dataset on a One vs. Rest principle throughout our experiments. This method designates a single class as the norm while treating all other classes as anomalies, converting a multi-class classification problem into a binary anomaly detection task. Within the unsupervised setting, the training dataset contains only normal instances, while semi-supervised training incorporates a limited number of labeled anomalous samples.

Our graph-based approach significantly reduces the feature dimensionality compared to image-based anomaly detection methods. The reduction is dependent on the chosen image segmentation parameters per dataset. Since storing explicit edge information noticeably increases data size, we provide a more compact alternative

by storing only node centroid coordinates and dynamically building edges with corresponding edge attributes at runtime.

MNIST

Split	Classes									
	0	1	2	3	4	5	6	7	8	9
Train	5923	6742	5958	6131	5842	5421	5918	6265	5851	5949
Test	980	1135	1032	1010	982	892	958	1028	974	1009

Table 4.1: Image distribution of MNIST for the train and test split over all classes [59]

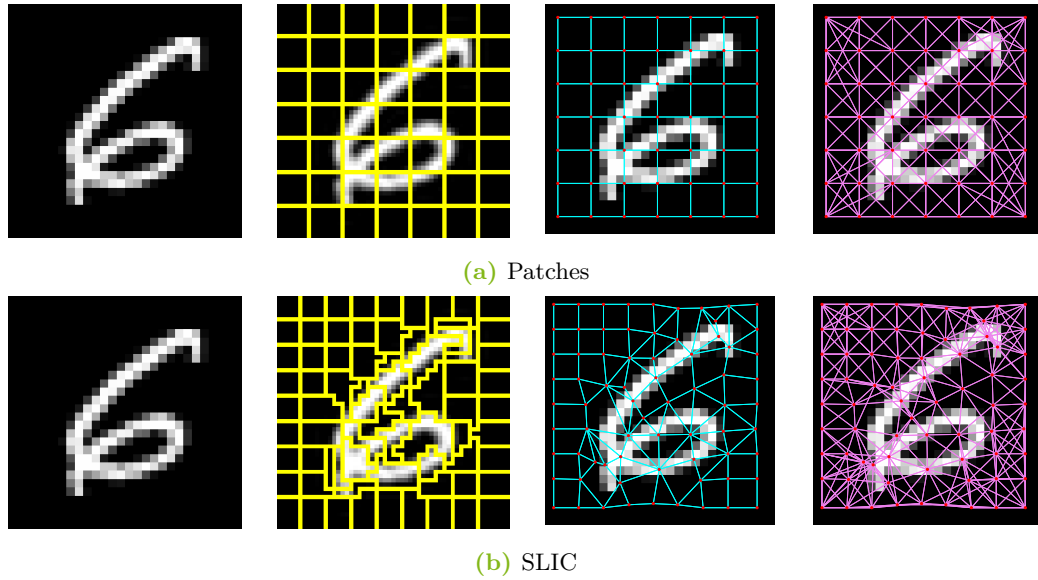


Figure 4.1: MNIST “Six” represented (1) as the raw image, (2) segmented, (3) as a graph with RAG edges, and (4) with 8-NN edges [59]

Our first dataset to benchmark is the MNIST (Modified National Institute of Standards and Technology) dataset [59] with 70,000 images of handwritten digits in grayscale and each image being 28×28 pixels. This dataset is divided into 60,000 training and 10,000 testing images. Each image is of a digit between 0 and 9, thus posing a 10-class classification problem. The class distribution shown in Table 4.1 is evenly spread, meaning there are approximately equal numbers of samples (≈ 6000) for each digit.

	Parameters	$ \mathcal{V} _{\min}$	$ \mathcal{V} _{\text{avg}}$	$ \mathcal{V} _{\max}$	$ \mathcal{E} _{\text{avg}}^{\text{rag}}$	$ \mathcal{E} _{\text{avg}}^{\text{knn}}$
Patch	$(p = 4)$	49	49	49	168	432
SLIC	$(K = 75, m = 10)$	52	65.65	81	267.77	589.28

Table 4.2: Graph metrics for the MNIST dataset

While visually simple, MNIST still presents challenges: variations in individual writing styles, differences in stroke thickness, minor rotations, and translations mean that even normality within a single digit class shows considerable variance. An effective anomaly detection model must learn to capture the essential features defining a specific digit while being robust to these acceptable variations.

Figure 4.1 visualizes the outcomes of different segmentation methods and their corresponding graph representations. It showcases both the RAG and k -NN approaches, with the latter implemented using $k = 8$.

Table 4.2 presents graph metrics for each segmentation method. Notably, the metrics for patch-based segmentation remain constant, a property observed across all subsequent datasets, whereas those associated with the SLIC method show severe variation. The choice of presented parameter values was motivated by balancing the number of nodes and edges between methods. However, for MNIST, selecting the number of segments K was mainly guided by the work of [60].

While the original MNIST images contain $28 \cdot 28 = 784$ features, our graph representation utilizes only $28/4 \cdot 28/4 = 49$ node features for patch-based graphs and ≈ 65.65 node features for SLIC-based graphs. Accounting for the necessary centroid positions (+2 features per node) used to calculate the edges and their attributes, this leads to an effective data reduction of 81.25% and 74.88%, respectively.

Fashion-MNIST

Split	Classes									
	Ankle boot	Bag	Coat	Dress	Pullover	Sandal	Shirt	Sneaker	T-shirt/top	Trouser
Train	6000	6000	6000	6000	6000	6000	6000	6000	6000	6000
Test	1000	1000	1000	1000	1000	1000	1000	1000	1000	1000

Table 4.3: Image distribution of Fashion-MNIST for the train and test split over all classes [61]

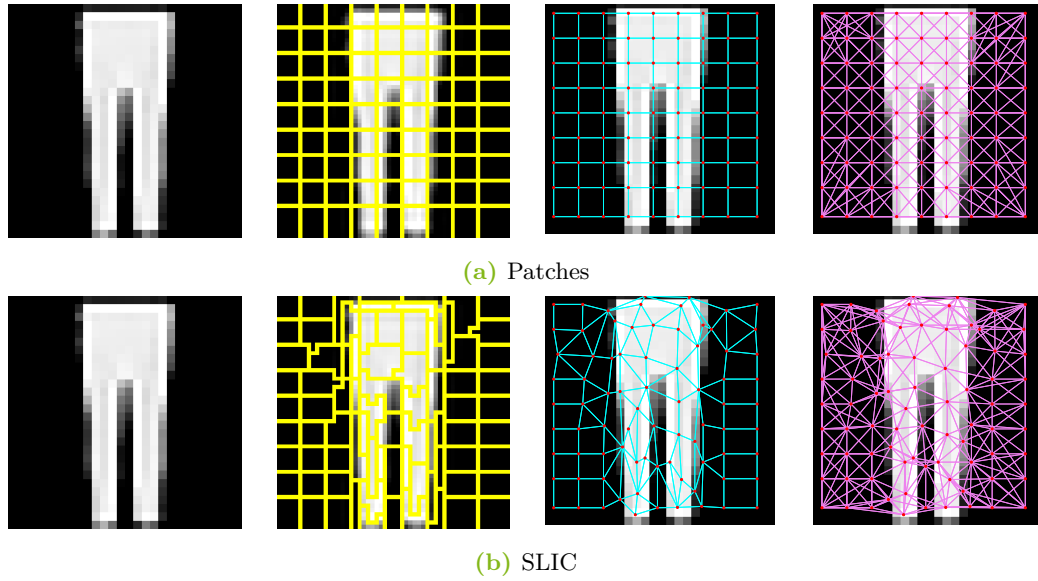


Figure 4.2: Fashion-MNIST “Trouser” represented (1) as the raw image, (2) segmented, (3) as a graph with RAG edges, and (4) with 8-NN edges [61]

The original MNIST dataset is now considered too simplistic for evaluating modern machine learning models. This limitation encouraged the creation of the Fashion-MNIST dataset [61] as a direct replacement. Fashion-MNIST matches the original MNIST dataset in scale and structure, with the same total images, image dimensions, train-test split, and number of classes. However, both the class distribution and classes are different from MNIST and can be gathered from Table 4.3.

Fashion-MNIST is more challenging than MNIST because of the greater complexity and diversity of the depicted items. While numbers have limited topology, clothing items possess a broader variety of shapes and textures within the same class. For example, the “Shirt” category could include blouses, polo shirts, and tank tops.

Moreover, categorizing some items like “Pullover”, “Coat” and “Shirt” is inherently more difficult than simple digit distinction.

Parameters		$ \mathcal{V} _{\min}$	$ \mathcal{V} _{\text{avg}}$	$ \mathcal{V} _{\max}$	$ \mathcal{E} _{\text{avg}}^{\text{rag}}$	$ \mathcal{E} _{\text{avg}}^{\text{knn}}$
Patch	$(p = 3)$	81	81	81	288	696
SLIC	$(K = 100, m = 10)$	27	68.37	92	297.97	622.54

Table 4.4: Graph Metrics for the Fashion-MNIST dataset

Given the additional complexity compared to MNIST, we reduce the features from 784 to $\lceil 28/3 \rceil^2 = 81$ node features for graphs built with patches, and ≈ 68.37 features with SLIC. Again, accounting for the centroid positions, this leads to a data reduction of 69% for patches and 73.84% for SLIC on the Fashion-MNIST dataset.

4.1.2 CIFAR-10

Split	Classes									
	Airplane	Automobile	Bird	Cat	Deer	Dog	Frog	Horse	Ship	Truck
Train	5000	5000	5000	5000	5000	5000	5000	5000	5000	5000
Test	1000	1000	1000	1000	1000	1000	1000	1000	1000	1000

Table 4.5: Image distribution of CIFAR-10 for the train and test split over all classes [62]

The CIFAR-10 dataset [62] is a further step towards real-world image complexity compared to MNIST and FashionMNIST. This dataset was curated by researchers at the Canadian Institute for Advanced Research (CIFAR) and contains 60,000 RGB color images, each sized 32×32 pixels. They divided the dataset into 50,000 training and 10,000 testing images and evenly distributed them across 10 classes, as reflected in Table 4.5.

CIFAR-10 presents several challenges to One vs. Rest classification. The low resolution of 32×32 pixels forces the model to learn meaningful representations from restricted visual information. Being color images (RGB), traditional models must effectively process multi-channel data. They also contain significant background clutter, and the objects can vary considerably in scale, viewpoint, pose, and illumination conditions within the same class.

The graph representation lowers the data dimensionality from the original $32 \cdot 32 \cdot 3 = 3072$ to $256 \cdot (3 + 2) = 1280$ for patch-based representations and approximately

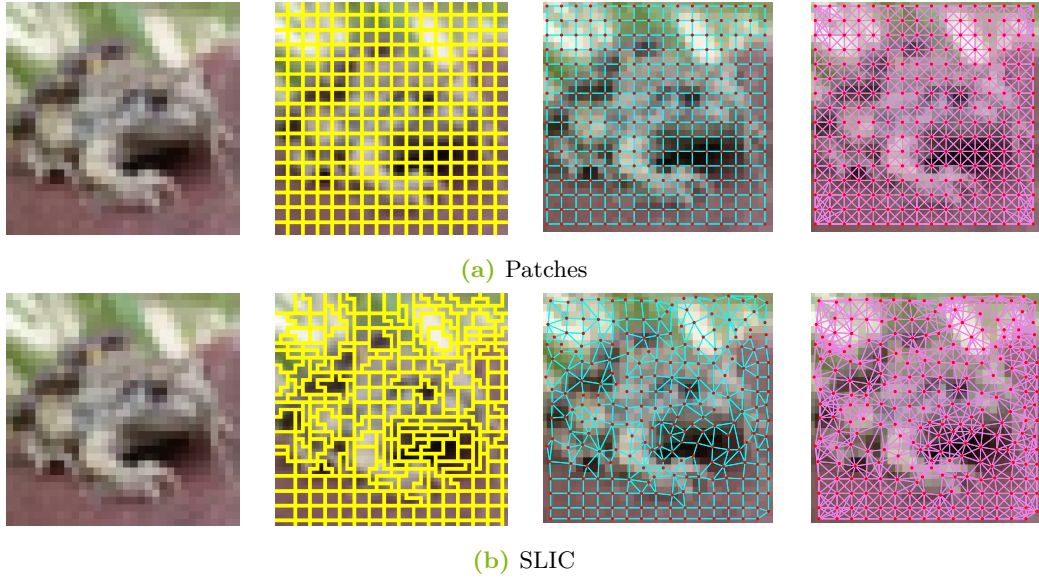


Figure 4.3: CIFAR-10 “Frog” represented (1) as the raw image, (2) segmented, (3) as a graph with RAG edges, and (4) with 8-NN edges [62]

	Parameters	$ \mathcal{V} _{\min}$	$ \mathcal{V} _{\text{avg}}$	$ \mathcal{V} _{\max}$	$ \mathcal{E} _{\text{avg}}^{\text{rag}}$	$ \mathcal{E} _{\text{avg}}^{\text{knn}}$
Patch	$(p = 2)$	256	256	256	960	2124
SLIC	$(K = 250, m = 5)$	179	230.78	269	990.39	2042.44

Table 4.6: Graph Metrics for the transformed CIFAR-10 dataset

$230.78 \cdot (3 + 2) = 1153.9$ for SLIC-based representations. This cuts down the data by 58.33% for patches and 62.44% for SLIC compared to the raw CIFAR-10 images.

HAM10000

Split	Classes						
	akiec	bcc	bkl	df	mel	nv	vasc
Train	262	417	887	86	895	5356	109
Test	65	97	212	29	218	1349	33

Table 4.7: Image distribution of HAM10000 for the train and test split over all classes [63]

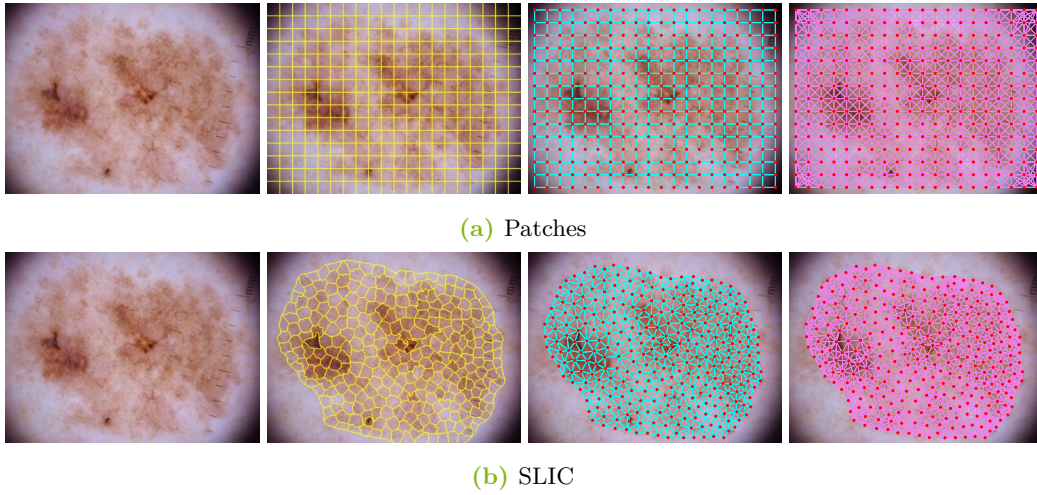


Figure 4.4: HAM10000 “Melanoma” represented (1) as the raw image, (2) segmented, (3) as a graph with RAG edges, and (4) with 8-NN edges [63]

Moving into a specialized, high-resolution domain, the HAM10000 (“Human Against Machine with 10000 training images”) dataset [63] provides a challenging benchmark based on real-world medical imagery. It is a collection of 10,015 dermatoscopic images of common pigmented skin lesions and designated binary masks [64], sourced from different populations and captured using various dermatoscopes, resulting in images of varying resolutions. These images were centered and cropped around the lesion, resulting in resolutions of 600×450 . The dataset is categorized into seven distinct diagnostic classes, with the majority based on histopathological confirmation (study based on skin tissue) but also through follow-up examination, expert consensus, or confirmation by in-vivo confocal microscopy: Actinic keratoses and intraepithelial carcinoma / Bowen’s disease (akiec), Basal cell carcinoma (bcc), Benign keratosis-like lesions (bkl), Dermatofibroma (df), Melanoma (mel), Melanocytic nevi (nv), and Vascular lesions (vasc) [63]. As shown in Table 4.7, class distribution is imbalanced,

4 Evaluation

	Mask	Parameters	$ \mathcal{V} _{\min}$	$ \mathcal{V} _{\text{avg}}$	$ \mathcal{V} _{\max}$	$ \mathcal{E} _{\text{avg}}^{\text{rag}}$	$ \mathcal{E} _{\text{avg}}^{\text{knn}}$
Patch	$\times$	$(p = 30)$	300	300	300	1130	2482
Patch	$\checkmark$	$(p = 30)$	300	300	300	1732	3084
SLIC	$\times$	$(K = 300, m = 10)$	194	295.97	301	1619.98	2531.40
SLIC	$\checkmark$	$(K = 300, m = 10)$	277	301.24	303	2251.05	3234.82

Table 4.8: Graph Metrics for the transformed HAM10000 dataset

with most images stemming from the Melanocytic nevi class (commonly verbalized as “moles”) and Melanoma, the most fatal type of skin cancer. In the absence of a predefined split in the dataset, we allocated 80% of the images for training and the remaining 20% for testing.

The HAM10000 dataset is particularly relevant for assessing anomaly detection models due to the real-life impact of distinguishing potentially malignant lesions (anomalies) from benign ones (normal). Melanocytic nevi (‘nv’), being the most common type of benign lesion, is designated as the only ‘normal’ class in this case, primarily due to class imbalance. The dataset poses significant challenges: Fine-grained visual similarity between different classes (e.g., certain nevi and early melanomas), high intra-class variability within lesion types, and the presence of various artifacts such as hairs, ink markings, ruler guides, or air bubbles. Furthermore, variations in lighting, skin tone, and camera equipment add complexity. Given these factors and the often irregular shape of lesions, image segmentation masks provided with the dataset were utilized during preprocessing to focus the analysis primarily on the lesion area rather than the surrounding skin or artifacts.

Our strategy concerning graph construction differs when utilizing segmentation masks. For patches, the segmentation occurs on the masked image, while the scikit-learn implementation of SLIC allows concentrated segmentation on the region provided by the binary mask.

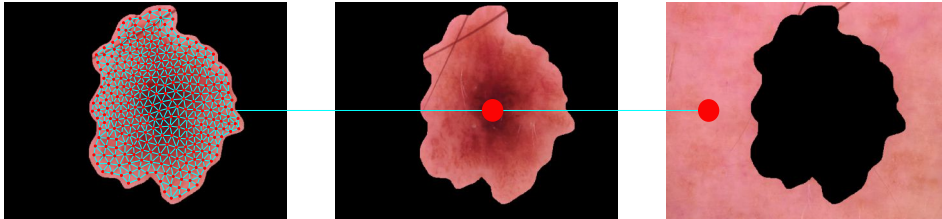


Figure 4.5: Virtual nodes used on HAM10000 [63]. (1) Superpixel nodes are each connected to the (2) lesion node, which in turn is connected to the (3) skin node.

Feature Type	Count	Data Reduction (300 nodes) in %
RGB_{avg}	$3 + 2$	99.81
Color 3.1.4 (RGB, HSV, LAB)	$27 + 2$	98.93
Texture 3.1.4 (GLCM, LBP)	$29 + 2$	98.85
Shape 3.1.4	$38 + 2$	98.50
All (Color + Texture)	$56 + 2$	97.85
All (Color + Texture + Shape)	$94 + 2$	96.44

Table 4.9: Feature count (incl. centroid positions) and data reduction for patch-based (Color, Texture, Color + Texture) and SLIC-based (Color, Texture, Shape, Color + Texture + Shape) feature sets

If we apply masks, we optionally introduce two virtual nodes as depicted in Figure 4.5. The first virtual node v_{n+1} represents the entire segmented skin lesion, while the second node v_{n+2} represents the surrounding skin. We establish connections between these virtual nodes and the rest of the graph to enhance information flow, as described in Chapter 3.1.6. Specifically, we link $\mathcal{E} \cup \{(v, v_{n+1}), (v_{n+1}, v) \mid v \in \mathcal{V}\} \cup \{(v_{n+1}, v_{n+2}), (v_{n+2}, v_{n+1})\}$ leading to a significant increase of edges as observed in Table 4.8.

For this dataset, the images consist of $600 \cdot 450 \cdot 3 = 810000$ pixel values. We present resulting data reductions in Table 4.9.

4.1.3 Experiments

We conducted two separate sets of experiments: An unsupervised and a semi-supervised setup. With the unsupervised setting, the training dataset contained only samples of the predefined normal class, while the testing dataset consisted of both normal and anomalous instances. In the semi-supervised setup, the training dataset consisted of 95% unlabeled normal data and 5% labeled anomalies. These anomalies were chosen from the first five classes, excluding the normal class of the training dataset. Consequently, the remaining anomaly classes were effectively treated as novel anomalies during testing. If the number of labeled anomalies was insufficient to reach the 5% threshold, we filled labeled normal samples to maintain this ratio. Further concrete findings with a higher ratio of labeled anomalies ($1/3$) can be found in Appendix B.

Benchmark results were evaluated with GAE (NNConv) and OCGTL (GINConv) from Chapter 3.2 across the datasets addressed in Chapter 4.1.1, the two segmentation methods (Patch, SLIC) of Chapter 3.1.2, and the two edge construction

methods (RAG, k -NN) introduced in Chapter 3.1.3. This setup allows us to assess the effects of both the image-to-graph transformation and the graph anomaly detection framework on the performance. Henceforth, a *configuration* refers to a specific combination of Model, Segmentation Method, and Edge Construction.

For the standard `torchvision` datasets (MNIST, Fashion-MNIST, and CIFAR-10), the constructed graphs utilized only the mean RGB (or grayscale) intensity as node features. For grayscale images specifically, SLIC was applied after repeating the single grayscale channel three times to create a 3-channel input.

Since we only considered one class normal in the HAM10000 dataset, we had room to conduct comprehensive ablation studies. These analyses focused on evaluating performances with different node feature sets (color, texture, shape) revealed in Chapter 3.1.4, the effect of removing virtual nodes, and the performance variations when using different GNN Convolutions (GAE with SplineConv, OCGTL with GATConv).

Regarding model hyperparameters, the hidden dimension for the GAE was configured to 32 for MNIST and 64 for all other datasets. The latent dimension of the GAE was set as $\lfloor (d + k)/2 \rfloor$, where d and k are the numbers of node and edge features. As for the loss function L_{GAE} , we utilized $\lambda_{\text{node}} = 0.3$ and $\lambda_{\text{edge}} = 1.0$. Supplementary experiments with $\lambda_{\text{node}} = 1$ and $\lambda_{\text{edge}} = 0$ found in Appendix C solidified that achieving competitive performance requires a dedicated loss function for edge attributes.

The OCGTL model used a hidden dimension of 32 across all datasets. The GNN ensemble consists of 6 GNN modules, each with 4 GNN convolution layers. After each convolution, outputs are normalized with GraphNorm, and the final graph embedding is obtained via global sum pooling (see Equation 2.20).

Each configuration was trained for 20 epochs on mini-batches of size 32 with a learning rate of 0.001 by the *Adam optimizer* [65], a variant of SGD. All node features, except the mean RGB values, were reduced via PCA to retain up to 95% variance. Other possible modifications used by image-based approaches, e.g., data augmentation of datasets, L2 regularization, or adaptive schedulers that modify the learning rate depending on the training duration, were excluded.

4.1.4 Execution Environment

The image-to-graph transformations, models, and experiments were implemented with the help of libraries such as PyTorch, PyTorch Geometric, scikit-learn, and scikit-image on Python 3.11.6. We utilized PyTorch Geometric for handling graph data

and the implementation of graph-level anomaly detectors proposed in Chapter 3.2 (in conjunction with PyTorch). Scikit-learn provided various evaluation metrics and scikit-image implementations for SLIC, RAG, and feature extraction methods like GLCM and LBP.

A computer running macOS 15.4 with an 11-Core Apple M3 Pro processor and 18 GB RAM was used for preprocessing the image data into PyG Data graphs, as well as for training and evaluating the graph anomaly detection models on MNIST, Fashion-MNIST, and CIFAR-10. Despite the availability of GPU acceleration via PyTorch’s Metal Performance Shader (MPS) for Apple Silicon, we encountered memory leak issues. Consequently, we opted to utilize only the CPU for our computations. The ablation studies on HAM10000 were performed on a GPU server equipped with an Intel Core i7-3930K CPU, 48 GB RAM, and an NVIDIA GeForce GTX 1080 GPU running Ubuntu 22.04.5. For these experiments, we leveraged CUDA to accelerate computation through the GPU. However, due to the slightly outdated CPU, we preprocessed the HAM10000 dataset on the Mac and transferred the data over the Internet.

4.2 Results

In this section, we present results expressed as the AUC-ROC and AUC-PR metrics discussed in Chapter 2.4.4. We compared against conventional image-based anomaly detection techniques by referencing results from Ruff et al. [5] on MNIST and CIFAR-10, and, by virtue of missing data in the Deep SVDD paper, Golan et al. [66] for Fashion-MNIST in our unsupervised setup. For the semi-supervised setting, we benchmarked our scores against Deep SAD from [6]. However, the authors of this paper used a slightly different experimental setup than ours. The significant difference is that they incorporated 10% anomaly pollution into the unlabeled portion of the training dataset. Furthermore, their paper did not report ROC-AUC values per class but as an aggregated average.

The inherent uniform class distribution of `torchvision` classification datasets and the nature of the One vs. Rest classification create an abundance of anomalous data samples during testing. This abundance violates the underlying assumption of PR curves, which are designed for scenarios with a rare anomaly class. Consequently, the resulting PR-AUC values become less informative than ROC-AUC scores. For this reason, the PR-AUC scores on `torchvision` datasets have been moved to Appendix A. The following tables highlight the best results in **bold** and the second best results in underline.

4.2.1 MNIST

c_i	GRAPH AUTOENCODER				OCGTL				DEEP
	Patch		SLIC		Patch		SLIC		SVDD
	RAG	k -NN	RAG	k -NN	RAG	k -NN	RAG	k -NN	[5]
0	96.0±3.2	98.7±0.2	86.3±2.3	93.4±0.7	90.5±7.2	78.7±2.7	57.2±4.3	66.4±6.0	<u>98.0±0.7</u>
1	98.8±0.4	<u>99.4±0.1</u>	92.8±1.1	93.9±1.4	92.8±5.8	99.2±0.3	86.9±1.5	94.6±0.9	99.7±0.1
2	88.2±4.9	92.0±1.2	66.1±3.4	74.5±2.6	63.6±1.8	71.6±3.2	52.8±3.0	57.0±2.2	<u>91.7±0.8</u>
3	<u>86.2±2.9</u>	91.9±0.7	68.1±0.6	79.4±2.3	72.0±1.1	78.1±1.4	53.7±1.5	57.3±2.4	91.9±1.5
4	87.7±4.1	<u>94.1±0.4</u>	75.1±2.6	79.8±2.8	79.0±2.3	82.4±1.6	61.3±3.1	60.6±1.7	94.9±0.8
5	78.6±6.0	<u>86.4±0.8</u>	62.9±1.6	69.7±1.7	62.9±7.9	62.0±4.2	54.3±3.7	57.2±2.4	88.5±0.9
6	95.6±0.8	<u>97.1±0.4</u>	82.3±1.7	90.1±3.0	74.6±3.0	72.5±9.7	57.3±2.8	59.7±1.9	98.3±0.5
7	92.9±1.4	<u>93.6±0.8</u>	83.6±0.7	87.9±0.4	75.1±1.5	81.2±1.6	65.0±2.7	65.8±2.2	94.6±0.9
8	88.8±2.9	<u>93.6±0.7</u>	69.9±1.5	79.2±2.3	86.6±1.2	84.8±0.8	67.8±2.7	64.5±5.3	93.9±1.6
9	92.2±1.6	<u>94.7±0.6</u>	81.7±1.0	88.3±0.8	81.7±1.1	84.6±0.6	63.8±2.3	65.8±1.7	96.5±0.3
avg	90.5	<u>94.1</u>	76.9	83.6	77.9	79.5	62.0	64.9	94.8

Table 4.10: Unsupervised setup reporting average ROC-AUC and StdDev values in % for the MNIST dataset over 5 seeds.

In Table 4.10, we showcase our findings for MNIST with the unsupervised setup. The leftmost column, c_i , represents the normal class used in our One vs. Rest experiments. ROC-AUC scores were presented across four columns based on image segmentation and edge construction methods for each graph-level anomaly detection method. Deep SVDD, the image-based deep learning baseline, underwent a two-stage training process: 350 epochs of pretraining on a Deep Convolutional Autoencoder (DCAE), followed by an additional 250 epochs of further optimization [5].

The results suggest that while Deep SVDD is the best overall performer achieving an average ROC-AUC of 94.8%, graph-based methods can compete with an appropriate graph structure and anomaly detection method. For MNIST, there are unambiguously superior methods concerning GAD, segmentation, and edge construction with the best configuration achieving an ROC-AUC of 94.1%.

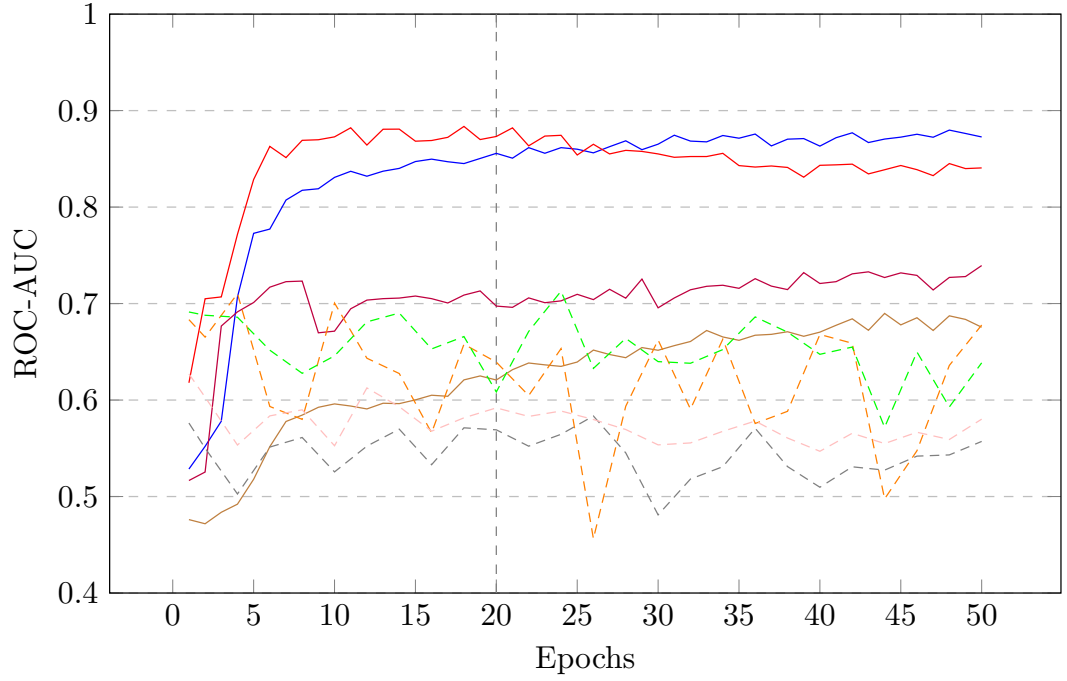
The GAE outperforming OCGTL aligns with our assumption that Euclidean coordinates are beneficial for representing images as graphs, especially in an unsupervised GAD setting. Another expected result was k -NN performing better than RAG with $k = 8$. The more than double amount of edges in k -NN compared to RAG allows message passing on a larger scale, improving anomaly detection rate. Contrary to expectations of prior works [15], [17], [18] that mainly favored superpixels for their tasks, SLIC segmentation yielded inferior scores to patch segmentation despite generating more nodes and edges. This outcome indicates that our anomaly detection models favor uniformly structured graphs with an equal number of nodes over irregular ones.

c_i	GRAPH AUTOENCODER				OCGTL				DEEP
	Patch		SLIC		Patch		SLIC		SAD
	RAG	k -NN	RAG	k -NN	RAG	k -NN	RAG	k -NN	[6]
0	98.9±0.5	99.2±0.3	95.7±1.0	95.3±1.9	98.2±0.8	93.3±2.6	74.7±6.9	76.9±2.1	—
1	99.6±0.1	99.8±0.0	98.6±0.2	98.5±0.5	<u>99.7±0.1</u>	99.5±0.1	97.4±1.0	97.5±0.6	—
2	97.4±0.5	<u>96.6±0.5</u>	86.9±1.3	80.7±1.2	66.8±6.0	79.5±4.4	60.2±3.4	60.8±2.5	—
3	96.7±0.9	<u>96.0±0.6</u>	90.8±3.1	88.2±2.1	87.7±4.3	87.9±4.3	55.8±5.0	61.0±1.8	—
4	<u>93.3±3.2</u>	96.0±0.7	86.5±2.7	85.2±2.5	89.2±1.8	90.5±1.7	70.0±3.2	67.4±3.7	—
5	<u>93.3±3.1</u>	95.3±1.7	76.1±4.6	77.9±5.9	68.5±6.7	69.4±6.0	56.1±2.0	60.1±1.3	—
6	98.9±0.5	<u>98.2±0.9</u>	92.9±1.2	89.5±2.5	88.8±2.8	93.1±1.2	61.0±7.3	65.0±4.0	—
7	<u>96.2±0.4</u>	97.3±0.7	87.8±3.5	88.6±1.6	86.7±4.5	89.3±2.5	68.9±7.2	68.9±4.6	—
8	<u>94.0±0.9</u>	94.7±2.0	85.5±1.6	82.7±2.4	93.1±0.6	87.6±2.0	69.1±3.5	67.4±3.9	—
9	<u>94.3±1.4</u>	96.0±0.8	87.4±1.4	87.1±2.4	89.2±1.1	89.0±1.4	72.8±3.1	71.5±2.5	—
avg	96.3	<u>96.9</u>	88.8	87.4	86.8	87.9	68.6	69.7	97.3

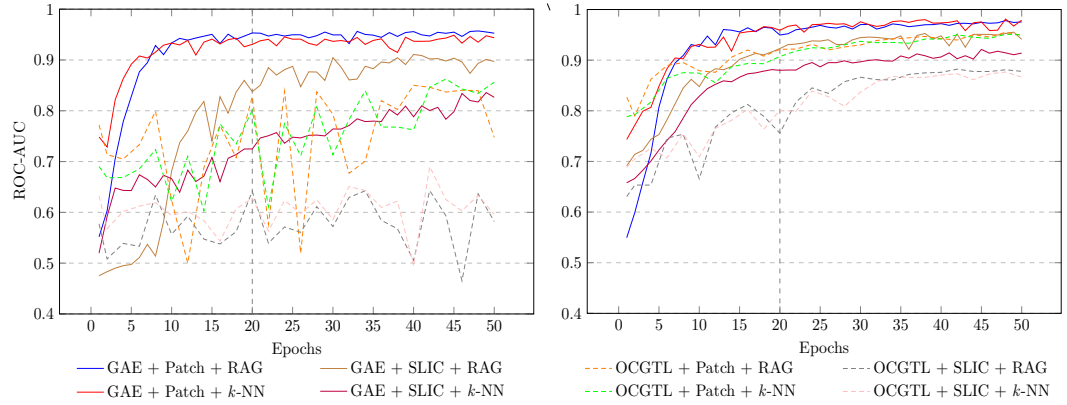
Table 4.11: Semi-supervised setup: Average ROC-AUC and StdDev values in % for the MNIST dataset over 5 seeds.

We report the ROC-AUC values of the semi-supervised experiments in Table 4.11. As previously mentioned, while Deep SAD is disadvantaged due to the anomaly pollution in its unlabeled training dataset, it still benefits from more features and longer training epochs, allowing it to marginally outperform the best graph-based method by 0.4 percentage points (%pt). Nevertheless, by incorporating just 5% labeled anomalies, all graph-based models performed better than in the unsupervised setup, with the best GAE model improving by 2–12%pt and OCGTL by 5–9%pt.

Figure 4.6 visually illustrates the performance improvements for class c_5 , which was the most challenging class based on our findings. Although OCGTL showed erratic and stagnant ROC-AUC values, the model showed progressive improvements over epochs as more and more labels were introduced. This erratic behavior may be attributed to the small batch size (32). With a lower probability of including labeled anomalies into small batches, the gradient updates could have been predominantly influenced by unlabeled data.



(a) Unsupervised ROC-AUC values



(b) Semi-supervised ROC-AUC values with (1) $1/20$ and (2) $1/3$ labeled anomalies.
See Table B.1 for concrete results of (2).

Figure 4.6: Comparison of configurations trained on MNIST with normal class “5” over 50 epochs.

4.2.2 Fashion-MNIST

c_i	GRAPH AUTOENCODER				OCGTL				OC-
	Patch		SLIC		Patch		SLIC		SVM
	RAG	k -NN	RAG	k -NN	RAG	k -NN	RAG	k -NN	[66]
0	<u>98.6±0.3</u>	98.7±0.2	96.1±0.5	98.4±0.3	97.4±0.2	98.1±0.2	70.7±2.5	88.4±1.9	98.2
1	<u>93.3±1.3</u>	94.6±0.7	75.3±2.4	90.6±1.1	65.4±2.8	62.4±4.8	52.6±0.8	54.8±0.4	90.3
2	<u>91.0±0.2</u>	91.2±0.1	81.5±1.5	90.5±0.6	82.8±1.4	82.3±0.8	58.8±1.8	61.9±1.6	90.7
3	94.2±0.5	<u>93.1±0.3</u>	86.6±1.5	90.1±1.1	83.6±0.7	86.9±1.1	62.3±3.7	63.1±5.5	94.2
4	<u>90.6±0.4</u>	91.0±0.2	80.4±1.9	87.7±1.2	75.7±0.7	68.9±2.1	52.6±1.8	56.8±0.8	89.4
5	85.4±0.3	85.7±0.4	81.4±2.2	<u>86.0±0.9</u>	81.5±5.5	79.0±1.7	61.1±1.8	69.1±2.0	91.8
6	79.3±0.7	83.6±0.6	65.5±2.1	80.4±0.4	75.1±1.5	68.9±0.5	50.7±3.0	54.6±2.9	<u>83.4</u>
7	98.6±0.1	<u>98.7±0.1</u>	96.8±0.5	97.8±0.1	94.2±1.3	94.5±0.7	76.9±3.9	74.3±4.1	98.8
8	<u>90.8±1.4</u>	89.8±0.8	79.1±0.8	88.5±0.7	77.0±2.0	72.8±3.7	52.3±1.5	54.8±1.6	91.9
9	<u>98.6±0.1</u>	98.5±0.0	97.6±0.4	98.0±0.3	92.7±1.1	95.8±0.3	78.6±3.4	78.8±1.7	99.0
<i>avg</i>	92.0	<u>92.5</u>	84.0	90.8	82.6	81.0	61.7	65.6	92.8

Table 4.12: Unsupervised setup: Average ROC-AUC and StdDev values in % for the Fashion-MNIST dataset over 5 seeds.

Turning to the unsupervised results of Fashion-MNIST shown in Table 4.12, the classes c_i were adjusted from the `torchvision` ordering, [T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot] to the alphabetical order [Ankle boot, Bag, Coat, Dress, Pullover, Sandal, Shirt, Sneaker, T-shirt/top, Trouser] to improve comparability with ROC-AUC values of the image-based baseline, OC-SVM, in [66].

This model employed an RBF kernel and used the raw input, i.e., the flattened array representation of the image, to identify a set of closed regions in the input space containing most of the training samples. Any samples falling outside these regions were classified as anomalous [66]. An important consideration is that this model was provided with an unfair significant advantage by applying supervised hyperparameter optimization.

We observed similar performance patterns to those seen with MNIST. Once again, the image-based baseline slightly outperformed the best graph-based configuration by 0.3%pt. Additionally, the performance gap between k -NN and RAG edge construction narrows for patch-segmented graphs compared to MNIST, while k -NN continues to offer a significant improvement for graphs derived from SLIC segmentation. The performance of other configurations slightly improved relative to MNIST, which may be attributed to the increased node count.

Consistent with the MNIST results, the semi-supervised experiments on Fashion-MNIST (see Table 4.13) identified the GAE (with patched segmentations and k -NN

4 Evaluation

c_i	GRAPH AUTOENCODER				OCGTL				DEEP
	Patch		SLIC		Patch		SLIC		SAD
	RAG	k -NN	RAG	k -NN	RAG	k -NN	RAG	k -NN	[6]
0	98.4±0.3	98.7±0.2	95.4±0.6	98.4±0.3	98.7±0.2	98.3±0.1	93.6±1.5	95.7±0.3	—
1	89.0±7.9	95.9±1.0	85.6±2.6	89.8±2.3	80.6±5.3	84.8±2.5	54.8±1.6	59.6±1.5	—
2	92.6±0.4	93.7±0.3	85.5±1.1	91.8±0.3	87.3±1.0	88.8±1.1	68.9±3.9	68.1±1.9	—
3	95.9±0.5	96.1±0.2	92.3±0.3	93.9±0.5	90.0±0.7	93.1±0.7	72.3±2.2	76.3±2.1	—
4	92.2±0.8	93.4±1.0	88.5±2.5	90.4±0.7	86.6±1.1	85.7±2.5	72.5±0.9	79.5±2.3	—
5	89.4±1.5	87.3±0.3	93.5±1.5	90.6±1.4	89.0±1.7	85.8±1.0	76.9±1.4	81.3±0.3	—
6	85.2±1.3	87.6±1.2	74.5±2.3	82.6±0.6	75.7±7.7	74.0±1.0	57.0±6.5	57.4±3.7	—
7	98.5±0.1	98.6±0.1	94.9±0.7	96.9±0.3	96.4±0.6	96.5±0.2	88.9±1.7	90.7±2.0	—
8	95.0±0.5	95.7±0.5	90.3±1.2	93.9±1.3	84.7±3.7	83.4±2.0	58.1±4.4	61.7±3.8	—
9	99.1±0.2	99.1±0.2	97.2±0.6	98.6±0.3	96.6±0.7	97.1±0.3	86.3±3.1	88.2±1.1	—
avg	93.5	94.6	89.8	92.7	88.6	88.7	72.9	75.9	87.3

Table 4.13: Semi-supervised setup: Average ROC-AUC and StdDev values in % for the Fashion-MNIST dataset over 5 seeds.

edges) as the top performing model, generally outperforming OCGTL. While the semi-supervision benefited both methods, the improvement was more significant for OCGTL (6–11%pt) than for GAE (2–6%pt). As illustrated in Figure 4.7, further increases of labeled anomalies to $1/3$ provided minimal benefits to GAE and, in some instances, performed worse. In contrast, OCGTL continued to show improvements. This discrepancy further highlights OCGTL’s disadvantage when processing image-derived graphs without the guidance of labels or inherent positional information. Although Figure 4.7 showcases the potential for further improvements with more training epochs, we opted to maintain the current training duration for consistent comparison to unsupervised results.

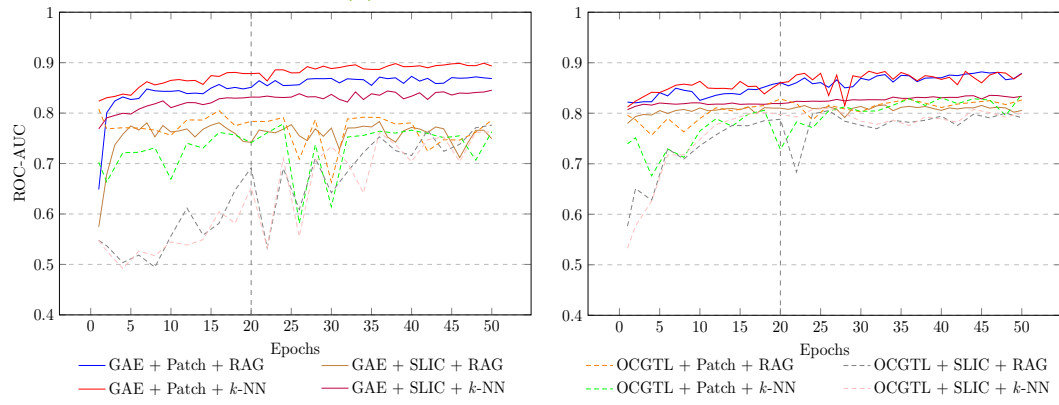
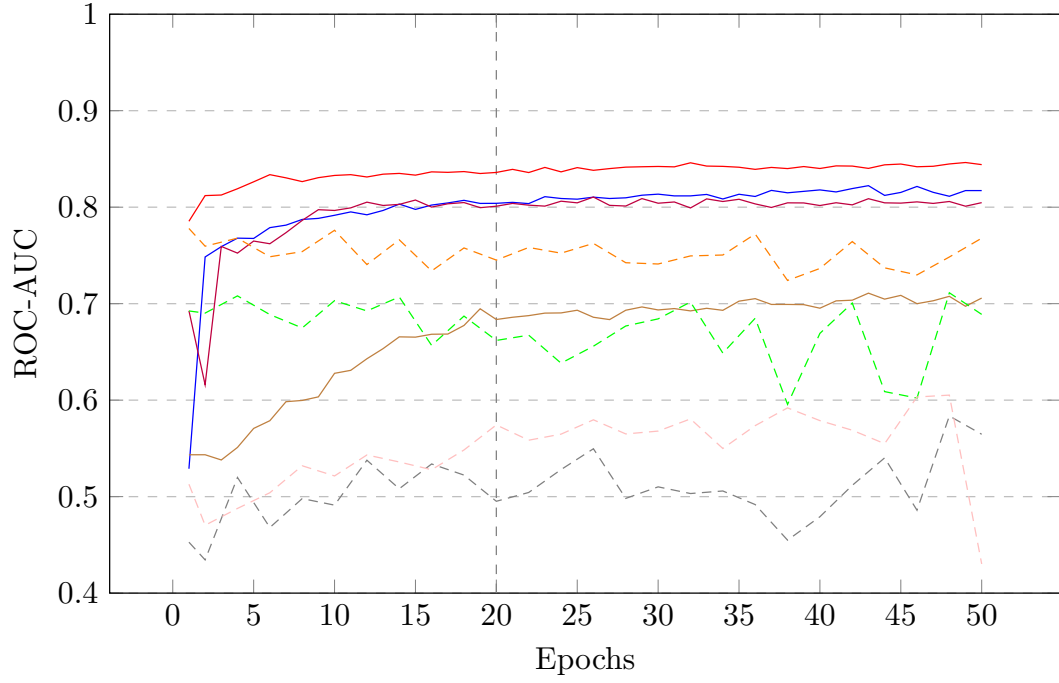


Figure 4.7: Comparison of configuration trained on Fashion-MNIST with normal class “Shirt” (c_6) over 50 epochs.

4.2.3 CIFAR-10

c_i	GRAPH AUTOENCODER				OCGTL				DEEP
	Patch		SLIC		Patch		SLIC		SVDD
	RAG	k -NN	RAG	k -NN	RAG	k -NN	RAG	k -NN	[5]
0	<u>74.0±1.1</u>	78.4±0.5	55.0±3.1	72.3±3.0	43.0±2.5	45.2±2.7	43.9±4.3	44.5±4.1	61.7±4.1
1	71.7±1.1	59.3±5.1	59.6±1.6	59.8±2.0	60.7±1.9	60.2±1.6	53.3±0.6	54.5±0.7	<u>65.9±2.1</u>
2	49.6±5.9	62.5±1.3	47.3±0.5	<u>56.3±3.1</u>	49.6±0.9	50.6±0.9	49.3±1.7	48.8±1.7	<u>50.8±0.8</u>
3	55.8±6.6	56.9±4.6	<u>57.6±2.5</u>	57.5±3.5	55.1±1.6	55.9±0.2	54.1±0.6	55.5±0.8	59.1±1.4
4	<u>63.7±4.0</u>	66.7±1.5	50.0±2.0	62.5±3.2	52.5±1.3	50.1±1.2	57.9±3.8	55.2±3.3	60.9±1.1
5	55.3±6.4	57.9±2.7	58.9±2.5	<u>59.3±1.3</u>	56.6±0.8	57.1±1.2	54.3±1.1	55.6±1.1	65.7±2.5
6	60.5±2.7	<u>63.5±0.6</u>	59.4±3.7	61.8±2.1	57.9±0.5	57.0±1.2	59.8±3.6	57.6±4.8	67.7±2.6
7	68.4±1.5	65.1±5.0	67.6±2.5	65.9±0.8	56.5±1.3	56.1±0.4	55.7±0.8	55.6±1.2	<u>67.3±0.9</u>
8	83.9±0.2	<u>79.7±1.1</u>	58.1±3.1	74.5±2.6	52.4±2.2	53.7±2.5	50.8±8.2	49.6±9.3	75.9±1.2
9	72.8±0.3	<u>67.0±4.9</u>	65.5±3.3	67.8±1.8	62.7±1.0	62.9±0.6	59.4±1.0	60.7±2.2	73.1±1.2
avg	<u>65.6</u>	65.7	57.9	63.8	54.7	54.9	53.9	53.7	64.8

Table 4.14: Unsupervised setup: Average ROC-AUC and StdDev values in % for the CIFAR-10 dataset over 5 seeds.

Table 4.14 demonstrates the unsupervised results obtained from CIFAR-10. Unlike with Fashion-MNIST, the `torchvision` implementation of CIFAR-10 already sorts the classes in alphabetical order: [Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck].

Here, our best graph-based approach slightly outperformed the image-based Deep SVDD method. As observed in previous datasets, the GAE combined with patch segmentation and k -NN edges proved the most effective graph approach. Nonetheless, CIFAR-10 presented unique behaviors: Within the GAE/Patch setup, RAG edges unexpectedly achieved significantly better AUC scores when compared to k -NN for c_1 (Automobile). Conversely, for c_2 (Bird), the same GAE configuration sometimes performed worse than random guessing. This issue of sub-random performance was even more pronounced with OCGTL, occurring in up to three normal classes.

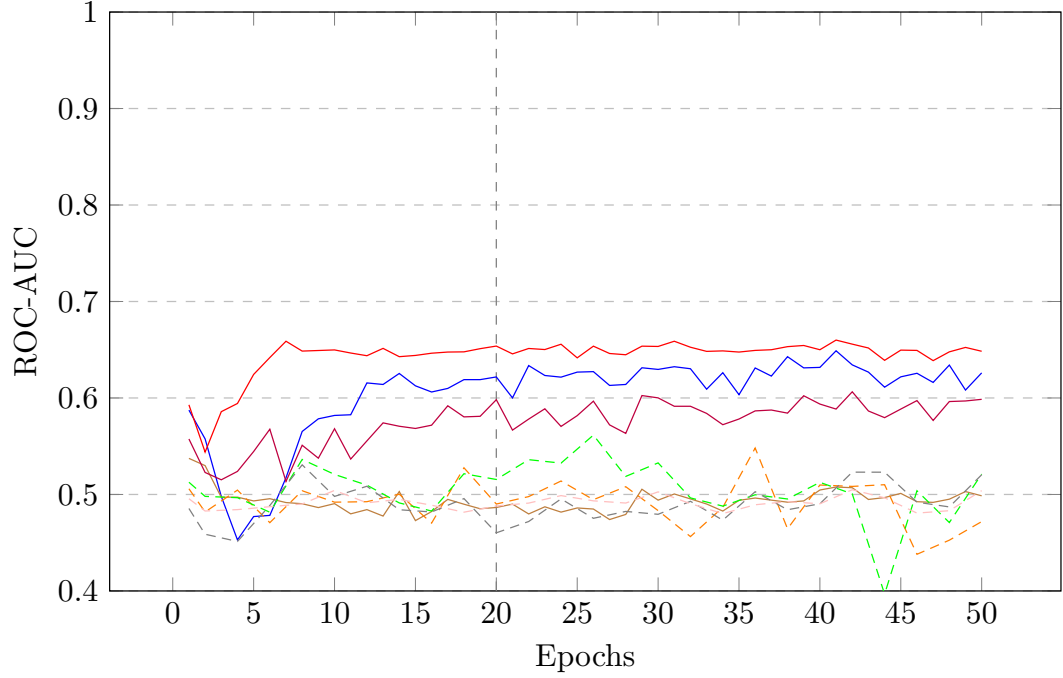
While graph-based models outperformed the image-based baselines without supervision, they did not manage to repeat this feat in our semi-supervised experiments. As presented in Table 4.15, Deep SAD, even with a polluted training dataset, outperformed the best graph-based configuration by 3.6%pt. Nevertheless, semi-supervision on CIFAR-10 led to an improvement of 7–11%pt for GAE, while OCGTL improved by 1–2%pt.

These results show that when the discriminative factors between classes of color images primarily lie in the structure of objects, OCGTL performs rather poorly. Figure 4.8a shows that with 5% labeled anomalies, extending the training epochs

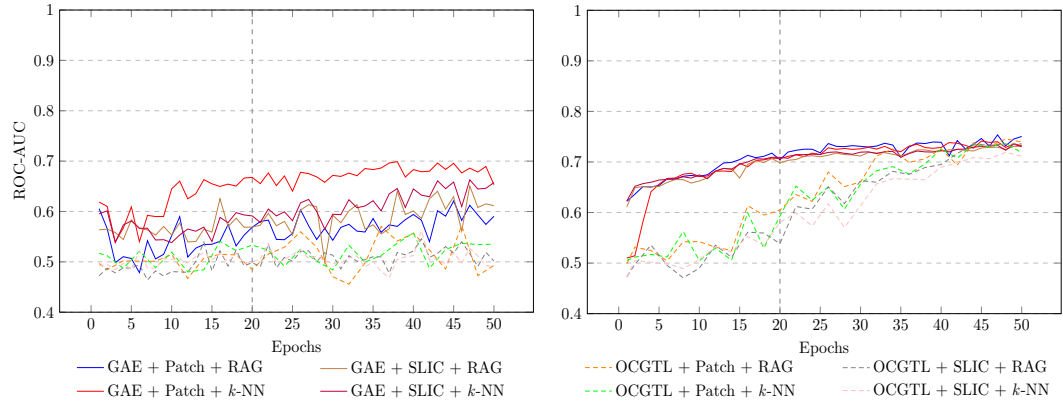
c_i	GRAPH AUTOENCODER				OCGTL				DEEP
	Patch		SLIC		Patch		SLIC		SAD
	RAG	k -NN	RAG	k -NN	RAG	k -NN	RAG	k -NN	[6]
0	80.9±0.6	83.3±0.5	76.0±2.0	<u>81.2±2.4</u>	45.6±3.4	45.1±4.9	43.9±1.7	45.4±5.7	—
1	<u>78.4±1.8</u>	81.3±1.9	76.3±2.7	<u>76.4±2.3</u>	63.7±2.2	61.5±1.8	54.2±2.8	56.4±1.8	—
2	55.4±3.0	66.8±1.4	57.4±2.5	<u>60.4±3.0</u>	51.2±4.8	51.8±4.0	49.4±1.5	50.6±2.6	—
3	60.7±3.3	68.1±1.4	60.5±1.4	<u>63.5±1.0</u>	55.0±1.2	55.0±1.7	52.7±1.7	54.8±2.3	—
4	73.8±1.7	78.5±0.5	72.2±2.1	<u>75.4±1.3</u>	56.1±1.6	52.1±1.2	59.3±0.9	56.1±0.8	—
5	<u>62.8±2.9</u>	69.0±6.0	61.7±1.1	<u>62.0±1.1</u>	57.6±1.2	57.7±1.0	55.8±1.0	56.5±1.1	—
6	<u>75.0±2.4</u>	76.4±1.7	71.8±1.7	<u>70.4±1.5</u>	58.4±1.7	57.7±2.5	61.5±2.3	60.1±2.6	—
7	<u>75.9±2.4</u>	78.3±2.3	69.1±1.5	<u>68.1±3.3</u>	57.0±1.2	56.0±0.9	59.2±1.3	59.6±0.6	—
8	84.1±3.5	<u>83.8±2.0</u>	74.0±4.4	<u>80.3±1.4</u>	57.6±4.0	55.4±3.6	59.4±6.9	59.7±5.8	—
9	<u>77.7±1.3</u>	82.2±0.9	76.7±1.5	<u>80.5±1.4</u>	63.8±1.5	65.6±1.4	61.4±2.3	63.1±1.1	—
avg	72.5	<u>76.8</u>	69.6	<u>71.8</u>	56.6	55.8	55.7	56.2	80.4

Table 4.15: Semi-supervised setup: Average ROC-AUC and StdDev values in % for the CIFAR-10 dataset over 5 seeds.

on GAE can lead to minor improvements. However, these gains are limited by the small amount of labeled data (5000 unlabeled normal, ≈ 262 labeled anomalies). As demonstrated by Figure 4.8b, with a considerable proportion of labeled anomalies ($1/3$), both GAE and OCGTL configurations show a more noticeable trend of performance gain over more epochs.



(a) Unsupervised ROC-AUC values



(b) Semi-supervised ROC-AUC values with (1) $1/20$ and (2) $1/3$ labeled anomalies.
See Table B.1 for concrete results of (2).

Figure 4.8: Comparison of configurations trained on CIFAR-10 with normal class “Bird” (c_2) over 50 epochs.

4.2.4 HAM10000

Fixed	Modified	GRAPH AUTOENCODER				OCGTL			
		Patch		SLIC		Patch		SLIC	
		RAG	k-NN	RAG	k-NN	RAG	k-NN	RAG	k-NN
Mask ✗	RGB _{avg}	73.4±1.7	<u>69.6±2.3</u>	51.7±2.5	63.8±0.8	63.5±2.1	66.6±1.3	58.9±2.2	60.4±1.6
Virtual nodes ✗	Color	<u>74.7±0.9</u>	74.8±1.2	65.1±1.5	67.0±1.6	65.6±1.3	64.5±1.0	63.1±1.8	61.1±2.5
NNConv (GAE)	Texture	58.9±0.9	59.9±1.1	<u>59.4±1.0</u>	58.0±0.4	54.0±2.7	55.6±2.4	59.9±1.4	56.2±4.4
GINConv (OCGTL)	Shape	—	—	69.7±0.7	<u>68.3±0.7</u>	—	—	62.0±3.4	61.5±1.0
	All	68.5±0.5	70.1±0.6	68.5±0.5	67.1±0.1	62.1±1.7	62.9±1.6	62.7±2.6	61.2±2.4
	Best	<u>74.7</u>	74.8	69.7	68.3	65.6	66.6	63.1	61.5
Mask ✗	NNConv	<u>74.7±0.9</u>	74.8±1.2	65.1±1.5	67.0±1.6	—	—	—	—
Virtual nodes ✗	SplineConv	<u>74.7±0.7</u>	75.2±0.8	65.5±1.3	67.6±1.7	—	—	—	—
Color (Features)	GINConv	—	—	—	—	65.6±1.3	<u>64.5±1.0</u>	63.1±1.8	61.1±2.5
	GATConv	—	—	—	—	<u>54.7±2.4</u>	56.3±2.6	53.3±1.1	53.3±1.8
	Best	<u>74.7</u>	75.2	65.5	67.6	65.6	64.5	63.1	61.1
Mask ✓									
Color (Features)	Virtual nodes ✗	64.3±0.6	63.2±0.6	72.6±0.8	<u>72.0±0.7</u>	61.8±1.1	59.5±1.3	63.7±2.1	63.0±5.2
NNConv (GAE)	Virtual nodes ✓	64.3±0.9	63.3±0.6	74.4±1.0	<u>75.2±1.0</u>	62.1±0.7	59.5±1.6	73.2±1.4	75.4±2.6
GINConv (OCGTL)									
	Best	64.3	63.3	74.4	<u>75.2</u>	62.1	59.5	73.2	75.4

Table 4.16: Unsupervised setup: Average ROC-AUC and StdDev values in % for the HAM10000 dataset on normal class “nv” over 5 seeds.

Fixed	Modified	GRAPH AUTOENCODER				OCGTL			
		Patch		SLIC		Patch		SLIC	
		RAG	k-NN	RAG	k-NN	RAG	k-NN	RAG	k-NN
Mask ✗	RGB _{avg}	53.2±1.6	<u>50.8±1.1</u>	33.8±1.9	45.8±0.5	41.9±2.0	44.9±0.9	38.1±1.3	38.7±1.3
Virtual nodes ✗	Color	52.1±0.6	<u>52.0±1.7</u>	44.9±1.4	45.7±1.6	45.0±1.3	44.1±1.5	41.7±1.3	40.1±1.0
NNConv (GAE)	Texture	37.8±0.7	38.2±0.7	40.1±1.2	37.4±0.3	35.5±2.0	36.7±1.3	<u>39.8±1.4</u>	37.2±2.4
GINConv (OCGTL)	Shape	—	—	49.6±0.8	<u>46.8±0.8</u>	—	—	40.7±2.1	40.7±1.0
	All	44.4±0.5	45.5±0.7	45.5±0.5	44.2±0.2	41.0±1.6	42.2±1.8	43.5±2.0	40.8±1.3
	Best	53.2	<u>52.0</u>	49.6	46.8	45.0	44.9	43.5	40.8
Mask ✗	NNConv	51.7±0.5	52.5±1.4	45.1±0.8	45.5±0.5	—	—	—	—
Virtual nodes ✗	SplineConv	<u>51.9±0.4</u>	52.7±1.5	45.2±1.2	45.9±1.5	—	—	—	—
Color (Features)	GINConv	—	—	—	—	44.9±1.9	<u>43.6±1.1</u>	42.4±1.1	40.2±0.9
	GATConv	—	—	—	—	<u>38.9±1.9</u>	40.0±1.9	38.3±0.6	38.1±1.6
	Best	<u>51.9</u>	52.7	45.2	45.9	44.9	43.6	42.4	40.2
Mask ✓									
Color (Features)	Virtual nodes ✗	48.0±0.8	47.6±0.8	53.3±0.6	<u>51.2±0.6</u>	43.0±1.2	41.7±1.0	46.7±1.6	45.9±4.3
NNConv (GAE)	Virtual nodes ✓	48.0±1.0	47.5±1.1	52.7±1.1	54.2±1.4	43.2±0.9	41.3±1.0	<u>55.0±1.4</u>	57.6±3.9
GINConv (OCGTL)									
	Best	48.0	47.6	53.3	54.2	43.2	41.7	<u>55.0</u>	57.6

Table 4.17: Unsupervised setup: Average PR-AUC and StdDev values in % for the HAM10000 dataset on normal class “nv” over 5 seeds.

HAM10000 is a unique dataset in that all classes exhibit some form of homogeneity in structure, i. e., all skin lesions display surface irregularities without a consistent or well-defined structure. As a result, structural differences are smaller factors when distinguishing between classes compared to previously examined datasets. Therefore, we expected closer AUC scores between GAE and OCGTL for our unsupervised

experiments.

Our ablation study results in Table 4.16 and 4.17 reveal several noteworthy discoveries.

1. Among the node features we evaluated, the PCA reduced color features (RGB, HSV, CIELAB) introduced in Chapter 3.1.4 performed the best on our standard setup with an ROC-AUC value of 74.8%. On the other hand, texture features from Chapter 3.1.4 barely achieved ROC-AUC values above 54%. This finding suggests that texture does not play a significant role in identifying malignant skin lesions. Supporting this, aggregating texture features with color (and shape) features also lead to decreased performance compared to using color (or shape) features alone.
2. Combining color features with different graph convolutions resulted in minor improvements of 0.4%pt for GAE with SplineConv but significantly worse performance of -10% pt for OCGTL with GATConv.
3. Regarding masked segmentation, graphs constructed from patches significantly underperformed compared to those without masks by dropping -11.5% pt. Conversely, SLIC segmentation provided noticeable improvements of 14%pt. This contrast is logical since patch segmentation removes surrounding skin information (i.e., while the graph nodes cover the whole image, informative features are only taken from within the mask). SLIC, on the other hand, focuses its segmentation directly on the masked lesion (≈ 300 nodes on the lesion). The addition of virtual nodes offered minor improvements for GAE. OCGTL, however, was able to fully realize its potential with these additional nodes and edges, obtaining the best overall ROC-AUC value.
4. The performance metrics revealed a significant disparity between ROC-AUC and PR-AUC. While the optimal configuration (GINConv-based OCGTL with masked SLIC segmentation, virtual nodes, k -NN edges, and PCA reduced color features) obtained a 75.4% ROC-AUC, its PR-AUC was only 57.6%. However, accounting for the test dataset’s imbalance with 32.65% abnormal samples, a random classifier provides a baseline PR-AUC of 32.65% (see Chapter 2.4.4). In this context, the observed PR-AUC values, like the 57.6% from the optimal configuration, are well above random performance despite their moderate value.

Adding semi-supervision paints a similar picture. All previous points for the unsupervised setup still stand for semi-supervised results presented in Table 4.18 and 4.19. OCGTL, especially when combining SLIC with virtual nodes, improved considerably. This setup achieved a ROC-AUC score of 86.7%, increasing its performance by 11.3%pt over its unsupervised counterpart. GAE also benefited from the labeled

data and reached a peak ROC-AUC value of 78.1%, representing a 2.9%pt increase. This gain, however, is minor when compared to that observed for OCGTL.

Figure 4.9 illustrates some strengths and weaknesses of the various models when applied to this dataset. Without supervision, the OCGTL model on SLIC segmentation, utilizing virtual nodes and k -NN edges, demonstrates a decline in performance over an extended training period. However, incorporating even a small number of labeled anomalies reverses this trend, turning this configuration into the best-performing one trained on 5% labeled anomalies. With GAE on patch-based k -NN graphs, we find continuously improving ROC-AUC values over the epochs for both unsupervised and semi-supervised modes. Nevertheless, the peak performance achieved by either GAE version does not reach the level of semi-supervised OCGTL. This performance hierarchy persists even when raising the ratio of labeled instances to $1/3$ with 5272 unlabeled normal, 2547 labeled anomalous, and 84 labeled normal data samples.

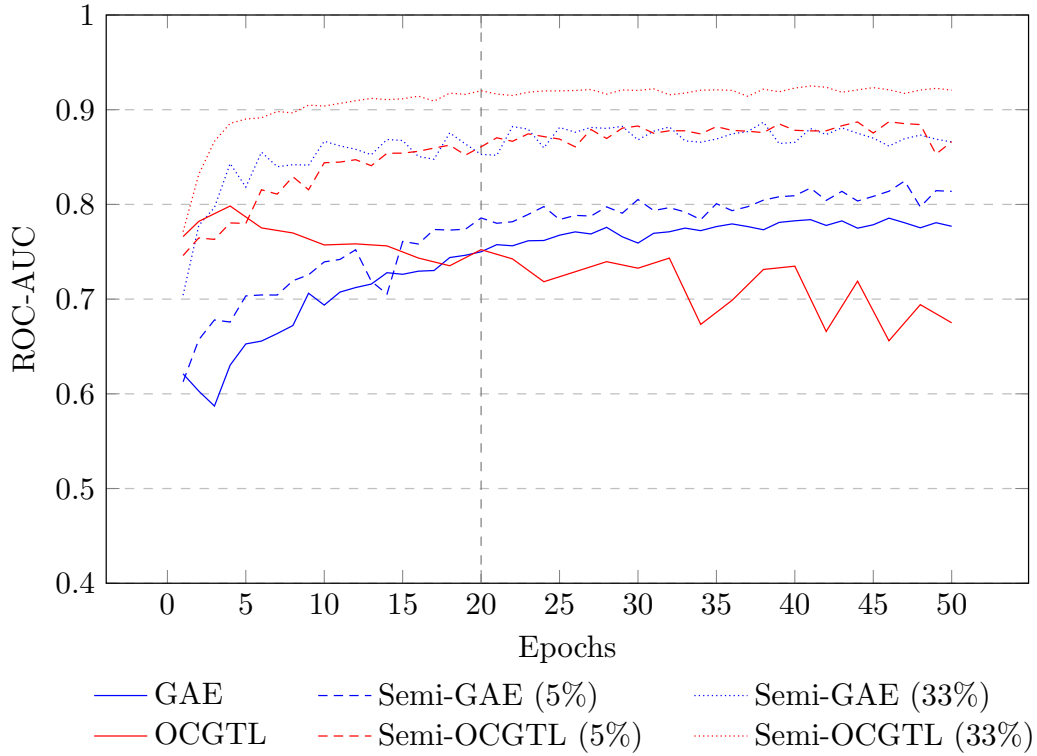


Figure 4.9: Comparison between unsupervised and semi-supervised ROC-AUC values of the best model configurations trained on HAM10000 with normal class “nv” over 50 epochs. See Table B.3 for concrete results of Semi-OCGTL (33%).

4 Evaluation

Fixed	Modified	GRAPH AUTOENCODER				OCGTL			
		Patch		SLIC		Patch		SLIC	
		RAG	k -NN	RAG	k -NN	RAG	k -NN	RAG	k -NN
Mask ✗	RGB _{avg}	<u>71.9±3.9</u>	76.3±1.4	52.7±6.2	71.8±4.7	62.0±3.1	65.7±3.8	60.2±1.0	61.9±1.2
Virtual nodes ✗	Color	<u>74.4±2.0</u>	77.6±0.7	71.2±1.9	73.1±1.6	69.2±6.1	69.6±4.4	68.7±0.8	65.3±2.3
NNConv (GAE)	Texture	59.0±7.5	62.3±4.1	61.5±1.4	59.9±2.0	<u>62.7±3.3</u>	59.0±4.7	68.8±2.0	62.3±6.6
GINConv (OCGTL)	Shape	—	—	68.6±0.3	<u>68.2±0.7</u>	—	—	65.7±2.1	64.8±2.6
	All	70.1±1.7	71.7±0.9	<u>70.3±0.5</u>	68.3±0.6	69.5±3.5	69.8±3.3	67.6±6.4	65.2±5.1
	Best	<u>74.4</u>	77.6	71.2	73.1	69.5	69.8	68.8	65.3
Mask ✗	NNConv	<u>74.4±2.0</u>	76.3±1.4	71.2±1.8	72.7±3.0	—	—	—	—
Virtual nodes ✗	SplineConv	<u>73.5±2.3</u>	78.1±0.8	71.6±0.9	72.9±2.1	—	—	—	—
Color (Features)	GINConv	—	—	—	—	<u>68.7±4.5</u>	69.8±4.7	66.3±2.0	65.6±1.2
	GATConv	—	—	—	—	<u>56.7±1.9</u>	57.6±2.1	54.4±1.4	56.1±1.8
	Best	<u>74.4</u>	78.1	71.6	72.9	68.7	69.8	66.3	65.6
Mask ✓	Color (Features)	69.9±1.5	71.1±1.8	76.8±0.9	77.8±1.3	69.8±2.5	67.6±4.6	76.2±4.2	75.0±3.8
NNConv (GAE)	Virtual nodes ✓	70.0±1.4	71.7±1.4	77.8±1.6	76.6±2.8	70.7±1.5	68.9±2.0	86.7±1.4	<u>86.6±0.4</u>
GINConv (OCGTL)		—	—	—	—	—	—	—	—
	Best	70.0	71.7	77.8	77.8	70.7	68.9	86.7	<u>86.6</u>

Table 4.18: Semi-supervised setup: Average ROC-AUC and StdDev values in % for the HAM10000 dataset on normal class “nv” over 5 seeds.

Fixed	Modified	GRAPH AUTOENCODER				OCGTL			
		Patch		SLIC		Patch		SLIC	
		RAG	k -NN	RAG	k -NN	RAG	k -NN	RAG	k -NN
Mask ✗	RGB _{avg}	52.2±3.3	54.1±1.9	35.1±4.8	<u>52.3±5.1</u>	41.2±3.2	44.2±2.0	39.5±0.5	40.0±0.8
Virtual nodes ✗	Color	51.8±2.4	56.0±1.2	50.8±2.1	51.5±1.6	52.6±8.9	<u>53.1±6.4</u>	49.9±1.2	44.9±1.3
NNConv (GAE)	Texture	38.0±3.9	40.2±3.4	41.5±1.8	38.7±2.1	<u>45.5±3.7</u>	42.4±5.4	51.4±4.0	45.4±7.3
GINConv (OCGTL)	Shape	—	—	47.8±0.5	<u>46.2±0.6</u>	—	—	45.5±3.6	45.3±2.9
	All	45.5±1.4	46.9±0.8	46.5±0.4	<u>44.9±0.5</u>	53.8±6.1	53.8±4.3	<u>48.7±7.1</u>	45.4±5.5
	Best	<u>52.2</u>	56.0	50.8	52.3	<u>53.8</u>	<u>53.8</u>	51.4	45.4
Mask ✗	NNConv	<u>52.5±1.6</u>	55.8±1.3	50.7±1.7	51.3±2.9	—	—	—	—
Virtual nodes ✗	SplineConv	<u>51.7±2.3</u>	56.8±1.3	51.4±0.8	51.3±2.1	—	—	—	—
Color (Features)	GINConv	—	—	—	—	<u>52.1±6.3</u>	53.3±6.9	46.6±2.8	44.6±1.2
	GATConv	—	—	—	—	<u>40.4±1.5</u>	41.2±1.6	39.1±1.3	40.0±1.7
	Best	51.7	55.8	51.4	51.3	52.1	<u>53.3</u>	46.6	44.6
Mask ✓	Color (Features)	55.1±1.0	57.0±2.1	58.0±1.3	57.3±1.5	54.1±2.5	51.2±4.2	64.0±5.5	61.8±4.5
NNConv (GAE)	Virtual nodes ✓	55.1±0.8	58.2±2.0	59.3±2.3	57.0±4.6	55.3±1.6	51.5±2.7	<u>76.9±2.4</u>	77.2±0.5
GINConv (OCGTL)		—	—	—	—	—	—	—	—
	Best	55.1	58.2	59.2	57.3	55.3	51.5	<u>76.9</u>	77.2

Table 4.19: Semi-supervised setup: Average PR-AUC and StdDev values in % for the HAM10000 dataset on normal class “nv” over 5 seeds.

4.3 Runtime Analysis

	Model	Method	Edges	RUNTIME [in s]		
				Preprocessing	Training & Evaluation	Σ
MNIST	GAE	Patch	RAG	46.97	36.51 (40.90)	83.48 (87.87)
	GAE	Patch	k -NN	29.97	76.25 (82.28)	106.22 (112.25)
	GAE	SLIC	RAG	71.42	52.02 (53.07)	123.44 (123.49)
	GAE	SLIC	k -NN	44.65	105.89 (108.09)	150.54 (154.80)
	OCGTL	Patch	RAG	46.97	110.15 (142.16)	157.12 (189.13)
	OCGTL	Patch	k -NN	29.97	126.45 (165.35)	156.42 (195.32)
	OCGTL	SLIC	RAG	71.42	146.46 (157.33)	217.88 (228.75)
	OCGTL	SLIC	k -NN	44.65	187.85 (182.67)	232.50 (227.32)
	Deep SVDD	—	—	—	36.23 + 22.98	59.21
Fashion-MNIST	GAE	Patch	RAG	64.24	104.11 (113.50)	168.35 (177.74)
	GAE	Patch	k -NN	37.27	224.08 (238.72)	261.35 (275.99)
	GAE	SLIC	RAG	79.44	114.61 (118.99)	194.05 (198.43)
	GAE	SLIC	k -NN	46.75	207.19 (225.04)	253.94 (271.79)
	OCGTL	Patch	RAG	64.24	165.23 (192.35)	229.47 (256.59)
	OCGTL	Patch	k -NN	37.27	197.87 (221.22)	235.14 (258.48)
	OCGTL	SLIC	RAG	79.44	150.96 (151.48)	230.40 (230.92)
	OCGTL	SLIC	k -NN	46.75	185.40 (178.86)	232.15 (225.61)
	Deep SVDD	—	—	—	31.96 + 19.96	51.92
CIFAR-10	GAE	Patch	RAG	165.24	286.87 (314.65)	452.11 (479.89)
	GAE	Patch	k -NN	109.54	604.61 (623.74)	714.15 (733.28)
	GAE	SLIC	RAG	180.65	386.82 (289.77)	567.47 (470.42)
	GAE	SLIC	k -NN	121.12	618.40 (605.44)	739.52 (726.56)
	OCGTL	Patch	RAG	165.24	337.52 (326.23)	502.76 (491.47)
	OCGTL	Patch	k -NN	109.54	380.35 (393.47)	489.89 (503.01)
	OCGTL	SLIC	RAG	180.65	320.80 (316.96)	501.45 (497.61)
	OCGTL	SLIC	k -NN	121.12	371.29 (371.92)	492.41 (493.04)
	Deep SVDD	—	—	—	171.34 + 108.02	279.36
HAM10000	GAE	Patch	RAG	977.09	95.04 (115.38)	1072.13 (1092.47)
	GAE	Patch	k -NN	1002.59	139.00 (156.27)	1141.59 (1158.86)
	GAE	SLIC	RAG	1323.54	108.76 (128.36)	1432.30 (1451.90)
	GAE	SLIC	k -NN	1335.21	140.89 (160.95)	1476.10 (1496.16)
	OCGTL	Patch	RAG	977.09	377.35 (398.02)	1354.43 (1375.10)
	OCGTL	Patch	k -NN	1002.59	379.12 (402.36)	1381.71 (1404.95)
	OCGTL	SLIC	RAG	1323.54	381.81 (402.82)	1705.35 (1726.36)
	OCGTL	SLIC	k -NN	1335.21	384.01 (404.98)	1719.22 (1740.12)

Table 4.20: Runtime of preprocessing, training, and final evaluation for all configurations (averaged over all chosen normal classes and seeds). Semi-supervised training and evaluation times are indicated in parentheses.

Table 4.20 presents the total runtime for preprocessing all dataset images into graphs, training the anomaly detection models for 20 epochs, and the final evaluation for all models. These runtimes were achieved by appointing the mean RGB values for

node features and utilizing NNConv and GINConv as graph convolutions for GAE and OCGTL, respectively. Deep SVDD was run on torchvision datasets to deliver a comparative image-based baseline. We utilized the implementation provided by Ruff et al.¹, training the CAE for 20 epochs and optimizing Deep SVDD for another 20 epochs.

Considering the model runtimes, when trained for the same amount of epochs, Deep SVDD trains the fastest, even with pretraining. This speed is likely due to better optimization of the Euclidean convolution operation. Among graph-based models, GAE trains faster than OCGTL on the MNIST dataset, especially with graphs containing fewer edges. For GAE specifically, switching from RAG to k -NN graphs, which approximately doubles the edge count (for $k = 8$), also approximately doubles the training time on the CPU. This proportional increase occurs from the computation of reconstructed edge attributes of all edges and evaluating the associated loss function. The doubling of runtime is particularly disadvantageous for GAE compared to OCGTL on other datasets. While OCGTL is still slower than GAE on graphs with RAG edges, the effect of k -NN is less noticeable on the runtime. Consequently, OCGTL is faster on these larger graphs with k -NN edges than GAE when running on the processor.

As described in Chapter 4.1.4, the experiments on HAM10000 were conducted with GPU acceleration via CUDA. Compared to the CPU runtimes, we observe less of an increase from RAG to k -NN and faster runtimes on average compared to Fashion-MNIST for GAE and comparable runtimes compared to CIFAR-10 for OCGTL.

Dataset	Method	RUNTIMES [ms]			
		Segmentation	Mean RGB	RAG	KNN
MNIST	SLIC	0.729±0.061	1.317±0.133	2.038±0.165	0.254±0.021
	Patch	0.021±0.005	1.121±0.794	1.608±0.988	0.234±0.143
Fashion-MNIST	SLIC	0.779±0.087	1.408±0.122	2.306±0.172	0.277±0.030
	Patch	0.037±0.007	1.536±0.068	1.919±0.163	0.255±0.090
CIFAR-10	SLIC	1.101±0.035	5.596±0.209	6.160±0.274	0.654±0.072
	Patch	0.087±0.006	5.689±0.165	5.813±0.248	0.531±0.063
HAM10000	SLIC	145.836±2.595	338.404±30.285	15.531±0.755	1.043±0.366
	Patch	0.263±0.078	371.200±22.271	13.751±0.472	1.089±0.348

Table 4.21: Average and StdDev preprocessing times of one image over 32 random samples

¹Available at: <https://github.com/lukasruff/Deep-SVDD-PyTorch>

The preprocessing step is independent of model training. This independence allows the resulting graph dataset to be stored and reused, avoiding redundant computation. The reported times in Table 4.21 were obtained through parallelization using 5 CPU cores on an Apple M3 Pro.

Table 4.21 presents a detailed breakdown of these preprocessing runtimes per image. Two key trends can be observed: First, patch-based segmentation is consistently faster than SLIC across all datasets due to its simpler implementation. Second, constructing RAGs is generally more computationally demanding than building k -NN graphs. This difference arises because RAG considers all pixel coordinates for edge connections, whereas k -NN only requires the centroid coordinates of segmented regions. However, for high-resolution images like those in HAM10000, the computation of node features becomes the dominant factor in the overall runtime. Consequently, the RAG and k -NN edge construction variants require similar total preprocessing durations for this dataset, showing only minor variance.

5 Conclusion and Outlook

This thesis examined whether image-derived graphs powered by graph-level anomaly detection frameworks could compete with image-based anomaly detectors. We identified two bottlenecks that could affect the performance of our methodology: The image-to-graph transformation and the graph anomaly detection. Within the image-to-graph pipeline, we presented two different ways to appoint the nodes of the graphs: superpixels with SLIC and patch-wise decomposition, as well as multiple ways to construct edges, among which are the region adjacency graph and the k -NN method. Further extensions, such as different kinds of node and edge features, were introduced in Chapter 3.1 and analyzed in Chapter 4.2.4. We studied two frameworks for anomaly detectors: One-class classification through OCGTL and the reconstruction-based graph autoencoder with edge attributes and their performance with different graph convolutions. Notably, we hypothesized that including and utilizing the two-dimensional Euclidean coordinates would lead to significant improvements over methods that did not.

Our experimental findings concluded that although graph-based anomaly detection on images did not always outperform similar image-based methods, they came very close. However, this comparable performance often came at the cost of increased runtime. We observed substantial preprocessing time to convert images into graphs, a step unnecessary for image-based detectors. Furthermore, the graph-based models themselves were generally slower. Conversely, one notable advantage was the ability to significantly reduce data size through the graph conversion process while maintaining performance. Beyond this, our results provided insights into specific graph-based approaches, including evidence supporting the hypothetical advantages of GAE over OCGTL using relative Cartesian coordinates as edge attributes, and the unexpected finding that SLIC-based graphs performed worse than patch-based graphs on `torchvision` datasets.

The model architecture also allowed for the relatively simple incorporation of semi-supervision objectives inspired by Deep SAD. We showed that the inclusion of just 5% labeled anomalies could drastically improve the anomaly detection rate and lead to improvements over longer epochs. At the same time, we demonstrated that on specialized datasets such as HAM10000, OCGTL with specific graph constructions involving virtual nodes outclassed GAE in a semi-supervised setting.

Retrospectively, there are various improvements to consider:

- Since SLIC segmentation does not always yield the exact specified number of segments, a modified algorithm could try to over-segment the image and merge color-similar superpixels until the exact number of segments is reached. This approach would resolve the issue of potentially dissimilar numbers of nodes, even among graphs within the same class.
- For patch decomposition, we could dynamically calculate the needed patch size to a specified number of segments, allowing for flexibility with datasets of images of varying dimensions. Image datasets with segmentation masks could also be handled elegantly by only considering the masked region.
- Semi-supervised performance could be further improved upon with hyperparameter tuning. Given labeled data, hyperparameters such as epochs, learning rate, and weight decay can be tuned via grid search or other optimization techniques. The semi-supervised objective function based on the Deep SAD approach could also be refined further through experimentation.

Future work stemming from this thesis can be explored in several different ways:

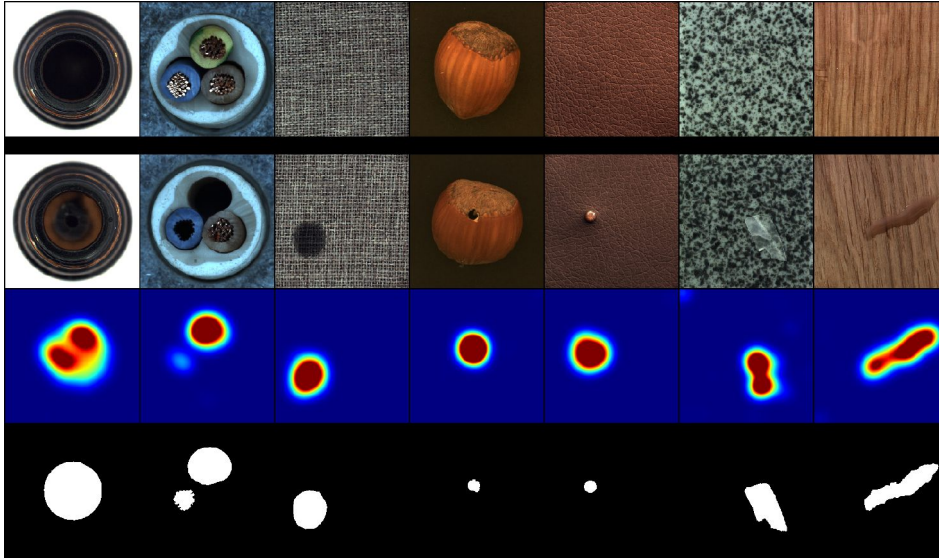


Figure 5.1: Anomaly localization on the MVTec dataset [67] with FCDD [68]

Explainable anomaly localization While our graph-based models offer an excellent approach to detecting anomalies, they suffer from limited explainability, i. e., they do not provide clear, conclusive reasons for why an image was classified as anomalous.

One emerging concept within explainable machine learning is the notion of anomaly localization [69] through anomaly maps (see Figure 5.1). In our thesis, the GAE provides limited interpretability that is accessible through the reconstruction error of node features. While reconstruction errors of node features can be associated with the individual segments they represent, high error values may not necessarily indicate anomalies within those segments themselves. Instead, these errors could result from the message passing process propagating aggregated features across neighboring regions. Another aspect to consider is that gaining an interpretable representation of the reconstruction error of edge attributes is more complex.

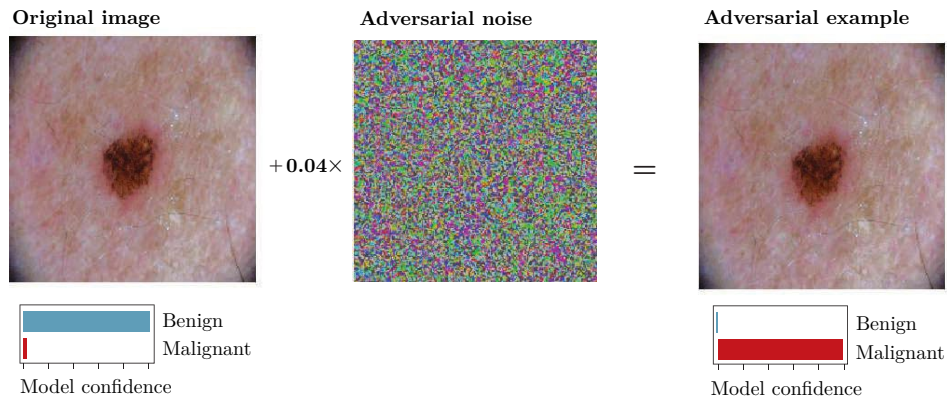


Figure 5.2: Example of how an adversarial attack might be executed on medical machine learning systems [70]

Adversarial attacks Adversarial attacks by applying hardly noticeable perturbation have been shown to be an effective vulnerability against deep learning models [71]. As illustrated in Figure 5.2, such vulnerabilities could lead image-based detectors to falsely classify benign moles as malignant. This kind of misclassification could have severe consequences, particularly for insurance companies that rely on machine learning to issue reimbursement decisions, as bad actors might potentially exploit these weaknesses [70]. An interesting consideration for further investigation is whether the aggregation of regional features, as done in our approach, could offer a robust defense against adversarial attacks.

Graph classification Some research [15], [17], [18] already investigated graph classification techniques applied to superpixel representations of images. The current thesis demonstrated that graphs created from simple grid patches performed better than those generated from more complex SLIC superpixels for anomaly detection tasks. These results pose the question of whether a more straightforward and regular

patch-based representation can also be impactful or superior to superpixels for graph-based image classification tasks. Alternatively, following [72], we could reformulate the One vs. Rest approach to output probabilities rather than anomaly scores. This modification would naturally extend our method to multiclass classification problems.

A PR-ROC Values on Torchvision Datasets

	c_i	GRAPH AUTOENCODER				OCGTL			
		Patch		SLIC		Patch		SLIC	
		RAG	k -NN	RAG	k -NN	RAG	k -NN	RAG	k -NN
MNIST	0	99.4±0.5	99.8±0.0	97.9±0.3	99.1±0.1	98.8±0.9	96.8±0.4	92.0±0.8	94.8±1.1
	1	99.8±0.1	99.9±0.0	98.8±0.2	98.9±0.3	98.9±0.9	99.9±0.0	98.1±0.2	99.2±0.1
	2	98.3±0.8	98.8±0.2	93.5±0.7	95.5±0.7	94.2±0.3	95.3±0.6	90.5±0.7	91.8±0.8
	3	98.0±0.4	98.9±0.1	94.1±0.1	96.7±0.4	95.8±0.1	96.7±0.3	91.1±0.4	92.5±0.6
	4	98.2±0.7	99.3±0.1	95.9±0.5	96.9±0.6	97.0±0.4	97.6±0.2	93.1±0.7	93.2±0.4
	5	97.0±0.9	98.1±0.2	93.9±0.4	95.4±0.2	94.4±1.3	94.3±0.6	92.1±1.4	93.1±0.6
	6	99.4±0.1	99.6±0.0	97.2±0.3	98.5±0.5	96.4±0.5	96.0±1.7	92.5±0.8	93.3±0.6
	7	99.0±0.2	99.0±0.1	97.3±0.2	98.2±0.1	95.9±0.3	96.9±0.3	93.7±0.8	93.9±0.8
	8	98.4±0.4	99.1±0.1	94.7±0.3	96.7±0.4	98.2±0.2	97.9±0.1	95.1±0.5	94.4±1.2
	9	98.9±0.2	99.3±0.1	97.0±0.1	98.3±0.1	97.4±0.2	97.8±0.1	93.5±0.5	94.1±0.3
	<i>avg</i>	98.6	99.2	96.0	97.4	96.7	96.9	93.2	94.0
Fashion-MNIST	0	99.8±0.0	99.8±0.0	99.5±0.1	99.8±0.0	99.6±0.0	99.8±0.0	95.2±0.5	98.4±0.3
	1	98.9±0.1	99.0±0.1	96.1±0.4	98.5±0.2	93.7±0.5	92.2±1.2	90.8±0.2	91.3±0.2
	2	98.9±0.0	98.9±0.0	97.3±0.2	98.8±0.1	97.4±0.2	97.0±0.1	92.2±0.4	93.1±0.3
	3	99.3±0.1	99.1±0.0	98.3±0.2	98.7±0.2	97.4±0.2	98.1±0.1	93.3±0.9	93.5±1.1
	4	98.9±0.0	98.9±0.0	97.3±0.3	98.5±0.2	96.3±0.1	93.9±0.4	90.6±0.5	91.9±0.3
	5	98.3±0.1	98.3±0.1	97.5±0.4	98.3±0.1	97.3±1.2	97.2±0.5	93.3±0.1	95.3±0.5
	6	97.2±0.1	97.9±0.1	94.8±0.4	97.4±0.1	96.1±0.3	94.1±0.1	89.5±0.9	90.7±0.9
	7	99.8±0.0	99.9±0.0	99.6±0.1	99.8±0.0	99.3±0.2	99.3±0.1	96.4±0.7	96.0±0.8
	8	98.7±0.2	98.6±0.1	96.7±0.2	98.4±0.1	95.9±0.3	94.7±0.6	90.4±0.2	90.7±0.4
	9	99.8±0.0	99.8±0.0	99.7±0.1	99.7±0.0	98.9±0.1	99.4±0.0	96.8±0.5	97.0±0.3
	<i>avg</i>	99.0	99.0	97.7	98.8	97.2	96.6	92.9	93.8
CIFAR-10	0	95.6±0.2	96.2±0.2	91.1±0.9	95.0±0.6	87.9±0.6	88.7±0.9	88.0±1.2	88.3±1.3
	1	95.5±0.2	92.2±1.0	93.0±0.4	92.4±0.5	93.3±0.4	93.2±0.4	90.9±0.2	91.5±0.1
	2	89.9±1.3	92.8±0.2	89.2±0.2	91.5±0.7	89.7±0.1	90.1±0.4	89.8±0.9	89.7±0.6
	3	91.8±1.4	91.9±1.4	92.4±0.6	92.1±0.9	91.4±0.4	91.8±0.0	91.3±0.3	91.8±0.3
	4	93.6±0.7	94.2±0.2	90.3±0.7	93.4±0.5	90.6±0.3	89.9±0.2	92.4±0.9	91.8±0.8
	5	91.8±1.3	92.3±0.7	92.7±0.6	92.5±0.3	92.2±0.2	92.3±0.3	91.7±0.3	92.0±0.4
	6	93.2±0.5	93.5±0.2	92.9±0.9	93.3±0.6	92.2±0.1	92.2±0.3	92.9±1.0	92.2±1.4
	7	94.7±0.2	94.0±1.2	94.7±0.5	94.1±0.2	92.2±0.2	92.0±0.1	92.1±0.3	92.1±0.3
	8	97.6±0.0	96.8±0.3	92.5±0.8	95.8±0.4	91.5±0.5	91.7±0.5	90.5±2.1	90.0±2.5
	9	95.8±0.2	94.4±0.8	94.3±0.7	94.6±0.4	93.8±0.3	93.8±0.2	92.8±0.3	93.1±0.5
	<i>avg</i>	94.0	93.8	92.3	93.5	91.5	91.6	91.2	91.3

Table A.1: Unsupervised setup: Average PR-AUC and StdDev values in % for torchvision datasets over 5 seeds. Values are over-inflated due to overrepresentation of positive (anomalous) samples.

	c_i	GRAPH AUTOENCODER				OCGTL			
		Patch		SLIC		Patch		SLIC	
		RAG	k -NN	RAG	k -NN	RAG	k -NN	RAG	k -NN
MNIST	0	99.9±0.1	99.9±0.0	99.5±0.1	99.4±0.2	<u>99.8±0.1</u>	99.2±0.4	96.1±1.2	96.8±0.4
	1	<u>99.9±0.0</u>	100.0±0.0	99.8±0.0	99.8±0.1	<u>99.9±0.0</u>	<u>99.9±0.0</u>	99.6±0.1	99.6±0.1
	2	99.7±0.1	<u>99.6±0.1</u>	98.1±0.2	97.0±0.2	94.9±1.0	97.0±0.7	92.5±0.8	92.9±0.6
	3	99.6±0.1	<u>99.5±0.1</u>	98.8±0.5	98.4±0.3	98.3±0.6	98.4±0.6	91.7±1.3	93.4±0.4
	4	<u>99.2±0.4</u>	99.6±0.1	98.3±0.4	98.1±0.4	98.7±0.2	98.8±0.2	95.3±0.9	94.9±0.7
	5	<u>99.2±0.4</u>	99.4±0.2	96.9±0.7	96.9±0.9	95.6±1.1	95.6±0.9	92.6±0.5	93.8±0.3
	6	99.9±0.1	<u>99.8±0.1</u>	99.1±0.2	98.5±0.4	98.5±0.3	99.1±0.1	93.3±2.0	94.6±1.0
	7	<u>99.5±0.1</u>	99.6±0.1	98.2±0.6	98.3±0.2	98.1±0.6	98.3±0.4	94.7±1.6	94.9±1.1
	8	<u>99.2±0.1</u>	99.3±0.3	98.3±0.2	97.6±0.4	99.1±0.1	98.4±0.3	95.3±0.7	95.0±0.9
	9	<u>99.2±0.2</u>	99.5±0.1	98.3±0.2	98.1±0.4	98.6±0.1	98.5±0.2	95.6±0.6	95.5±0.5
	<i>avg</i>	<u>99.5</u>	99.6	<u>98.5</u>	<u>98.2</u>	<u>98.2</u>	<u>98.3</u>	<u>94.7</u>	<u>95.2</u>
Fashion-MNIST	0	<u>99.8±0.0</u>	99.9±0.0	99.5±0.1	<u>99.8±0.0</u>	99.9±0.0	<u>99.8±0.0</u>	99.3±0.2	99.5±0.0
	1	98.6±1.1	99.5±0.1	98.0±0.5	<u>98.7±0.3</u>	96.7±0.7	97.9±0.4	91.2±0.3	92.1±0.3
	2	<u>99.1±0.1</u>	99.3±0.0	98.2±0.1	99.0±0.0	98.3±0.1	98.4±0.2	94.6±1.0	94.3±0.5
	3	99.5±0.1	99.5±0.1	98.9±0.1	99.1±0.1	98.6±0.1	99.0±0.1	95.6±0.5	96.3±0.5
	4	<u>99.1±0.1</u>	99.2±0.1	98.7±0.3	98.9±0.1	98.4±0.2	98.2±0.3	95.8±0.5	97.1±0.4
	5	98.8±0.2	98.6±0.0	99.3±0.2	<u>99.0±0.2</u>	98.7±0.2	98.4±0.1	97.0±0.2	97.7±0.1
	6	<u>98.0±0.2</u>	98.3±0.2	96.5±0.4	97.7±0.1	96.5±1.2	95.5±0.2	91.0±1.6	91.3±0.7
	7	99.8±0.0	99.8±0.0	99.4±0.1	<u>99.7±0.0</u>	99.6±0.1	99.6±0.0	98.7±0.2	98.9±0.3
	8	<u>99.3±0.1</u>	99.5±0.1	98.8±0.1	99.2±0.2	97.5±0.5	96.9±0.2	91.5±1.0	91.9±0.8
	9	99.9±0.0	99.9±0.0	99.6±0.1	<u>99.8±0.1</u>	99.6±0.1	99.6±0.0	98.2±0.4	98.5±0.1
	<i>avg</i>	<u>99.2</u>	99.3	<u>98.7</u>	<u>99.1</u>	<u>98.4</u>	<u>98.3</u>	<u>95.3</u>	<u>95.8</u>
CIFAR-10	0	<u>97.2±0.1</u>	97.4±0.1	96.3±0.3	97.0±0.4	88.5±1.0	88.7±1.1	88.1±0.6	88.6±1.7
	1	<u>96.8±0.3</u>	97.4±0.4	96.6±0.5	96.6±0.5	93.9±0.4	93.6±0.3	91.2±0.8	92.1±0.6
	2	91.2±0.8	94.0±0.4	91.9±0.6	92.5±0.7	90.0±1.4	90.5±1.0	90.0±0.7	90.4±0.8
	3	93.0±0.9	94.8±0.4	93.2±0.4	<u>93.9±0.2</u>	91.4±0.2	91.7±0.4	90.8±0.6	91.6±0.7
	4	95.5±0.4	96.5±0.1	95.6±0.4	<u>96.1±0.2</u>	91.8±0.4	90.6±0.2	92.9±0.2	92.2±0.3
	5	93.5±0.8	95.1±1.3	<u>93.6±0.3</u>	<u>93.6±0.3</u>	92.5±0.3	92.4±0.1	92.1±0.2	92.3±0.2
	6	<u>96.1±0.5</u>	96.4±0.4	95.7±0.2	95.5±0.3	92.5±0.3	92.6±0.4	93.3±0.7	93.0±0.8
	7	<u>96.0±0.4</u>	96.5±0.4	95.2±0.3	94.9±0.7	92.5±0.3	92.1±0.2	93.1±0.4	93.2±0.1
	8	97.7±0.7	97.5±0.5	95.9±0.8	97.0±0.3	92.8±0.9	92.3±1.0	93.1±1.5	93.3±1.3
	9	96.8±0.2	97.5±0.1	96.6±0.2	<u>97.2±0.2</u>	94.1±0.3	94.5±0.2	93.3±0.6	93.8±0.2
	<i>avg</i>	<u>95.4</u>	96.3	<u>95.1</u>	<u>95.4</u>	<u>92.0</u>	<u>91.9</u>	<u>91.8</u>	<u>92.0</u>

Table A.2: Semi-supervised setup: Average PR-AUC and StdDev values in % for torchvision dataset over 5 seeds. Values are over-inflated due to overrepresentation of positive (anomalous) samples.

B Results With More Labeled Anomalies

c_i	GRAPH AUTOENCODER				OCGTL				
	Patch		SLIC		Patch		SLIC		
	RAG	k -NN	RAG	k -NN	RAG	k -NN	RAG	k -NN	
MNIST	0	<u>99.3±0.2</u>	99.5±0.1	97.4±0.7	97.9±0.4	99.5±0.1	<u>99.3±0.1</u>	97.3±0.4	95.1±0.2
	1	<u>99.8±0.1</u>	99.9±0.0	99.2±0.1	99.3±0.2	<u>99.8±0.0</u>	<u>99.8±0.1</u>	99.2±0.1	99.3±0.1
	2	<u>96.5±0.5</u>	96.6±1.1	88.4±1.7	84.9±3.9	<u>93.4±0.4</u>	<u>94.8±1.5</u>	76.8±2.5	76.1±4.1
	3	97.3±0.4	<u>97.2±0.7</u>	95.0±0.7	91.7±0.5	94.2±1.3	96.1±0.5	84.1±2.0	79.2±4.1
	4	93.1±1.4	95.0±1.5	92.7±1.6	86.0±1.0	93.5±1.0	<u>94.9±1.2</u>	83.7±1.3	77.2±1.5
	5	96.7±0.7	95.5±1.5	90.2±1.7	87.1±2.1	91.4±1.0	90.8±0.6	81.2±2.4	79.8±0.9
	6	<u>98.3±0.4</u>	98.9±0.5	95.7±0.7	94.8±1.6	97.3±0.1	97.7±0.1	83.2±1.2	77.2±1.2
	7	96.3±0.5	98.4±0.7	95.3±0.8	92.5±1.7	96.4±0.2	<u>97.6±0.3</u>	88.9±2.1	86.0±2.0
	8	<u>96.6±1.3</u>	96.5±0.9	92.1±0.4	89.3±1.3	96.9±0.1	95.8±0.3	89.3±1.9	84.6±1.0
	9	96.3±0.3	95.3±2.7	92.1±1.3	88.7±1.4	<u>95.8±0.2</u>	95.4±0.3	89.6±0.8	82.1±1.4
<i>avg</i>	<u>97.0</u>	97.3	93.8	91.2	<u>95.8</u>	96.2	87.3	83.7	
Fashion-MNIST	0	97.8±1.5	98.8±0.1	94.8±1.0	98.1±0.4	98.8±0.2	<u>98.3±0.3</u>	95.9±0.7	95.5±0.9
	1	<u>92.4±1.7</u>	94.2±0.7	85.1±1.4	89.3±2.4	89.4±1.2	92.1±1.5	81.3±0.9	82.2±1.3
	2	91.5±3.5	93.8±0.9	87.2±0.8	90.2±0.7	<u>92.8±0.4</u>	92.7±0.5	84.4±1.4	86.7±0.4
	3	96.7±0.5	<u>96.6±0.4</u>	94.0±0.4	94.2±0.8	92.9±1.4	95.3±0.1	90.0±0.5	90.7±0.9
	4	<u>94.2±0.6</u>	94.4±0.3	90.9±0.3	90.7±0.7	91.9±0.5	92.1±0.3	88.9±1.1	89.4±1.2
	5	89.8±1.7	89.3±1.9	94.0±1.4	<u>92.4±1.5</u>	88.4±1.8	87.1±1.3	87.7±1.0	85.8±0.7
	6	<u>86.2±1.8</u>	87.5±0.6	81.3±0.2	82.8±1.1	76.9±7.9	77.8±3.0	76.8±1.9	78.1±1.3
	7	<u>98.2±0.3</u>	98.5±0.2	94.9±0.6	96.6±0.6	96.8±0.6	96.9±0.6	94.0±0.6	92.9±0.5
	8	<u>95.0±0.3</u>	96.0±0.6	92.6±0.9	<u>95.0±0.7</u>	94.1±0.5	94.4±0.6	85.6±0.9	90.8±0.8
	9	99.4±0.1	<u>99.2±0.1</u>	98.7±0.2	98.9±0.3	98.4±0.2	98.7±0.1	96.8±0.3	95.9±0.3
<i>avg</i>	<u>94.1</u>	94.8	91.4	92.8	92.0	92.5	88.1	88.8	
CIFAR-10	0	85.6±0.4	<u>85.2±0.6</u>	84.8±0.5	84.1±0.6	73.8±4.7	73.0±5.7	70.5±6.0	70.2±5.9
	1	85.9±1.0	<u>85.1±0.8</u>	82.4±0.8	82.3±1.1	73.6±1.3	71.0±0.9	71.4±1.2	67.7±1.9
	2	71.6±0.4	<u>71.3±1.2</u>	69.5±0.8	71.2±0.8	61.5±4.0	61.0±3.0	56.8±2.0	58.4±2.9
	3	72.1±1.6	<u>70.7±1.4</u>	68.0±1.9	69.4±1.6	62.9±1.6	60.9±1.3	62.1±1.1	60.5±1.6
	4	80.3±0.6	79.5±0.4	<u>79.8±0.6</u>	78.8±0.8	60.4±2.6	58.3±3.3	63.6±4.2	61.4±2.9
	5	<u>74.8±0.8</u>	76.2±1.2	68.7±1.4	69.1±2.6	61.9±1.9	60.8±0.9	60.7±2.3	59.6±1.9
	6	87.5±1.2	83.7±0.4	82.2±0.5	82.7±0.8	68.3±3.0	67.5±4.2	74.1±2.1	69.6±2.4
	7	<u>72.8±1.6</u>	77.2±1.6	68.0±0.7	70.4±1.0	63.3±1.7	62.3±1.8	63.6±1.4	63.3±0.7
	8	86.9±0.4	<u>86.8±0.7</u>	84.3±0.9	84.4±0.8	77.4±3.2	73.3±3.3	77.1±3.6	74.6±3.9
	9	84.5±0.9	85.3±0.5	81.1±0.3	83.2±0.8	73.7±1.3	74.6±0.9	74.2±3.2	72.7±2.1
<i>avg</i>	80.2	<u>80.1</u>	76.9	77.6	67.7	66.3	67.4	65.8	

Table B.1: Semi-supervised setup: Average ROC-AUC and StdDev values in % for torchvision datasets over 5 seeds with $1/3$ labeled anomalies during training. Compared to results in Tables 4.11, 4.13 and 4.15 we observe decreased performance gain for GAE, but further improvements for OCGTL.

GRAPH AUTOENCODER						OCGTL			
c_i	Patch		SLIC		Patch		SLIC		
	RAG	k -NN	RAG	k -NN	RAG	k -NN	RAG	k -NN	
MNIST	0	99.9±0.0	99.9±0.0	99.7±0.1	99.8±0.0	99.9±0.0	99.9±0.0	99.7±0.0	99.4±0.0
	1	100.0±0.0	100.0±0.0	99.9±0.0	99.9±0.0	100.0±0.0	100.0±0.0	99.9±0.0	99.9±0.0
	2	99.6±0.1	99.6±0.1	98.4±0.3	97.8±0.6	99.1±0.0	<u>99.3±0.2</u>	96.6±0.4	96.4±0.7
	3	99.7±0.0	99.7±0.1	99.4±0.1	99.0±0.1	99.3±0.2	<u>99.5±0.1</u>	97.8±0.3	97.1±0.6
	4	99.2±0.1	99.4±0.2	99.1±0.2	98.3±0.2	<u>99.3±0.1</u>	99.4±0.1	98.0±0.1	96.9±0.2
	5	99.7±0.1	<u>99.5±0.2</u>	98.9±0.2	98.5±0.3	99.0±0.1	98.9±0.1	97.6±0.3	97.5±0.1
	6	<u>99.8±0.0</u>	99.9±0.1	99.5±0.1	99.4±0.2	99.7±0.0	99.7±0.0	97.8±0.2	96.9±0.1
	7	99.5±0.1	99.8±0.1	99.4±0.1	99.0±0.2	99.5±0.0	<u>99.7±0.0</u>	98.5±0.3	98.1±0.3
	8	99.6±0.1	99.6±0.1	99.1±0.0	98.6±0.2	99.6±0.0	<u>99.5±0.0</u>	98.6±0.2	98.0±0.1
	9	99.5±0.0	<u>99.4±0.3</u>	99.0±0.2	98.5±0.2	99.5±0.0	<u>99.4±0.0</u>	98.6±0.1	97.5±0.2
<i>avg</i>	<u>99.6</u>	99.7	99.2	98.9	99.5	99.5	98.3	97.8	
Fashion-MNIST	0	99.7±0.2	99.9±0.0	99.4±0.1	<u>99.8±0.0</u>	99.9±0.0	<u>99.8±0.0</u>	99.6±0.1	99.5±0.1
	1	<u>99.1±0.2</u>	99.3±0.1	98.1±0.2	98.7±0.3	98.7±0.1	99.0±0.2	97.5±0.1	97.6±0.2
	2	99.0±0.4	99.3±0.1	98.4±0.1	98.8±0.1	<u>99.1±0.0</u>	<u>99.1±0.1</u>	98.0±0.2	98.4±0.0
	3	99.6±0.1	99.6±0.1	99.2±0.1	99.2±0.1	99.1±0.2	<u>99.3±0.0</u>	98.5±0.0	98.8±0.1
	4	99.3±0.1	99.3±0.0	98.9±0.0	98.9±0.1	99.0±0.1	<u>99.1±0.0</u>	98.7±0.1	98.8±0.1
	5	98.9±0.2	98.8±0.2	99.3±0.2	<u>99.2±0.2</u>	98.7±0.2	98.5±0.1	98.6±0.1	98.4±0.1
	6	<u>98.1±0.2</u>	98.4±0.1	97.5±0.0	97.8±0.1	96.7±1.1	96.9±0.5	96.9±0.3	97.1±0.2
	7	99.8±0.0	99.8±0.0	99.4±0.1	99.6±0.1	99.6±0.1	<u>99.7±0.1</u>	99.3±0.1	99.2±0.0
	8	<u>99.4±0.0</u>	99.5±0.1	99.1±0.1	<u>99.4±0.1</u>	99.2±0.1	99.3±0.1	98.0±0.1	98.8±0.1
	9	99.9±0.0	99.9±0.0	99.8±0.0	<u>99.8±0.1</u>	<u>99.8±0.0</u>	<u>99.8±0.0</u>	99.6±0.0	99.4±0.0
<i>avg</i>	<u>99.3</u>	99.4	98.9	99.1	99.0	99.1	98.5	98.6	
CIFAR-10	0	98.1±0.1	<u>97.9±0.0</u>	97.8±0.1	97.6±0.2	95.8±0.8	95.5±1.1	95.3±1.2	95.0±1.2
	1	98.1±0.2	<u>98.0±0.1</u>	97.6±0.1	97.6±0.2	96.1±0.3	95.7±0.1	95.7±0.2	95.1±0.3
	2	95.5±0.1	<u>95.4±0.3</u>	94.8±0.2	95.2±0.2	93.2±0.9	93.1±0.6	92.4±0.4	92.5±0.5
	3	95.7±0.4	<u>95.5±0.3</u>	94.8±0.4	95.1±0.3	93.7±0.3	93.3±0.2	93.6±0.2	93.3±0.4
	4	97.0±0.2	96.6±0.1	<u>96.8±0.1</u>	96.5±0.2	93.3±0.4	92.7±0.5	94.0±0.9	93.5±0.7
	5	<u>96.1±0.2</u>	96.4±0.2	94.9±0.2	95.0±0.5	93.7±0.3	93.4±0.3	93.5±0.6	93.2±0.4
	6	98.3±0.2	<u>97.8±0.0</u>	97.4±0.1	97.5±0.1	95.2±0.4	95.0±0.7	96.2±0.3	95.4±0.4
	7	<u>95.8±0.2</u>	96.5±0.3	95.2±0.1	95.5±0.2	94.0±0.3	93.8±0.3	94.3±0.3	94.2±0.1
	8	98.2±0.1	98.2±0.1	<u>97.8±0.1</u>	<u>97.8±0.1</u>	96.9±0.4	96.3±0.5	96.9±0.5	96.5±0.6
	9	<u>97.8±0.1</u>	98.0±0.1	97.3±0.1	97.6±0.1	96.2±0.2	<u>96.3±0.1</u>	<u>96.2±0.5</u>	<u>96.0±0.3</u>
<i>avg</i>	97.1	<u>97.0</u>	96.4	96.6	94.8	94.5	94.8	94.5	

Table B.2: Semi-supervised setup: Average PR-AUC and StdDev values in % for torchvision datasets over 5 seeds with $1/3$ labeled anomalies during training. Compared to results in Table A.2 we observe decreased performance gain for GAE, but further improvements for OCGTL.

Fixed	Modified	GRAPH AUTOENCODER				OCGTL			
		Patch		SLIC		Patch		SLIC	
		RAG	k -NN	RAG	k -NN	RAG	k -NN	RAG	k -NN
Mask ✗ Virtual nodes ✗ NNConv (GAE) GINConv (OCGTL)	RGB _{avg}	86.7±0.5	<u>86.1±0.2</u>	85.7±0.9	84.9±0.6	82.7±2.5	81.8±1.9	79.4±2.7	79.5±2.4
	Color	86.4±0.8	87.4±0.5	87.5±0.7	87.6±0.8	89.6±0.5	88.8±1.4	<u>89.1±0.7</u>	87.9±1.2
	Texture	82.4±2.9	84.8±2.1	82.7±1.5	84.1±2.2	<u>86.2±1.1</u>	85.5±1.4	86.3±0.7	85.1±1.3
	Shape	—	—	78.4±0.3	78.5±0.9	—	—	79.5±1.1	<u>78.9±1.0</u>
	All	86.2±0.9	86.6±1.0	82.9±2.2	84.0±1.0	91.5±0.7	90.2±0.8	<u>90.4±0.9</u>	88.9±1.0
	Best	86.7	87.4	87.5	87.6	91.5	90.2	<u>90.4</u>	88.9
Mask ✗ Virtual nodes ✗ Color (Features)	NNConv	86.5±0.8	<u>87.4±1.0</u>	<u>87.4±0.7</u>	87.9±0.7	—	—	—	—
	SplineConv	86.4±0.9	<u>87.6±1.1</u>	87.1±0.9	87.9±0.4	—	—	—	—
	GINConv	—	—	—	—	89.5±0.4	89.0±1.0	<u>89.1±0.8</u>	88.1±0.8
	GATConv	—	—	—	—	<u>66.6±1.7</u>	66.8±2.3	<u>66.0±2.6</u>	66.2±3.6
	Best	86.5	87.6	87.4	87.9	89.5	89.0	<u>89.1</u>	88.1
Mask ✓ Color (Features) NNConv (GAE) GINConv (OCGTL)	Virtual nodes ✗	85.5±1.9	86.6±1.3	89.7±0.6	90.4±0.7	87.0±1.0	86.0±0.5	<u>90.0±1.1</u>	<u>90.0±0.6</u>
	Virtual nodes ✓	84.4±2.9	86.5±1.9	86.6±0.2	86.5±0.4	87.1±0.8	85.9±0.7	<u>91.0±0.3</u>	91.4±0.3
	Best	85.5	86.6	89.7	90.4	87.1	86.0	<u>91.0</u>	91.4

Table B.3: Semi-supervised setup: Average ROC-AUC and StdDev values in % for the HAM10000 dataset on normal class “nv” over 5 seeds with $1/3$ labeled anomalies during training.

Fixed	Modified	GRAPH AUTOENCODER				OCGTL			
		Patch		SLIC		Patch		SLIC	
		RAG	k -NN	RAG	k -NN	RAG	k -NN	RAG	k -NN
Mask ✗ Virtual nodes ✗ NNConv (GAE) GINConv (OCGTL)	RGB _{avg}	74.7±1.1	73.8±0.9	<u>74.4±1.6</u>	71.8±1.0	68.6±2.8	66.3±2.8	67.0±2.2	65.8±2.5
	Color	77.4±1.0	78.3±0.5	<u>78.9±0.7</u>	79.4±1.0	81.7±0.9	<u>80.7±1.5</u>	80.3±0.7	78.8±1.4
	Texture	73.8±4.2	76.8±3.6	74.2±1.8	76.2±2.3	79.3±0.9	78.7±1.9	<u>79.2±0.8</u>	77.7±1.5
	Shape	—	—	65.3±0.6	65.1±1.2	—	—	67.4±0.9	<u>67.2±1.2</u>
	All	78.6±1.3	79.3±1.5	<u>72.4±4.7</u>	75.0±1.7	86.2±0.5	84.5±0.6	<u>84.7±0.6</u>	<u>83.4±1.1</u>
----- Best -----		<u>78.6</u>	<u>79.3</u>	<u>78.9</u>	<u>79.4</u>	86.2	84.5	<u>84.7</u>	<u>83.4</u>
Mask ✗ Virtual nodes ✗ Color (Features)	NNConv	77.7±0.8	78.4±0.8	<u>78.6±0.8</u>	79.7±0.8	—	—	—	—
	SplineConv	77.1±1.2	<u>78.8±1.5</u>	<u>78.2±1.1</u>	79.7±0.4	—	—	—	—
	GINConv	—	—	—	—	81.6±0.7	<u>80.9±1.3</u>	80.3±0.8	79.0±1.1
	GATConv	—	—	—	—	46.9±0.7	47.5±1.3	48.2±2.2	49.0±3.7
	Best	<u>77.7</u>	<u>78.8</u>	<u>78.6</u>	<u>79.7</u>	81.6	<u>80.9</u>	<u>80.3</u>	79.0
Mask ✓ Color (Features) NNConv (GAE) GINConv (OCGTL)	Virtual nodes ✗	76.7±1.9	78.2±1.2	81.4±0.7	82.1±0.9	77.6±0.8	76.6±0.7	83.0±0.8	82.9±0.7
	Virtual nodes ✓	75.6±2.9	78.1±1.9	77.8±0.3	77.3±0.7	77.9±0.9	76.5±0.6	<u>84.6±0.6</u>	85.0±0.7
	Best	<u>76.7</u>	<u>78.2</u>	<u>81.4</u>	<u>82.1</u>	<u>77.9</u>	<u>76.5</u>	<u>84.6</u>	85.0

Table B.4: Semi-supervised setup: Average PR-AUC and StdDev values in % for the HAM10000 dataset on normal class “nv” over 5 seeds with $1/3$ labeled anomalies during training. As more labeled data becomes available, a greater variety of features can be effectively associated with distinct patterns. Our results show that using all node features on patch-segmented RAG graphs yields the highest performance. It is likely that the optimal setup in this scenario would involve combining all node features with segmentation masks and virtual nodes in the OCGTL framework.

C Unsupervised Results Without Edge Loss

c_i	ROC-AUC				PR-AUC				
	Patch		SLIC		Patch		SLIC		
	RAG	k -NN	RAG	k -NN	RAG	k -NN	RAG	k -NN	
MNIST	0	41.7±6.7	69.4±7.0	39.1±7.1	41.5±14.5	88.5±2.0	95.3±1.2	88.5±1.8	88.0±4.3
	1	98.5±0.7	99.1±0.3	97.3±0.6	94.9±1.8	99.8±0.1	99.9±0.1	99.6±0.1	99.2±0.3
	2	38.7±2.4	50.4±6.1	39.5±4.6	41.2±4.9	87.8±1.0	90.9±1.4	87.5±1.3	88.3±1.2
	3	53.5±0.9	48.9±7.0	46.4±4.0	45.6±2.0	91.5±0.4	90.2±1.8	89.6±0.9	88.8±0.4
	4	58.6±3.4	68.5±2.4	57.0±3.3	54.9±2.5	93.2±0.9	95.2±0.6	92.8±0.9	92.4±0.6
	5	53.1±2.1	56.3±3.6	45.1±4.3	47.5±2.3	92.9±0.4	93.1±0.7	90.8±1.2	90.9±0.4
	6	49.0±9.2	52.7±8.7	45.9±3.5	52.0±3.3	90.7±2.6	92.1±1.6	89.8±0.9	91.1±0.9
	7	68.4±5.4	70.3±3.3	71.2±3.0	65.6±2.9	95.1±1.2	95.6±0.6	95.6±0.7	94.4±0.6
	8	44.8±4.6	55.6±14.8	38.9±2.5	45.6±5.0	89.5±0.9	91.8±3.7	87.9±0.9	89.0±1.3
	9	63.7±6.3	71.2±4.3	58.7±3.0	56.3±5.1	94.1±1.4	95.3±0.7	92.9±0.8	92.0±1.4
avg	57.0	64.2	53.9	54.5	92.3	93.9	91.5	91.4	
Fashion-MNIST	0	53.1±8.3	51.9±6.2	52.8±3.6	59.3±8.3	92.8±1.5	92.4±1.1	92.8±0.7	93.9±1.3
	1	41.9±8.0	37.7±6.6	39.9±3.8	39.3±3.3	88.4±2.5	86.5±2.5	88.2±1.3	87.4±0.8
	2	57.3±6.3	49.3±12.2	30.5±7.2	53.7±12.8	92.9±1.1	91.0±2.9	85.6±2.6	91.6±3.1
	3	70.0±7.1	79.4±5.0	67.8±6.1	65.7±7.6	95.7±1.1	97.2±0.7	95.0±0.9	94.6±1.6
	4	50.6±17.5	64.0±16.1	42.1±11.9	54.2±14.2	89.9±4.7	93.6±3.9	88.0±3.7	91.2±4.2
	5	80.4±2.7	87.2±2.6	88.7±2.1	85.5±4.2	97.3±0.4	98.4±0.4	98.6±0.3	98.1±0.6
	6	49.6±12.2	55.3±13.8	37.7±4.0	44.4±7.6	90.0±3.6	91.6±3.6	86.8±1.0	88.7±2.1
	7	84.4±1.5	92.2±3.6	87.7±3.3	86.1±2.8	98.1±0.2	99.1±0.4	98.5±0.4	98.3±0.4
	8	49.9±7.5	62.8±9.3	40.7±2.8	37.3±4.8	90.1±2.1	93.4±1.9	88.2±0.7	87.3±1.5
	9	86.9±6.4	89.7±4.4	77.2±10.8	80.8±6.4	98.3±0.8	98.6±0.5	96.9±1.6	97.5±0.8
avg	62.4	67.0	56.5	60.6	93.4	94.2	91.9	92.8	
CIFAR-10	0	59.2±1.3	59.3±2.3	58.9±2.1	57.2±6.8	92.4±0.1	92.4±0.3	92.3±0.3	92.0±1.2
	1	57.4±0.8	58.0±2.0	55.9±0.9	57.1±0.7	92.4±0.2	92.5±0.4	92.1±0.2	92.3±0.1
	2	49.7±1.0	51.1±1.3	50.4±1.4	51.3±1.1	89.8±0.3	90.3±0.8	89.8±0.5	90.3±0.7
	3	60.8±1.9	60.5±0.7	61.3±1.5	60.5±1.7	92.7±0.5	92.6±0.2	92.8±0.5	92.6±0.4
	4	44.0±1.7	50.1±7.9	48.3±1.2	49.7±5.5	88.3±0.5	90.0±2.2	89.7±0.6	90.0±1.8
	5	50.6±1.1	51.2±2.6	53.0±3.2	50.9±1.0	90.3±0.2	90.4±0.4	90.9±0.9	90.3±0.2
	6	47.7±1.3	50.3±3.2	53.0±4.4	52.5±2.6	89.3±0.4	89.9±0.9	91.0±1.3	90.9±0.6
	7	46.6±0.3	46.6±0.9	48.0±3.5	46.6±2.0	90.1±0.1	90.1±0.1	90.5±0.9	90.1±0.6
	8	59.9±3.8	60.5±3.6	62.1±1.7	61.8±1.9	93.3±0.6	93.4±0.6	93.7±0.3	93.7±0.3
	9	55.2±1.3	54.0±1.1	55.3±1.4	54.4±1.1	92.5±0.3	92.3±0.3	92.5±0.4	92.4±0.2
avg	53.1	54.2	54.6	54.2	91.1	91.4	91.5	91.5	

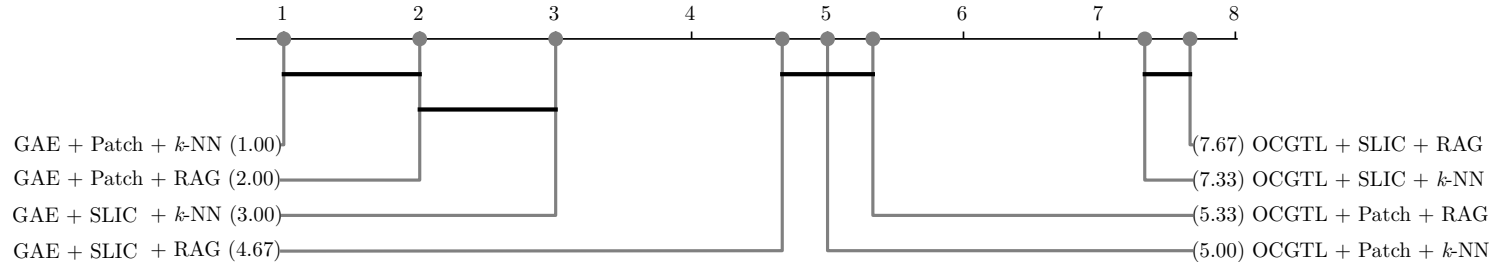
Table C.1: Average and StdDev AUC values in % for torchvision datasets on the GAE without edge attribute loss term ($\lambda_{\text{node}} = 1, \lambda_{\text{edge}} = 0$). The results stand in stark contrast to those presented in Tables 4.10, 4.12 and 4.14. Mainly, GAE without the edge attribute loss term showed a significant drop in performance compared to default GAE or even OCGTL. Furthermore, the results validate the assumption that augmenting OCGTL with graph convolutions such as NNConv offers limited performance gains over its existing configuration.

D Additional Figures

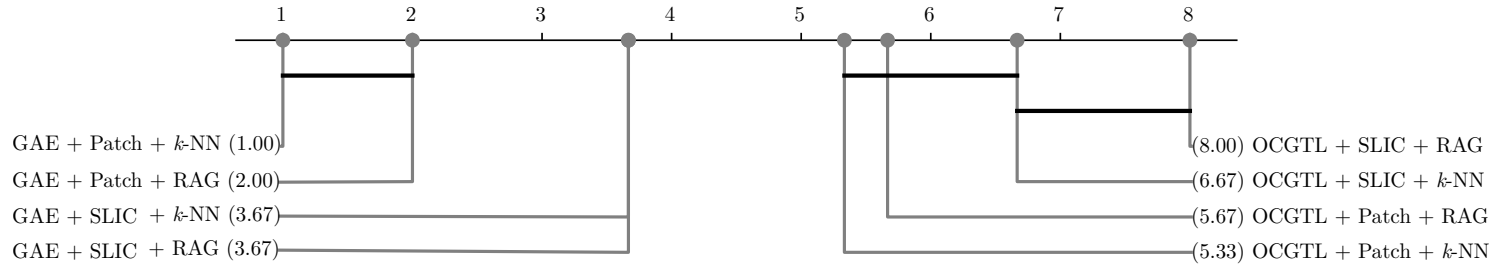
The evaluation of different configurations on torchvision datasets display a general trend visualized in Figure D.1 and D.2. The most significant influence on the performance of the tested configurations came from the chosen architecture for graph-based anomaly detection. Specifically, the Graph Autoencoder achieved significantly better performance in all cases compared to the alternative OCGTL model, highlighting the effectiveness of edge features for this task.

The second most influential factor in performance was the segmentation method used to process the image data. Here, patch segmentation asserted itself over SLIC segmentation. In most configurations, the patch-based method led to better results in the context of anomaly detection, suggesting its more suitable structuring of the input graph.

Finally, the edge determination strategy in the graph had the least but still noticeable effect. Here, the k-NN showed a slight but not statistically significant advantage. In particular, the higher number of edges compared to the RAG method likely allowed a better representation of image data as a graph and thus improved model performance.



(a) Unsupervised ranks of configurations on torchvision datasets



(b) Semi-supervised ranks of configurations on torchvision datasets

Figure D.1: Critical difference diagrams comparing the statistical differences of different configurations' ranks. Configurations linked with a horizontal crossbar are not deemed statistically different.

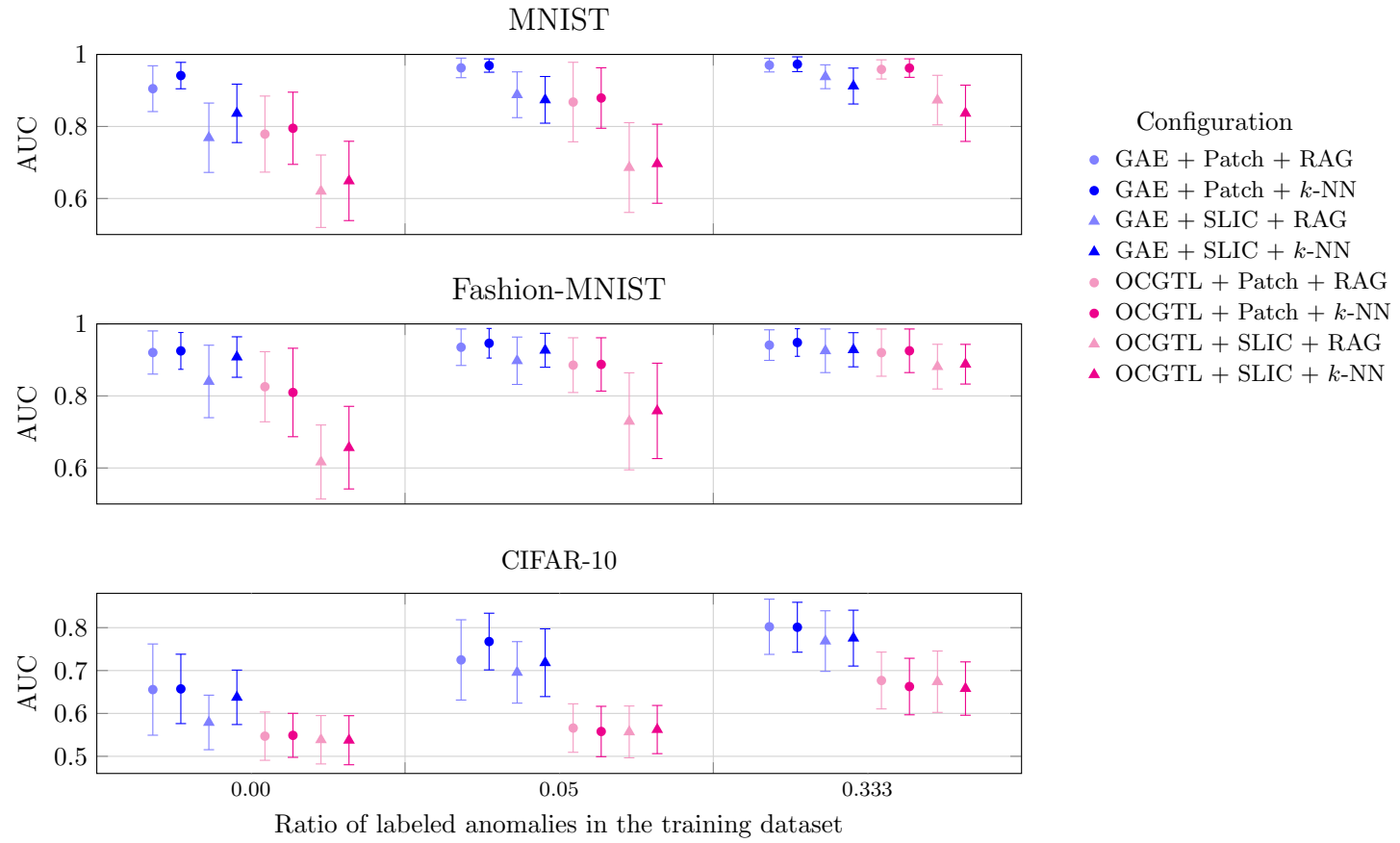


Figure D.2: Summary of average ROC-AUC and StdDev on torchvision datasets.

Bibliography

- [1] L. Zhao and L. Akoglu. “On Using Classification Datasets to Evaluate Graph-Level Outlier Detection: Peculiar Observations and New Insights.” arXiv: 2012.12931.
- [2] D. M. Hawkins, *Identification of Outliers*. Dordrecht: Springer Netherlands, 1980, ISBN: 978-94-015-3996-8 978-94-015-3994-4. DOI: 10.1007/978-94-015-3994-4.
- [3] T. Cover and P. Hart, “Nearest neighbor pattern classification,” *IEEE Trans. Inform. Theory*, vol. 13, no. 1, pp. 21–27, Jan. 1967, ISSN: 0018-9448, 1557-9654. DOI: 10.1109/TIT.1967.1053964.
- [4] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “LOF: Identifying density-based local outliers,” *SIGMOD Rec.*, vol. 29, no. 2, pp. 93–104, Jun. 2000, ISSN: 0163-5808. DOI: 10.1145/335191.335388.
- [5] L. Ruff *et al.*, “Deep One-Class Classification,” in *PMLR*, vol. 80, 2018, pp. 4393–4402. [Online]. Available: <https://proceedings.mlr.press/v80/ruff18a.html>.
- [6] L. Ruff *et al.* “Deep Semi-Supervised Anomaly Detection.” arXiv: 1906.02694.
- [7] M. Vazirgiannis, F. D. Malliaros, and G. Nikolentzos, “GraphRep: Boosting Text Mining, NLP and Information Retrieval with Graphs,” presented at the CIKM ’18: The 27th ACM International Conference on Information and Knowledge Management, Torino Italy: ACM, Oct. 17, 2018, pp. 2295–2296, ISBN: 978-1-4503-6014-2. DOI: 10.1145/3269206.3274273.
- [8] M. Gori, G. Monfardini, and F. Scarselli, “A new model for learning in graph domains,” presented at the International Joint Conference on Neural Networks 2005, vol. 2, Montreal, Que., Canada: IEEE, 2005, pp. 729–734, ISBN: 978-0-7803-9048-5. DOI: 10.1109/IJCNN.2005.1555942.
- [9] F. Scarselli, M. Gori, Ah Chung Tsoi, M. Hagenbuchner, and G. Monfardini, “The Graph Neural Network Model,” *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009, ISSN: 1045-9227, 1941-0093. DOI: 10.1109/TNN.2008.2005605.

-
- [10] X. Ma *et al.*, “A Comprehensive Survey on Graph Anomaly Detection With Deep Learning,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12012–12038, Dec. 1, 2023, ISSN: 1041-4347, 1558-2191, 2326-3865. DOI: 10.1109/TKDE.2021.3118815.
 - [11] H. Qiao, H. Tong, B. An, I. King, C. Aggarwal, and G. Pang. “Deep Graph Anomaly Detection: A Survey and New Perspectives.” arXiv: 2409.09957.
 - [12] L. Ruff *et al.*, “A Unifying Review of Deep and Shallow Anomaly Detection,” *Proc. IEEE*, vol. 109, no. 5, pp. 756–795, May 2021, ISSN: 0018-9219, 1558-2256. DOI: 10.1109/JPR0C.2021.3052449.
 - [13] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Ssstrunk, “SLIC Superpixels Compared to State-of-the-Art Superpixel Methods,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 11, pp. 2274–2282, Nov. 2012, ISSN: 0162-8828, 2160-9292. DOI: 10.1109/TPAMI.2012.120.
 - [14] P. Neubert and P. Protzel, “Superpixel Benchmark and Comparison,” in *Forum Bildverarbeitung*, vol. 6, 2012.
 - [15] P. H. C. Avelar, A. R. Tavares, T. L. T. da Silveira, C. R. Jung, and L. C. Lamb. “Superpixel Image Classification with Graph Attention Networks.” arXiv: 2002.05544.
 - [16] M. Fey, “Convolutional Neural Networks auf Graphreprsentationen von Bildern,” M.S. thesis, Technische Universitt Dortmund, Jul. 31, 2017. Accessed: Dec. 30, 2024. [Online]. Available: <https://github.com/rusty1s/deep-learning-on-graphs/tree/master/masterthesis>.
 - [17] L. V. Linh and C.-H. Youn, “Dynamic Graph Neural Network for Super-Pixel Image Classification,” presented at the 2021 International Conference on Information and Communication Technology Convergence (ICTC), IEEE, Oct. 20, 2021, pp. 1095–1099, ISBN: 978-1-6654-2383-0. DOI: 10.1109/ICTC52510.2021.9621101.
 - [18] R. A. Cosma, L. Knobel, P. Van Der Linden, D. M. Knigge, and E. J. Bekkers, “Geometric Superpixel Representations for Efficient Image Classification with Graph Neural Networks,” in *2023 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, Paris, France: IEEE, Oct. 2, 2023, pp. 109–118, ISBN: 979-8-3503-0744-3. DOI: 10.1109/ICCVW60793.2023.00018.
 - [19] C. Qiu, M. Kloft, S. Mandt, and M. Rudolph. “Raising the Bar in Graph-level Anomaly Detection.” arXiv: 2205.13845, Accessed: Nov. 5, 2024. [Online]. Available: <http://arxiv.org/abs/2205.13845>.
 - [20] R. J. Wilson, *Introduction to Graph Theory*, 4. ed. Harlow Munich: Prentice Hall, 2009, 171 pp., ISBN: 978-0-582-24993-6.

- [21] W. L. Hamilton, *Graph Representation Learning* (Synthesis Lectures on Artificial Intelligence and Machine Learning). Cham: Springer International Publishing, 2020, ISBN: 978-3-031-00460-5 978-3-031-01588-5. DOI: 10.1007/978-3-031-01588-5.
- [22] V.-Q. Loc, *Neuron and myelinated axon, with signal flow from inputs at dendrites to outputs at axon terminals*, 2012. [Online]. Available: <https://commons.wikimedia.org/wiki/File:Neuron3.png>.
- [23] C. C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*. Cham: Springer International Publishing, 2018, ISBN: 978-3-319-94462-3 978-3-319-94463-0. DOI: 10.1007/978-3-319-94463-0.
- [24] I. Basheer and M. Hajmeer, "Artificial neural networks: Fundamentals, computing, design, and application," *Journal of Microbiological Methods*, vol. 43, no. 1, pp. 3–31, Dec. 2000, ISSN: 01677012. DOI: 10.1016/S0167-7012(00)00201-3.
- [25] Osmose. "Neuron action potential - physiology. "[Online]. Available: <https://www.youtube.com/watch?v=BbUcWbtVjT4>.
- [26] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain.," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958, ISSN: 1939-1471, 0033-295X. DOI: 10.1037/h0042519.
- [27] V. Nair and G. E. Hinton, "Rectified Linear Units Improve Restricted Boltzmann Machines," presented at the 27th International Conference on Machine Learning, Haifa, Israel, Jun. 2010.
- [28] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, Jan. 1989, ISSN: 08936080. DOI: 10.1016/0893-6080(89)90020-8.
- [29] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural Networks*, vol. 4, no. 2, pp. 251–257, 1991, ISSN: 08936080. DOI: 10.1016/0893-6080(91)90009-T.
- [30] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986, ISSN: 0028-0836, 1476-4687. DOI: 10.1038/323533a0.
- [31] D. Masters and C. Luschi. "Revisiting Small Batch Training for Deep Neural Networks." arXiv: 1804.07612.
- [32] M. A. Nielsen, *Neural Networks and Deep Learning*. Determination Press, 2015.
- [33] P. Johnson, *MAS341: Graph Theory*, University of Sheffield. [Online]. Available: <https://ptwiddle.github.io/MAS341-Graph-Theory/Chapter1.pdf>.

-
- [34] J. A. Bondy and U. S. R. Murty, *Graph Theory with Applications*. New York, NY: North Holland, 1985, ISBN: 978-0-444-19451-0.
 - [35] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. “Neural Message Passing for Quantum Chemistry.” arXiv: 1704.01212.
 - [36] L. Wu, P. Cui, J. Pei, and L. Zhao, Eds., *Graph Neural Networks: Foundations, Frontiers, and Applications*. Singapore: Springer Nature Singapore, 2022, ISBN: 978-981-16-6053-5 978-981-16-6054-2. DOI: 10.1007/978-981-16-6054-2.
 - [37] K. Xu, W. Hu, J. Leskovec, and S. Jegelka. “How Powerful are Graph Neural Networks?” arXiv: 1810.00826.
 - [38] T. N. Kipf and M. Welling. “Semi-Supervised Classification with Graph Convolutional Networks.” arXiv: 1609.02907.
 - [39] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. “Graph Attention Networks.” arXiv: 1710.10903.
 - [40] B. Y. Weisfeiler and A. A. Leman, “The Reduction of a Graph to Canonical Form and the Algebra which appears therein,” *Nauchno-Technicheskaya Informatsia*, 2nd ser., pp. 12–16, 1968.
 - [41] D. E. Rumelhart and J. L. McClelland, “Learning internal representations by error propagation,” in *Parallel Distributed Processing: Explorations in the Microstructure of Cognition: Foundations*, 1987, pp. 318–362.
 - [42] D. H. Ballard, “Modular learning in neural networks,” in *Proceedings of the Sixth National Conference on Artificial Intelligence - Volume 1*, ser. AAAI’87, Seattle, Washington: AAAI Press, 1987, pp. 279–284, ISBN: 0-934613-42-7.
 - [43] G. Cottrell, P. Munro, and D. Zipser, “Learning internal representations from gray-scale images: An example of extensional programming,” Jan. 1987.
 - [44] M. A. Kramer, “Nonlinear principal component analysis using autoassociative neural networks,” *AIChE Journal*, vol. 37, no. 2, pp. 233–243, Feb. 1991, ISSN: 0001-1541, 1547-5905. DOI: 10.1002/aic.690370209.
 - [45] N. Japkowicz, C. Myers, and M. Gluck, “A novelty detection approach to classification,” in *Proceedings of the 14th International Joint Conference on Artificial Intelligence - Volume 1*, ser. IJCAI’95, Montreal, Quebec, Canada and San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1995, pp. 518–523, ISBN: 1-55860-363-8.
 - [46] D. M. Tax and R. P. Duin, “Support Vector Data Description,” *Machine Learning*, vol. 54, no. 1, pp. 45–66, Jan. 2004, ISSN: 0885-6125. DOI: 10.1023/B:MACH.0000008084.60811.49.

- [47] L. Akoglu, H. Tong, and D. Koutra, “Graph based anomaly detection and description: A survey,” *Data Min Knowl Disc*, vol. 29, no. 3, pp. 626–688, May 2015, ISSN: 1384-5810, 1573-756X. DOI: 10.1007/s10618-014-0365-y.
- [48] B. Schölkopf, R. C. Williamson, A. J. Smola, J. Shawe-Taylor, and J. C. Platt, “Support Vector Method for Novelty Detection,” in *Advances in Neural Information Processing Systems*, vol. 12, 1999.
- [49] R. M. Haralick, K. Shanmugam, and I. Dinstein, “Textural Features for Image Classification,” *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-3, no. 6, pp. 610–621, Nov. 1973, ISSN: 0018-9472, 2168-2909. DOI: 10.1109/TSMC.1973.4309314.
- [50] T. Ojala, M. Pietikainen, and T. Maenpaa, “Multiresolution gray-scale and rotation invariant texture classification with local binary patterns,” *IEEE Trans. Pattern Anal. Machine Intell.*, vol. 24, no. 7, pp. 971–987, Jul. 2002, ISSN: 0162-8828. DOI: 10.1109/TPAMI.2002.1017623.
- [51] Ming-Kuei Hu, “Visual pattern recognition by moment invariants,” *IEEE Trans. Inform. Theory*, vol. 8, no. 2, pp. 179–187, Feb. 1962, ISSN: 0018-9448. DOI: 10.1109/TIT.1962.1057692.
- [52] R. Bellman, “Dynamic Programming,” *Science*, vol. 153, no. 3731, pp. 34–37, Jul. 1966, ISSN: 0036-8075, 1095-9203. DOI: 10.1126/science.153.3731.34.
- [53] T. Strohmer, “Mathematical Algorithms for Artificial Intelligence and Big Data Analysis,” Lecture, Apr. 7, 2017. [Online]. Available: <https://www.math.ucdavis.edu/~strohmer/courses/180BigData/>.
- [54] I. T. Jolliffe, *Principal Component Analysis* (Springer Series in Statistics). New York: Springer-Verlag, 2002, ISBN: 978-0-387-95442-4. DOI: 10.1007/b98835.
- [55] T. Cai, S. Luo, K. Xu, D. He, T.-Y. Liu, and L. Wang. “GraphNorm: A Principled Approach to Accelerating Graph Neural Network Training.” arXiv: 2009.03294.
- [56] O. Atkinson, A. Bhardwaj, C. Englert, V. S. Ngairangbam, and M. Spannowsky, “Anomaly detection with convolutional Graph Neural Networks,” *J. High Energ. Phys.*, vol. 2021, no. 8, p. 80, Aug. 2021, ISSN: 1029-8479. DOI: 10.1007/JHEP08(2021)080.
- [57] M. Fey, J. E. Lenssen, F. Weichert, and H. Müller. “SplineCNN: Fast Geometric Deep Learning with Continuous B-Spline Kernels.” arXiv: 1711.08920.
- [58] L.-I. Gliga, J. Zegers, C. T. Gomez, and P. Bovijn, “Self, Semi and Fully Supervised Learning for Autoencoders using Ternary Classification,” in *2024 IEEE International Conference on Prognostics and Health Management (ICPHM)*, Spokane, WA, USA: IEEE, Jun. 17, 2024, pp. 33–40, ISBN: 979-8-3503-7447-6. DOI: 10.1109/ICPHM61352.2024.10627326.

-
- [59] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998, ISSN: 00189219. DOI: 10.1109/5.726791.
 - [60] F. Monti, D. Boscaini, J. Masci, E. Rodolà, J. Svoboda, and M. M. Bronstein. "Geometric deep learning on graphs and manifolds using mixture model CNNs." arXiv: 1611.08402.
 - [61] H. Xiao, K. Rasul, and R. Vollgraf. "Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms." arXiv: 1708.07747.
 - [62] A. Krizhevsky, "Learning multiple layers of features from tiny images," pp. 32–33, 2009. [Online]. Available: <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
 - [63] P. Tschandl, C. Rosendahl, and H. Kittler, "The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions," *Sci Data*, vol. 5, no. 1, p. 180161, Aug. 14, 2018, ISSN: 2052-4463. DOI: 10.1038/sdata.2018.161.
 - [64] P. Tschandl *et al.*, "Human–computer collaboration for skin cancer recognition," *Nat Med*, vol. 26, no. 8, pp. 1229–1234, Aug. 2020, ISSN: 1078-8956, 1546-170X. DOI: 10.1038/s41591-020-0942-0.
 - [65] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2015. DOI: 10.48550/arXiv.1412.6980.
 - [66] I. Golan and R. El-Yaniv. "Deep Anomaly Detection Using Geometric Transformations." arXiv: 1805.10917.
 - [67] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD — A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Long Beach, CA, USA: IEEE, Jun. 2019, pp. 9584–9592, ISBN: 978-1-7281-3293-8. DOI: 10.1109/CVPR.2019.00982.
 - [68] P. Liznerski, L. Ruff, R. A. Vandermeulen, B. J. Franks, M. Kloft, and K.-R. Müller. "Explainable Deep One-Class Classification." arXiv: 2007.01760.
 - [69] X. Tao, X. Gong, X. Zhang, S. Yan, and C. Adak, "Deep Learning for Unsupervised Anomaly Localization in Industrial Images: A Survey," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–21, 2022, ISSN: 0018-9456, 1557-9662. DOI: 10.1109/TIM.2022.3196436. arXiv: 2207.10298.
 - [70] S. G. Finlayson, J. D. Bowers, J. Ito, J. L. Zittrain, A. L. Beam, and I. S. Kohane, "Adversarial attacks on medical machine learning," *Science*, vol. 363, no. 6433, pp. 1287–1289, 2019. DOI: 10.1126/science.aaw4399.

- [71] C. Szegedy *et al.* “Intriguing properties of neural networks.” arXiv: 1312.6199.
- [72] H.-P. Kriegel, P. Kroger, E. Schubert, and A. Zimek, “Interpreting and Unifying Outlier Scores,” in *Proceedings of the 2011 SIAM International Conference on Data Mining*, Society for Industrial and Applied Mathematics, Apr. 28, 2011, pp. 13–24, ISBN: 978-0-89871-992-5 978-1-61197-281-8. DOI: 10.1137/1.9781611972818.2.

Eidesstattliche Versicherung

(Affidavit)

Xu, Dehn

183919

Name, Vorname
(surname, first name)

Matrikelnummer
(student ID number)

☒ Bachelorarbeit
(Bachelor's thesis)

☐ Masterarbeit
(Master's thesis)

Titel
(Title)

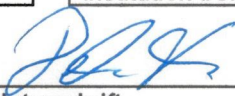
Deep Graph-Based Anomaly Detection on Image-Derived Representations

Ich versichere hiermit an Eides statt, dass ich die vorliegende Abschlussarbeit mit dem oben genannten Titel selbstständig und ohne unzulässige fremde Hilfe erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

I declare in lieu of oath that I have completed the present thesis with the above-mentioned title independently and without any unauthorized assistance. I have not used any other sources or aids than the ones listed and have documented quotations and paraphrases as such. The thesis in its current or similar version has not been submitted to an auditing institution before.

Dortmund, 12.05.2025

Ort, Datum
(place, date)


Unterschrift
(signature)

Belehrung:

Wer vorsätzlich gegen eine die Täuschung über Prüfungsleistungen betreffende Regelung einer Hochschulprüfungsordnung verstößt, handelt ordnungswidrig. Die Ordnungswidrigkeit kann mit einer Geldbuße von bis zu 50.000,00 € geahndet werden. Zuständige Verwaltungsbehörde für die Verfolgung und Ahndung von Ordnungswidrigkeiten ist der Kanzler/die Kanzlerin der Technischen Universität Dortmund. Im Falle eines mehrfachen oder sonstigen schwerwiegenden Täuschungsversuches kann der Prüfling zudem exmatrikuliert werden. (§ 63 Abs. 5 Hochschulgesetz - HG -).

Die Abgabe einer falschen Versicherung an Eides statt wird mit Freiheitsstrafe bis zu 3 Jahren oder mit Geldstrafe bestraft.

Die Technische Universität Dortmund wird ggf. elektronische Vergleichswerkzeuge (wie z.B. die Software „turnitin“) zur Überprüfung von Ordnungswidrigkeiten in Prüfungsverfahren nutzen.

Die oben stehende Belehrung habe ich zur Kenntnis genommen:

Official notification:

Any person who intentionally breaches any regulation of university examination regulations relating to deception in examination performance is acting improperly. This offense can be punished with a fine of up to EUR 50,000.00. The competent administrative authority for the pursuit and prosecution of offenses of this type is the Chancellor of TU Dortmund University. In the case of multiple or other serious attempts at deception, the examinee can also be unenrolled, Section 63 (5) North Rhine-Westphalia Higher Education Act (*Hochschulgesetz, HG*).

The submission of a false affidavit will be punished with a prison sentence of up to three years or a fine.

As may be necessary, TU Dortmund University will make use of electronic plagiarism-prevention tools (e.g. the "turnitin" service) in order to monitor violations during the examination procedures.

I have taken note of the above official notification:*

Dortmund, 12.05.2025

Ort, Datum
(place, date)


Unterschrift
(signature)

*Please be aware that solely the German version of the affidavit ("Eidesstattliche Versicherung") for the Bachelor's/ Master's thesis is the official and legally binding version.